enova 365

Poznaj oprogramowanie ERP,
które pracuje na Twój sukces
przez 365 dni w roku!

**Systemowy rozwój
Twojej firmy.**

API - Konstrukcje zaawansowane

Spis treści

API - Konstrukcje zaawansowane 3
- Generowanie własnych tabel 4
  - Struktura plików business.xml 5
  - Struktura plików convert.sql 14
  - Plik *.convert.sql 15
  - Rozszerzanie tabeli 18
  - Plik *.dbinit.xml 23
- Wydruki 29
  - Użycie drukarki pdf w swoim zewnętrznym dodatku. 30
  - Użycie generatora wydruków - klasy AspGenerator. 32
  - Generowanie i podpisywanie załączników - Report Result 36
  - Task Globalny - ReportResult 43
  - Mnożenie Kolumny przez parametr w ASPX 50
- Mechanizmy komunikacji z bazami danych 53
  - Session.Save w szczegółach 54
  - Mechanizm Query 62
  - Mechanizm Query dodatkowe możliwości 65
- Filtrowanie kolekcji za pomocą Condition 67
  - Equal, NotEqual 68
  - In 69
  - Like 70
  - Null 71
  - Less, Greater, LessEqual, GreaterEqual 72
  - Exists 73
- Narzędzia diagnostyczne 74
  - Debugowanie kodu kalkulatorów 75
  - Logi systemowe oraz progress 76
  - WinTest 80
    - Biblioteka FormTest 81
    - Przykładowe Testy 92
  - Scripter HTML 100
- Inicjalizacja programu i modułu oraz klasy Schema 105

API - Konstrukcje zaawansowane

W skład grupy *Konstrukcje zaawansowane* wchodzą wszystkie artykuły zawierające wiedzę z programowania zaawansowanego API Soneta. W tym również rozwiązanie problemów typu *studium przypadku*.

Generowanie własnych tabel

W skład grupy *Generowanie własnych tabel* wchodzą wszystkie artykuły zawierające wiedzę z tworzenia własnych tabel z użyciem logiki firmowej.

Struktura plików business.xml

W tym rozdziale opisana zostały wszystkie elementy i atrybuty, które mogą być użyte do opisu obiektów biznesowych.

Element główny *module*

Główny element pliku XML zawierający całą strukturę definiowanego przez ten element modułu. Wszystkie elementy i atrybuty zostały zadeklarowane w xml-namespace http://www.enova.pl/schema/business_struct.xsd, dlatego konieczne jest użycie atrybutu xmlns, który określa ten właśnie namespace.

Atrybut *name* Ten atrybut jest wymagany. Określa nazwę definiowanego modułu. Nazwa ta wykorzystywana jest między innymi do utworzenia odpowiednich nazw dla generowanych klas C#. Uwaga, gdy dwa wczytane pliki *.dll (i posiadające własne tabele) będą posiadać takie same wartości wewnątrz atrybutu name to: zostanie wygenerowany błąd podczas uruchomienia *programu.*

Atrybut *namespace* Ten atrybut jest wymagany. Określa nazwę namespace, w którym będą znajdować się wszystkie obiekty biznesowe zdefiniowane w tym module.

Atrybut *versionName* Ten atrybut jest wymagany. Określa nazwę numerów wersji (wersjonowania) pozwalających na automatyczne generowanie konwersji bazy danych z baz utworzonych starszymi wersjami bibliotek. Nazwane numery wersji pozwalają na niezależne prowadzenie projektów i wersjonowanie poszczególnych modułów. Standardowo nazwy wersji to **system, soneta** oraz **enova** użyte są przez program, więc podanie tych nazw spowoduje generację błędu.

Atrybut *versionNumber* Określa numer wersji o danej nazwie. Liczba ta powinna być zwiększana zawsze, gdy dokonujemy zmian w module, który został opublikowany publicznie i konieczna będzie konwersja. Taka sama wartość atrybutu versionName może być przypisany wielu modułom. Ale wystarczy, że tylko jeden z nich będzie określał numer wersji atrybutem versionNumber.

Element *module/import*

Wartość tekstowa znajdująca się w tym elemencie określa ścieżkę dostępu do katalogu, w którym poszukiwane są pozostałe pliki *.business.xml wchodzące w skład projektu. Wczytywane są wszystkie pliki *.business.xml znajdujące się we wskazanym katalogu oraz pliki, które znajdują się w podkatalogach.

Elementów import może występować w projekcie wiele.

Element *module/using*

Określa nazwy namespace bibliotek, których identyfikatory mogą być użyte w deklaracjach obiektów biznesowych. Użycie tych elementów powoduje, że na początku generowanego pliku źródłowego business.cs zostaną umieszczone odpowiednie instrukcje using.

Element *module/simpletype*

Deklaracja typów prostych (jak int, string, date, FromTo, itp.), które mogą być używane w kolumnach obiektów biznesowych. Element zawiera wszelkie informacje o sposobie wykorzystania danego typu przez obiekty biznesowe.

Atrybut *name* Ten atrybut jest wymagany. Nazwa tworzonego typu prostego identyfikująca dany typ w plikach business.xml. Do wykorzystania w atrybucie .../row/@name.

Element *module/simpletype/cstype*

Ten atrybut jest wymagany. Nazwa typu C# reprezentująca dany typ prosty w kodzie.

Element *module/simpletype/cscreator*

Kod programu C#, którego zadaniem jest zainicjowanie wartości pola na podstawie wartości znajdujących się w obiekcie creator typu Soneta.Business.RowCreator zawierający aktualnie wczytywany rekord bazy danych. W kodzie można używać znaków {0} na określanie indeksu wartości we wczytanym rekordzie.

Element *module/simpletype/csinitializer*

Kod programu C#, którego zadaniem jest zainicjować wartość pola standardową pustą (zerową) wartością.

Element *module/simpletype/nullcondition*

Ten atrybut jest wymagany. Warunek C# sprawdzający, czy dana wartość jest pusta (zerowa). Wartość danego pola reprezentowana jest przez identyfikator value.

Element *module/simpletype/null*

Wyrażanie C# zawierające pustą wartość.

Element *module/simpletype/attribute*

Element może występować wielokrotnie. Element zawierający deklarację atrybutu umieszczanego przed każdym property w obiekcie biznesowym danego typu.

Element *module/simpletype/dbfield*

W elemencie simpletype musi występować jeden albo więcej elementów dbfield. Definiuje reprezentację danej wartości w bazie danych. Reprezentacja pola w bazie zdefiniowana jest przez atrybuty elementu.

Atrybut *type* Ten atrybut jest wymagany. Określa typ pola bazy danych obsługiwany przez bibliotekę Soneta.

Atrybut *postfix* Postfiks jest dodawany na końcu nazwy pola w bazie danych. Wykorzystywany jest w przypadku, gdy wartość danego property biznesowego zapisywana jest w bazie w postaci więcej niż jednego pól.

Element *module/subrow*

Element deklaruje grupę pól, która może być wykorzystywana, w wielokrotnie w różnych obiektach biznesowych. Pola te tworzą niezależny obiekt biznesowy, a ich umieszczenie w deklaracji danej tabeli powoduje dodanie do danej tabeli wszystkich tych pól. W przypadku użycia w jednej tabeli wiele razy tej samej grupy pól „pakiety" pól dokładane są wielokrotnie. Przykładem takiej subrow może być adres.

Generator utworzy klasę bazową XxxModule.NameRow. Programista w kodzie programu musi zadeklarować klasę Name, która będzie dziedziczyła z klasy bazowej utworzonej przez generator. W klasie tej będzie znajdować jest logika biznesowa tworzona przez programistę.

Atrybut *name* Ten atrybut jest wymagany. Nazwa tworzonego typu identyfikująca dany typ w plikach business.xml. Do wykorzystania w atrybucie .../row/@name.

Element *module/subrow/key*

Element może występować wielokrotnie. Element deklarujący klucz tworzący w możliwość sortowania danych według wskazanego klucza. Deklaracja opisana poniżej '''Element .../key'''.

Element *module/subrow/col*

Element występuje co najmniej raz. Element deklarujący pole obiektu biznesowego tworzącego daną grupę pól. Deklaracja opisana poniżej '''Element .../col'''.

Element *module/enum*

Deklaruje typ wyliczeniowy, który w bazie danych będzie reprezentowany przez pole zadeklarowane przy pomocy simpletype o nazwie enum. W bazie danych zostanie utworzone pole typu integer, natomiast obiekt biznesowy będzie używał odpowiedniego typu wyliczeniowego o tej samej nazwie.

Konieczne jest zdefiniowanie wskazanego typu wyliczeniowego w kodzie programu.

Atrybut *name* Ten atrybut jest wymagany. Nazwa tworzonego typu identyfikująca dany typ w plikach business.xml. Do wykorzystania w atrybucie .../row/@name.

Element *module/interface*

Deklaruje relację interface'ową. Działanie relacji polega na tym, że dane pole może wskazywać na jeden z obiektów biznesowych, które implementują daną relację. Relacja ta reprezentowana jest przez dwa pola: nazwa tabeli, w której znajduje się obiekt oraz pola ID wskazujący na odpowiedni rekord w tej tabeli.

Tworzone pola w bazie danych opisane są za pomocą deklaracji simpletype o nazwie interfaceGroup.

Biblioteka Soneta automatycznie generuje odpowiedni kod SQL (triggery), który będzie utrzymywać spójność bazy danych (referential integrity).

Do poprawnego działania relacji interface'owej konieczne jest zadeklarowanie w kodzie programu odpowiedniego interface, który dziedziczy z IRow. Wskazanie, że dany obiekt biznesowy implementuje daną relację następuje przez dopisanie identyfikatora relacji w deklaracji

programowanego obiektu oraz przez umieszczenie odpowiedniego elementu interface w elemencie table.

Atrybut *name* Ten atrybut jest wymagany. Nazwa tworzonego typu identyfikująca dany typ w plikach business.xml. Do wykorzystania w atrybucie .../row/@name.

UWAGA! Należy zwrócić szczególną uwagę na aspekt wydajnościowy tego typu relacji, ponieważ kasowanie obiektu nadrzędnego nie jest wykonywane klasycznie przez silnik bazy danych za pomocą kasowania kaskadowego. Jest tutaj użyty trigger, który jest wykonywany na każdej operacji usuwania z tabeli nadrzędnej. Trigger ten przeszukuje wszystkie relacje interfejsowe tego obiektu, czyli na każdej tablicy, która ma pole z relacją interfejsową jest wykonywany dodatkowe zapytanie DELETE. Stąd usuwanie takiego rekordu będzie dłuższe niż przy klasycznej relacji. **Zalecamy ograniczać stosowanie takiej relacji do obiektów, które są rzadko kasowane.**

Element *module/table*

Deklaruje tabelę bazy danych oraz obiekt biznesowy.

W wyniku umieszczenia danej deklaracji generowane są następujące klasy XxxModule.NameRow oraz XxxModule.TableNameTable. Są to bazowe implementacje biznesowych obiektów wiersza oraz tabeli. Zadaniem programisty przez przygotowanie odpowiednio klas o nazwach Name oraz TableName, które będą dziedziczyć po odpowiednich klasach bazowych oraz będą implementować logikę biznesową.

Biblioteka Soneta automatycznie generuje odpowiedni kod tworzący, konwertujący i zarządzający tabelą bazy danych.

Atrybut *name* Ten atrybut jest wymagany. Nazwa tworzonego typu identyfikująca dany typ w plikach business.xml. Do wykorzystania w atrybucie .../row/@name. Jest to również nazwa klasy reprezentującej pojedynczy obiekt biznesowy.

Atrybut *tablename* Ten atrybut jest wymagany. Nazwa obiektu biznesowego (**UWAGA: nazwa jest ograniczona do 16 znaków, aby możliwe było rejestrowanie zapisów w tabeli ChanegeInfos**) zawierające kolekcję wszystkich obiektów danego typu. Obiekt ten reprezentuje tabelę.

Atrybut *warnings* Atrybut przyjmuje wartości On lub Off. Standardową wartością, gdy atrybut nie jest wyszczególniony jest On. Podczas generowania kodu wykrywane są ustawienia „podejrzane", które powinny być skorygowane, jednak nie są błędami. Ustawienie wartości tego atrybutu na Off powoduje, że takie sytuacje nie będą zgłaszane.

Atrybut *name8* Skrócona, ośmioznakową nazwa obiektu biznesowego. W przypadku, gdy atrybut nie jest wyszczególniony przyjmowana jest ograniczona do 8 znaków nazwa obiektu name.

Atrybut *guided* Określa użycie obiektu w strukturze datapack, pozwalającej na eksport i import danych przy pomocy formaty XML. Dopuszczalne wartości to:

- Child – obiekt jest podrzędny w stosunku do innego obiektu głównego. Eksport jest możliwy tylko z tym nadrzędnym obiektem. Zmiany dokonane w tym obiekcie wymagają jednoczesnego zablokowania obiektu nadrzędnego. To jest standardowa wartość atrybutu.
- Root – główny element datapack'a. Obiekt ten dostaje dodatkowe pole Guid, które jest identyfikatorem globalnym (w skali świata) danego obiektu i będzie wykorzystywane do synchronizacji. Dla tych obiektów przechowywana jest również historia zmian obiektu.
- Exported – jest to obiekt taki jak Root uzupełniony o pole Exported, które przechowuje informację, czy dany obiekt biznesowy został wysłany i nie należy go już modyfikować.
- None – obiekty nie uwzględniane w mechanizmie datapack.

Atrybut *config* Atrybut przyjmuje wartości true i false. Standardową wartością atrybutu jest false. Określa, że obiekt jest obiektem konfiguracyjnym, którego wartości nie mogą być zmieniane w

trakcie normalnej pracy programu. Jest to tylko możliwe podczas, gdy program jest w specjalnym trybie modyfikacji konfiguracji. Obiekty konfiguracyjne są również cache-owane. Czyli wczytywane są z bazy danych tylko raz, chyba, że ustawienia konfiguracyjne zostaną zmienione.

Atrybut *caption* Czytelna dla użytkownika nazwa typu obiektu biznesowego. W przypadku, gdy atrybut nie jest podany jako nazwę obiektu biznesowego brany jest atrybut Name.

Atrybut *tablecaption* Czytelna dla użytkownika nazwa typu kolekcji obiektów biznesowych. W przypadku, gdy atrybut nie jest podany jako nazwę obiektu biznesowego brany jest atrybut TableName.

Atrybut *cached* Atrybut przyjmuje wartości true i false. Standardową wartością atrybutu jest false. Określa, czy obiekty mają być cache-owane. Polega to na jednokrotnym wczytaniu obiektu z bazy danych. W podobny sposób cache-owane są obiekty konfiguracyjne (atrybut config). Atrybut może się odnosić do obiektów niekonfiguracyjnych.

Atrybut *timestamp* Atrybut przyjmuje wartości true i false. Standardową wartością atrybutu jest true. Określa sposób lock-owania danych podczas edycji wartości pól. Mechanizm timestamp polega na dodaniu do rekordu bazy danych wartości int, która przy każdej zmianie rekordu jest zmieniana. Podczas zapisywania danych do bazy sprawdzane jest, czy wartość pola w bazie nie uległa zmianie. Gdy wartość pola uległa zmianie zgłaszany jest błąd.

Element *module/table/interface*

Element może występować wielokrotnie. Określa, jakie relacje interface-owe wskazują na dany obiekt biznesowy. Wyszczególnienie elementu nie zwalnia programisty od ręcznego zaimplementowania interface w obiekcie biznesowym. Wartość tekstowa elementu jest nazwą interface-u, która wyszczególniona w atrybucie name elementów interface (module/interface/@name).

Element *module/table/key*

Element może występować wielokrotnie. Element deklarujący klucz tworzący w możliwość sortowania danych znajdujących się w tabeli według wskazanego klucza. Deklaracja opisana poniżej '''Element .../key'''.

Element module/table/col

Element występuje co najmniej raz. Element deklarujący pole obiektu biznesowego tworzącego daną tabelę. Deklaracja opisana poniżej '''Element .../col'''.

Element .../key

Element deklaruje klucz sortujący dane w tabeli. Może występować w elementach table oraz subrow. Podanie klucza dodaje do obiektu kolekcji obiektów (tabeli) property zarządzającego danym kluczem, które pozwala na wyszukiwanie, sortowanie i przeglądanie obiektów wiersza.

Atrybut *name* Ten atrybut jest wymagany. Nazwa klucza używana jest również jako identyfikator property reprezentującego obiekt obsługujący dany klucz.

Atrybut *keyunique* Atrybut przyjmuje wartości true i false. Standardową wartością atrybutu jest false. Ustawienie atrybutu na wartość true wymusza unikalne wartości w kluczu. Oznacza to, że dwa różne obiekty biznesowe nie mogą zawierać tych samych wartości pól znajdujących się w kluczu. W przypadku, gdy klucz nie jest unikalny (wartość false) to pól kluczowych na końcu dokładane jest pole ID, dzięki czemu uzyskiwany jest brak unikalności klucza.

Atrybut *keyprimary* Atrybut przyjmuje wartości true i false. Standardową wartością atrybutu jest false. Określa, że dany klucz jest używany jako standardowy klucz sortowania tabeli, czyli gdy odwołujemy się do kolekcji obiektów używany będzie ten właśnie klucz. Tylko jeden klucz w tabeli może być ustawiony jako primary. Jeżeli żaden klucz nie będzie wskazany jako primary, zostanie użyty jeden, dowolny klucz.

Atrybut *keyclass* Atrybut, jeżeli występuje określa dodatkowe właściwości klucza. Po ustawieniu tego atrybutu konieczne jest wskazanie pola obsługującego daną funkcjonalność klucza przy użyciu atrybutu keyclasscol. Może przyjmować następujące wartości:

- Lp – klucz zawiera pole Lp, które automatycznie numeruje obiekty w relacji z obiektem nadrzędnym. Przykładem są pozycje faktury, które są numerowane. Pozwala to na zachowanie i kontrolę nad kolejnością rekordów.
- Date – klucz jest posortowany według pola daty, które nie mogą się powtarzać.
- FromTo – klucz zawiera obiekty biznesowe z określonymi okresami czasu, które nie mogą na siebie zachodzić.
- History – klucz zarządzający historią obiektów biznesowych. Pozawala to na stworzenie historii obiektów przez określenie okresu ważności tych obiektów. Zachowana jest przy tym ciągłość i zupełność czasu. Dla każdej wskazanej daty istnieje tylko jeden rekord historyczny.

Element .../key/keycol

Element występuje co najmniej raz. Element reprezentujący pole (property) wchodzące w skład klucza.

Atrybut *name* Ten atrybut jest wymagany. Nazwa pola w kluczu. Atrybut musi wskazywać na jedno z pól ../col/@name.

Element .../col

Element występuje co najmniej raz. Może występować w elementach table oraz subrow. Definiuje pole w bazie danych oraz property obiektu biznesowego.

Atrybut *name* Ten atrybut jest wymagany. Nazwa pola oraz property. Nazwa jest używana to tworzenia nazwy pola w bazie danych oraz nazwy property obiektu biznesowego.

Atrybut *type* Ten atrybut jest wymagany. Definiuje typ property. Zawiera jedną z nazw obiektów: table, subrow, interface (używanie interfejsu IRow jest zakazane, w celu użycia interfejsu IRow należy napisać własny interfejs, oraz odziedziczyć IRow), enum lub simpletype i odnosi się do atrybutu name tych elementów.

W przypadku, gdy typem pola jest relacja do obiektu biznesowego lub relacja interface-owa budowany jest odpowiedni klucz obsługujący daną relację. Do dodatkowego opisania działania klucza należy użyć innych atrybutów i elementów jak keyprimary, reldefault, relname, itp.

Atrybut *length* Określa ilość znaków pól typu string. Atrybut jest wymagany tylko, gdy typ pola określony jest jako string.

Atrybut *readonly* Atrybut przyjmuje wartości true, false lub set. Standardową wartością atrybutu jest false. Określa, czy property biznesowe ma posiadać accessor set pozwalający na ustawienie wartości. W przypadku, gdy atrybut jest true property można tylko odczytywać, natomiast utworzony zostanie konstruktor posiadający parametr inicjujący wartość tego property. Gdy atrybut przyjmuje wartość set property zawiera tylko accessor get, natomiast utworzona zostanie dodatkowa metoda protected void SetXxx(T value), przy pomocy, której można ustawić wartość property.

Atrybut *required* Atrybut przyjmuje wartości true, false lub noverified. Standardową wartością atrybutu jest false. Określa, czy property może przyjmować brak wartości. Przykładem może być pusty string „” lub 0. W przypadku, gdy atrybut jest true property nie może przyjmować zerowych wartości. Dodatkowo podczas dodawania nowego obiektu biznesowego dodawany jest weryfikator uniemożliwiający zapisanie obiektu do bazy bez uprzedniego wypełnienia wartości. Gdy wartością property jest noverified sprawdzane jest tylko ustawienie property bez dodawania inicjacji weryfikatorów podczas dodawania zapisu.

Atrybut *setonlynull* Atrybut przyjmuje wartości true lub false. Standardową wartością atrybutu jest false. Ustawienie atrybutu na wartość true oznacza, że można property można ustawić tylko, gdy wartość pola jest zerowa.

Atrybut *reldefault* Atrybut przyjmuje wartości true lub false. Standardową wartością atrybutu jest false. Atrybut jest używany, gdy typem pola (property) jest relacja do innego obiektu biznesowego lub relacja interface-owa. Ustawienie atrybutu oznacza, że ta relacja jest default-owa.

Atrybut *keyunique* Atrybut przyjmuje wartości true i false. Standardową wartością atrybutu jest false. Atrybut jest używany, gdy typem pola (property) jest relacja do innego obiektu biznesowego lub relacja interface-owa. Ustawienie atrybutu na wartość true wymusza unikalne wartości w kluczu. Oznacza to, że dwa różne obiekty biznesowe nie mogą zawierać tych samych wartości pól znajdujących się w kluczu. W przypadku, gdy klucz nie jest unikalny (wartość false) to pól kluczowych na końcu dokładane jest pole ID, dzięki czemu uzyskiwany jest brak unikalności klucza.

Atrybut *relname* Atrybut nie jest wymagany. Atrybut jest używany, gdy typem pola (property) jest relacja do innego obiektu biznesowego lub relacja interface-owa. Określa nazwę danej relacji, która będzie wykorzystywana podczas tworzenia komunikatów dotyczących tej relacji.

Atrybut *relguided* Atrybut przyjmuje wartości pustą, inner lub outer. Standardową wartością atrybutu jest pusta wartość. Atrybut jest używany, gdy typem pola (property) jest relacja do innego obiektu biznesowego lub relacja interface-owa. Atrybut opisuje sposób powiązania obiektów typu guided child z obiektem nadrzędnym.

Atrybut *delete* Atrybut przyjmuje wartości pustą lub cascade. Standardową wartością atrybutu jest pusta wartość. Atrybut jest używany, gdy typem pola (property) jest relacja do innego obiektu biznesowego lub relacja interface-owa. Przypisanie wartości cascade powoduje, że kasowanie obiektu nadrzędnego automatycznie kasuje wszystkie obiekty podrzędne.

Atrybut *category* Nazwa kategorii, do której przypisane jest property. Kategoria służy do grupowania wielu property przeznaczonych do tego samego celu.

Atrybut *modifier* Atrybut określający dostępność danego property biznesowego w kodzie. Znaczenie wartości jest takie jak odpowiednich słów kluczowych w języku C#. Można przypisywać wartości jak:

- public – to jest standardowa wartość,
- protected
- internal
- internal protected

Atrybut *selector* Określa pole najczęściej typu enum, które jest wykorzystywane do określenia przez bibliotekę typu klasy implementującej logikę biznesową. Dla jednego typu obiektów biznesowych można określić różne „podtypy” obiektów, które mogą różnić się w działaniu. Użycie podtypów umożliwia umieścić w jednej tabeli obiekty biznesowe różnych typów, ale o podobnym znaczeniu (działaniu). Wszystkie te obiekty będą oparte na tych samych polach bazodanowych obiektu bazowego. Przykładowy sposób dziedziczenia klas obiektów biznesowych pokazuje wykorzystanie narzędzia do zaimplementowania różnego typu składników:

```
PlaceModule.WypSkladnikRow
  WypSkladnik
    WypSkladnikGlowny
    WypSkladnikPomniejszenie
      WypSkladnikPomniejszenie.Pomn130
      WypSkladnikPomniejszenie.Proporcjonalne
      WypSkladnikPomniejszenie.Proporcjonalne1n
  WypSkladnikOdchylka
    WypSkladnikOdchylka.Plus
    WypSkladnikOdchylka.Minus
    WypSkladnikOdchylka.AkordMinus
```

Atrybut *children* Atrybut pozwala określić nazwę property w obiekcie biznesowym, do którego jest relacja zawierającego listę obiektów w relacji. Atrybut jest używany, gdy typem pola (property) jest relacja do innego obiektu biznesowego lub relacja interface-owa. Przykładowo, gdy pozycja dokumentu zawiera relację do dokumentu handlowego o nazwie Dokument, atrybut children może zawierać wartość Pozycje, które spowoduje, że w obiekcie dokumentu handlowego zostanie utworzone property o nazwie Pozycje zawierające listę pozycji podłączonych do dokumentu.

Atrybut *childrenclass* Atrybut określa typ klasy kolekcji zawierającej obiekty. Atrybut jest używany, gdy typem pola (property) jest relacja do innego obiektu biznesowego lub relacja interface-owa. Obiekt kolekcji określony jest przez atrybut children.

Atrybut *cstype* Jeżeli atrybut występuje określa typ property biznesowego. Pozwala to na określenie innego typu property niż to wynika z wartości atrybutu type.

Atrybut *relright* Atrybut przyjmuje wartości true i false. Standardową wartością atrybutu jest false. Atrybut jest używany, gdy typem pola (property) jest relacja do innego obiektu biznesowego lub relacja interface-owa. Określa, że dana relacja ma być wykorzystana do przekazywane prawa do obiektów podrzędnych. Przykładowo, pozycje dokumentu handlowego mają te same obiektowe prawa, co sam dokument handlowy. Natomiast dokument handlowy posiada prawa wynikające z jego przynależności do magazynu.

Atrybut *description* Atrybut zawierający opis działania property biznesowego.

Atrybut *caption* Czytelna nazwa property obiektu biznesowego. Standardowo nazwa wyliczana jest na podstawie nazwy property, np.: Dokument->Dokument, MagazynDocelowy->Magazyn docelowy. Natomiast, jeżeli czytelna nazwa nie wynika z nazwy property, należy posłużyć się atrybutem caption.

Atrybut *keyclass* Atrybut jest używany, gdy typem pola (property) jest relacja do innego obiektu biznesowego lub relacja interface-owa. Atrybut, jeżeli występuje określa dodatkowe właściwości klucza obsługującego relację. Po ustawieniu tego atrybutu konieczne jest wskazanie pola obsługującego daną funkcjonalność klucza przy użyciu atrybutu keyclasscol. Może przyjmować następujące wartości:

- Lp – klucz zawiera pole Lp, które automatycznie numeruje obiekty w relacji z obiektem nadrzędnym. Przykładem są pozycje faktury, które są numerowane. Pozwala to na zachowanie i kontrolę nad kolejnością rekordów.
- Date – klucz jest posortowany według pola daty, które nie mogą się powtarzać.
- FromTo – klucz zawiera obiekty biznesowe z określonymi okresami czasu, które nie mogą na siebie zachodzić.
- History – klucz zarządzający historią obiektów biznesowych. Pozawala to na stworzenie historii obiektów przez określenie okresu ważności tych obiektów. Zachowana jest przy tym ciągłość i zupełność czasu. Dla każdej wskazanej daty istnieje tylko jeden rekord historyczny.

Atrybut *keyclasscol* Atrybut jest używany, gdy typem pola (property) jest relacja do innego obiektu biznesowego lub relacja interface-owa. Atrybut występujący łącznie z atrybutem keyclass, który określa pole obiektu biznesowego implementujące zachowanie danego klucza relacji.

Atrybut *keyprimary* Atrybut przyjmuje wartości true i false. Standardową wartością atrybutu jest false. Atrybut jest używany, gdy typem pola (property) jest relacja do innego obiektu biznesowego lub relacja interface-owa. Określa, że dany klucz relacji jest używany jako standardowy klucz sortowania tabeli, czyli gdy odwołujemy się do kolekcji obiektów używany będzie ten właśnie klucz. Tylko jeden klucz w tabeli może być ustawiony jako primary. Jeżeli żaden klucz nie będzie wskazany jako primary, zostanie użyty jeden, dowolny klucz.

Atrybut *important* Atrybut przyjmuje wartości true i false. Standardową wartością atrybutu jest false. Określa, że dane property biznesowe należy traktować jako ważne. Zmiany wartości danego property powodują powstanie informacji ostrzegającej operatora o zmianie wartości. Obiekt biznesowy buduje dodatkowo prawo pozwalające na blokowanie edycji danego property.

Atrybut *specialaccess* Atrybut przyjmuje wartości true i false. Standardową wartością atrybutu jest false. Pozwala na umożliwienie dostępu do danego property biznesowego bez względu na system prawa bezpieczeństwa obiektu biznesowego.

Element *.../col/attribute*

Element może występować wielokrotnie. Wartość elementu zawiera deklarację atrybutu property biznesowego określającego jest działanie. Atrybut zostanie umieszczony w obiekcie biznesowym przed deklaracją property biznesowego w nawiasach [i].

Element *.../col/verifier*

Element może występować wielokrotnie. Określa weryfikatory, które mają być odpalone podczas dokonania zmian w danym property biznesowym.

Atrybut *name* Ten atrybut jest wymagany. Nazwa klasy C# weryfikatora implementującego dany weryfikator.

Atrybut *onadded* Atrybut przyjmuje wartości true i false. Standardową wartością atrybutu jest true. Określa, czy wskazany weryfikator ma być również dodawany podczas dodawania nowego zapisu.

Element *.../col/keycol*

Atrybut jest używany, gdy typem pola (property) jest relacja do innego obiektu biznesowego lub relacja interface-owa. W powyższym przypadku element występuje co najmniej raz. Element reprezentujący dodatkowe pole (property) wchodzące w skład klucza.

Atrybut *name* Ten atrybut jest wymagany. Nazwa pola w kluczu. Atrybut musi wskazywać na jedno z pól ../col/@name.

Struktura plików convert.sql

Plik zawiera instrukcje SQL, które mają być wykonane podczas robienia konwersji bazy danych. W pliku tym nie jest ważna jego nazwa. Ważne jest natomiast rozszerzenie pliku convert.sql, po którym biblioteka Soneta wyszukuje pliki konwersji SQL w zasobach systemowych.

Na początku tego pliku znajdują się informację dla mechanizmu konwersji np:

```
-- versionname:Punktacja version:2
```

W informacji wyszczególnione są atrybuty określające moment wywołania konwersji bazy danych. Mogą znajdować się następujące atrybuty:

versionname – nazwa numerów wersji bazy danych, która będzie sprawdzana. W tym przypadku jest to punktacja. Jest to ta sama nazwa, która znajduje się w pliku Punktacja.business.xml, w elemencie głównym module w atrybucie versionName.

version – numer wersji bazy danych, do która ma być konwertowana. Liczba 2 oznacza, że konwertowane będą bazy danych o numerze wersji mniejszej niż 2.

fromversion – określa explicite numer wersji bazy danych, dla której dana konwersja ma być wykonana. priority – liczba określająca kolejność wykonania konwersji w poszczególnych plikach, jeżeli jest ich więcej.

database – określenie typu bazy danych, dla którego dany plik ma być wykonany. Aktualnie możliwe są typy MSSQL i MySQL. Jeżeli typ jest nieokreślony, to instrukcje napisane są w sposób akceptowany przez obydwa typy bazy.

Instrukcje SQL wywołane są na bazie danych, do której zostały dodane wszystkie nowe pola bazy danych, ale nie zostały jeszcze skasowane stare. Możliwe jest zatem przeniesienie wartości pomiędzy kasowanymi i dodawanymi polami. Z bazy danych usunięte są również wszystkie indeksy oraz relacje pomiędzy tabelami. Należy o tym pamiętać podczas tworzenia skomplikowanych zapytań SQL. Z tego powodu mogą one być wykonywane niewspółmiernie długo.**

Plik *.convert.sql

Plik *.convert.sql określa reguły konwersji bazy danych. Nazwa pliku powinna zawierać numer wersji oraz rozszerzenie *.convert.sql. Zapytania zostaną wykonane tylko i wyłączne wtedy jeśli została zmieniona struktura bazy danych,sama zmiana "versionNumber" w pliku *.business.xml nie wymusza uruchomienia zapytań z pliku *.convert.sql.

Przykładowo:

110300.convert.sql

Oznacza to, że chcemy, aby konwersja danych wykonała się od wersji 11.3. Jest to konwersja SQL'owa w związku z tym należy stosować takie same konstrukcje jak w SQL Server Management Studio.

Przykładowo:

```
-- versionname:nazwa version:90100

UPDATE WFDefItems
SET IsVisibleInList = 1
WHERE IsVisibleInList IS NULL;
```

W tej konwersji następuje zmiana w tabeli *WFDefItems* (polecenie UPDATE WFDefItems) i dodanie kolumnie *IsVisibleInList* wartości 1 (SET IsVisibleInList = 1), jeśli *IsVisibleInList* jest null'em (WHERE IsVisibleInList IS NULL). Parametry używane w *.convert.sql zostały dokładnie opisane pod linkiem: https://dok.enova365.pl/Developers/21149.

Poniżej przedstawiam dwa przykłady, które pokazują działanie konwersji w enova365.

Do oby dwóch przypadków zostały wykorzystane poniższe kody:

```
-- versionname:nazwa version:120100

UPDATE BasicDocDefs
SET Name = 'WERSJA 120100'
WHERE ID=1
```

W tej konwersji odwołujemy się do tabeli BasicDocDefs (UPDATE BasicDocDefs), w której zmieniamy nazwę na 'WERSJA 120100' (SET Name = 'WERSJA 120100') dla pierwszego rekordu (WHERE ID=1). Cały kod zostanie zapisany w pliku o nazwie: 120100.convert.sql.

```
-- versionname:nazwa version:120300

UPDATE BasicDocDefs
SET Symbol = 'WERSJA 120300'
WHERE ID=1
```

W tej konwersji odwołujemy się do tabeli BasicDocDefs (UPDATE BasicDocDefs), w której zmieniamy symbol na 'WERSJA 120300' (SET Symbol = 'WERSJA 120300') dla pierwszego rekordu (WHERE ID=1). Cały kod zostanie zapisany w pliku o nazwie: 120300.convert.sql.

Przypadek I:

Aktualna wersja enova365: 12.3

Baza danych jest w wersji: 11.5

Pliki do konwersji w są wersji: 12.1, 12.3

Konwersja bazy będzie się odbywać z wersji 11.5 na 12.3.

Wynik wykonanej konwersji:

Konwersja została wykonana na plikach: 120100.convert.sql, 120300.convert.sql.

Przypadek II:

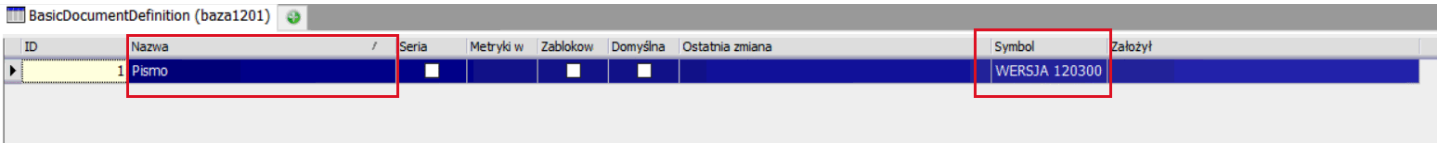
Aktualna wersja enova365: 12.3

Baza danych jest w wersji: 12.1

Pliki do konwersji w są wersji: 12.3

Konwersja bazy będzie się odbywać z wersji 12.1 na 12.3.

Wynik wykonanej konwersji:



Konwersja została wykonana na plikach: 120300.convert.sql.

WNIOSKI:

Pliki *.convert.sql są wykonywane raz na konkretnej bazie. Jeśli baza pochodzi z wersji 11.5 i ma być konwertowana na 12.3 zostaną uruchomione jedynie pliki, które zostały utworzone pomiędzy tymi wersjami, np. 12.1, 12.2, 12.3. Natomiast wcześniejsze pliki *.covert.sql (czyli np. z wersji 11.0), nie potrzebują już być wykonywane, ponieważ nowa baza zawiera już te zmiany.

Konwersja konkretnych baz danych

Pliki *.convert.sql mogą być również wykonywane tylko dla konkretnych baz, np. dla MySQL. Wtedy plik powienien mieć nazwę: 120100.MySQL.convert.sql (konwersja zostanie wykonana na enova365 w wersji 12.1 i bazie MySQL).

Przykładowy kod:

```
--databse:MySQL versionname:nazwa version:120100

UPDATE BasicDocDefs
```

```
SET Name = 'WERSJA 120100'
WHERE ID=1
```

W tej konwersji odwołujemy się do tabeli BasicDocDefs (UPDATE BasicDocDefs), w której zmieniamy nazwę na 'WERSJA 120100' (SET Name = 'WERSJA 120100') dla pierwszego rekordu (WHERE ID=1). Cały kod zostanie zapisany w pliku o nazwie: 120100.MySQL.convert.sql. Parametr database został ustawiony na MySQL, w związku z tym konwersja zostanie wykonana jedynie na bazach *MySQL*.

Rozszerzanie tabel

Można rozszerzyć tabele na dwa sposoby w zależności od tego czy można edytować istniejąca tabele lub nie. Jeżeli można ingerować w istniejące tabele to należy używać subrow, jeżeli nie to rozszerzamy tabele o kolejne pola za pomocą stworzenia tabeli rozszerzającej.

Subrow - jest to grupa pól, która w kodzie będzie miała reprezentacje w postaci samodzielnej klasy i co najistotniejsze – istnieje możliwość ponownego jej użycia w rożnych miejscach w kodzie. Spójrzmy na poniższy przykład:

```
<subrow name="Realizacja">
    <col name="Stan" type="StanRealizacji" caption="Stan realizacji" />
    <col name="Wynik" type="WynikRealizacji" caption="Wynik realizacji" readonly="set" />
</subrow>

<table name="DokumentHandlowy" tablename="DokHandlowe" guided="Exported"
     tablecaption="Dokumenty handlowe">
      <col name="CechaPodzialuKosztuDodatkowego" type="FeatureDefinition" category="Magazyn"
caption="Wartość podziało kosztu" description="Cecha licząca wartość do podziału kosztu."/>
      (...)
 <col name="Realizacja" type="Realizacja" description="Stan, wynik realizacji" />
 <col name="EDokumentInfo" type="EDokument" />
</table>
```

W powyższym przykładzie widać subrow „Realizacja” składający się z dwóch kolumn, które informują nas o stanie realizacji. Dzięki dodaniu kolumny o typie „Realizacja” do tabeli dokumentu handlowego uzyskamy wszystkie pola tego subrow bez konieczności ponownego ich definiowania w strukturze tabeli. Od strony bazodanowej będzie to tak wyglądać, ze do tabeli zostaną dodane dwie kolumny – dokładnie takie, jakie zostały zdefiniowane w subrow. Element typu subrow pozwala nam również na definiowanie wewnątrz niego kluczy tworzących możliwość sortowania danych wg wskazanego klucza jak i również pozwala na tworzenie relacji do innych tabel.

Tworzenie tabeli rozszerzającej

W sytuacji, gdy nie ma możliwości ingerowania w istniejąca tabele musimy w pliku business.xml zadeklarowac tabele rozszerzajaca, która bedzie w relacji 1:1 z tabela bazowa oraz bedzie posiadala dodatkowe, pożądane kolumny (dobrą praktyką bedzie umieszczenie dodatkowych kolumn w osobnym subrow, który będzie użyty w tabeli rozszerzającej). Spójrzmy na poniższy przykład:

```
<table name="Towar2" tablename="Towary2" name8="Towary2" caption="Towary - dodatkowe informacje">
    <col name="Host" type="Towary"
        required="true"
        keyunique="true"
        keyprimary="true"
        relname="Dodatkowe towarów"
        relguided="inner"
        delete="cascade"
```

```
        relright="true">
    </col>
(...)
</table>
```

Jest to tabela rozszerzająca tabele bazowa „towary” w relacji 1:1. Kluczem głównym i podstawowym jest kolumna o nazwie „Host”, czyli nie mogą znaleźć się w niej dwa identyczne towary. Jak widzimy – atrybut „delete” mówi nam o tym, że kasowanie obiektu nadrzędnego automatycznie skasuje rekord tabeli rozszerzającej. Trzeba pamiętać, ze przy dodawaniu rekordów w tabeli bazowej nie będą automatycznie dodawane wiersze tabeli rozszerzającej. W tym celu w klasie dziedziczącej po TowaryModule.Towar2Table należy stworzyć statyczny konstruktor, w którym obsłużymy zdarzenie uruchamiające się podczas dodawania rekordu do tabeli bazowej:

```
public class Towary2 : TowaryModule.Towary2Table
{
    static Towary2()
    {
        TowaryModule.TowarSchema.AddOnAdded(new Soneta.Business.RowDelegate<TowaryModule.TowarRow>
(Towar_Added));
    }
   static void Towar_Added(TowaryModule.TowarRow newRow)
   {
       Towar definicja = (Towar)newRow;
       Towar2 du = new Towar2(definicja);
       TowaryModule.GetInstance(newRow).Towar2.AddRow(du);
   }
}
```

Jak widać – przy dodawaniu rekordu nadrzędnego obsługujemy od razu dodawanie wpisu do tabeli rozszerzającej. Obiekt tabeli podstawowej przekazujemy w parametrze konstruktora obiektu tabeli rozszerzonej. Jest to skutek ustawienia atrybutu „required” dla kolumny „Host” w pliku business.xml. W powyższym przykładzie obsłużono dodawanie nowych rekordów do tabeli rozszerzającej. Jednak to nie wszystko – należy także pamiętać o sytuacji, gdy nie można usunąć wpisu z tabeli rozszerzającej, gdy rekord tabeli bazowej nie jest usuwany. Aby zabezpieczyć się przed usunięciem rekordu podrzędnego, gdy rekord nadrzędny nie jest usuwany, trzeba nadpisać metodę OnDeleting() znajdująca się w klasie Towar2 w następujący sposób:

```
public class Towar2: TowaryModule.Towar2Row
{
    (...)
    protected override void OnDeleting()
    {
        base.OnDeleting();
        RowStatus status = Host.Status;
        if ((status&RowStatus.Deleting)==0 && !Session.InImport)
        throw new RowException(this, "Dodatkowe informacje o towarze moga byc kasowane     tylko
razem z towarem glównym.");
    }
}
```

W powyższym przykładzie widać, ze przed usunięciem rekordu następuję sprawdzenie stanu rekordu nadrzędnego oraz sprawdzenie, czy sesja jest w trakcie importowania danych. Ostatnią sprawą, o której należy pamiętać, to konwersja bazy danych. Po dodaniu tabeli rozszerzającej, trzeba dodać do niej rekordy dla istniejących już rekordów tabeli bazowej. W zależności od tego jak dużo rekordów istnieje w tabeli bazowej można to zrobić poprzez plik convert.sql lub za pomocą klasy dziedziczącej po Business.App.IDatabaseInitializer. Jeżeli istnieje dużo rekordów – dobrą praktyką jest konwersja poprzez plik *.convert.sql ze względu na większa wydajność całego procesu.

```
insert into Towary2 (Host, ...)
        select ID, ... from Towary;
```

W powyższym przykładzie widać, że w tabeli rozszerzającej powstaną rekordy nawiązujące do rekordów w tabeli bazowej, co pozwoli na uzupełnienie kolumn rekordu podrzędnego pożądanymi danymi. Drugi sposób, który został wyżej wymieniony to stworzenie klasy DbInitializer<numer wersji=""> np.: Initializer5030, implementującą interfejs IDatabaseInitializer. Należy także pamiętać, aby przed deklaracją przestrzeni nazw w pliku umieścić atrybut Soneta.Business.App.DatabaseInit. Spójrzmy na poniższy kompletny przykład:</numer>

```
[assembly: Soneta.Business.App.DatabaseInit(
    "Konwersja danych dla Towary2(50300)",
    typeof(Soneta.Kadry.PracHistorie2.Initializer50300),
    "enova")]

namespace Soneta.Towary
{
    public class Towary2 : TowaryModule.Towary2Table
    {
        internal class Initializer50300 : Business.App.IDatabaseInitializer
        {
            void Business.App.IDatabaseInitializer.Initialize(Business.App.Login login, int version)
            {
                if (version == 0 || version >= 50300) return;
                using (var session = login.CreateSession())
                {
                    using (var transaction = session.Logout(true))
                    {
                        var towaryModule = TowaryModule.GetInstance(session);
                        var towary = towaryModule.Towary2.WgNazwa.ToList();
                        foreach (var towar in towary)
                        {
                            towaryModule.Towary2.AddRow(new Towar2(towar));
                        }
                        transaction.Commit();
                    }
                session.Save();
               }
            }
        }
    }
}
```

Jak widać – został odniesiony ten sam skutek jak przy tworzeniu pliku *.convert.sql. Dla wersji 50300 zostaną dodane rekordy do tabeli rozszerzającej, które w kolumnie „Host” nawiązują do rekordu tabeli bazowej.

Implementacja kluczy

Zaimplementowanie elementu key w pliku business.xml deklaruje klucz, dzięki któremu można wyszukiwać, sortować oraz przeglądać obiekty tabeli, na której zostanie stworzony indeks bazodanowy wpływający na wydajność pobierania danych. Element key może występować zarówno w obiekcie table, jak i w obiekcie subrow. Trzeba pamiętać również, że co najmniej jeden element key jest wymagany w tabeli. Klucze w plikach business.xml budujemy w następujący sposób:

```
    <key name="WgDatTypPozycja" keyunique="true">
        <keycol name="DataOd"></keycol>
        <keycol name="DataDo"></keycol>
        <keycol name="Typ"></keycol>
        <keycol name="Pozycja"></keycol>
    </key>
```

Jest to klucz unikalny zbudowany z czterech kolumn. Po zadeklarowaniu go zostanie wygenerowana klasa reprezentująca ten klucz. Posiada ona taka sama ilość indekserów jak ilość kolumn zadeklarowanych w elemencie klucza. Kolejność umieszczania kolumn w kluczu tabeli nie jest przypadkowa i musi być dobrze przemyślana. Kolumny klucza kolejno zawężają nam dane wyszukiwania.

```
public SubTable<ElemRozliczenia> this [Date dataod] => new SubTable<ElemRozliczenia> (this, dataod);
public SubTable<ElemRozliczenia> this [Date dataod, Date datado] => new SubTable<ElemRozliczenia>
(this, dataod, datado);
public SubTable<ElemRozliczenia> this [Date dataod, Date datado, TypElementuRozliczenia typ] => new
SubTable<ElemRozliczenia> (this, dataod, datado, typ);
public ElemRozliczenia this [Date dataod, Date datado, TypElementuRozliczenia typ,
PozycjaDokHandlowego pozycja] => (ElemRozliczenia)Find (dataod, datado, typ, pozycja);
```

W kluczu, na którym ustawiliśmy atrybut keyunique jako true, w momencie uzupełnienia parametrów wyszukiwanych danych do ostatniej kolumny klucza – jak można zauważyć w powyższym przykładzie, zostanie zwrócony pojedynczy element. Rożne obiekty biznesowe nie mogą zawierać dokładnie takich samych wartości pól, które znajdują się w kluczu. Inaczej dzieje się w kluczu, który nie jest ustawiony jako unikalny – ostatni indekser, tak samo jak i poprzednie, zwróci nam subtable:

```
<key name="WgPozycjaTyp">
        <keycol name="Pozycja"></keycol>
        <keycol name="Typ"></keycol>
</key>

public SubTable<ElemRozliczenia> this [PozycjaDokHandlowego pozycja] => new SubTable<ElemRozliczenia>
(this, pozycja);
public SubTable<ElemRozliczenia> this [PozycjaDokHandlowego pozycja, TypElementuRozliczenia typ] =>
new SubTable<ElemRozliczenia>(this, pozycja, typ);
```

Dzięki temu, ze klucz został zadeklarowany w tabeli ElemRozliczenia, co skutkuje dodaniem odpowiedniej właściwości w klasie ją reprezentującej:

```
public WgDatTypPozycjaKey WgDatTypPozycja =>
(WgDatTypPozycjaKey)Session.Keys[keyInfoElemRozliczeniaWgDatTypPozycja];
```

Praktyczne zastosowanie klucza można znaleźć w poniższym przykładzie metody walidującej, która pochodzi z weryfikatora EANUniqueVerifier: ColVerifier,

```
    protected override bool IsValid ()
    {
        if (Row.EAN! = "")
            foreach (var towar in Row.Table. WgEAN[Row.EAN])
                if (towar! = Row)
                {
                    info = towar.ToString ();
                    return false;
                }
        return true;
    }
```

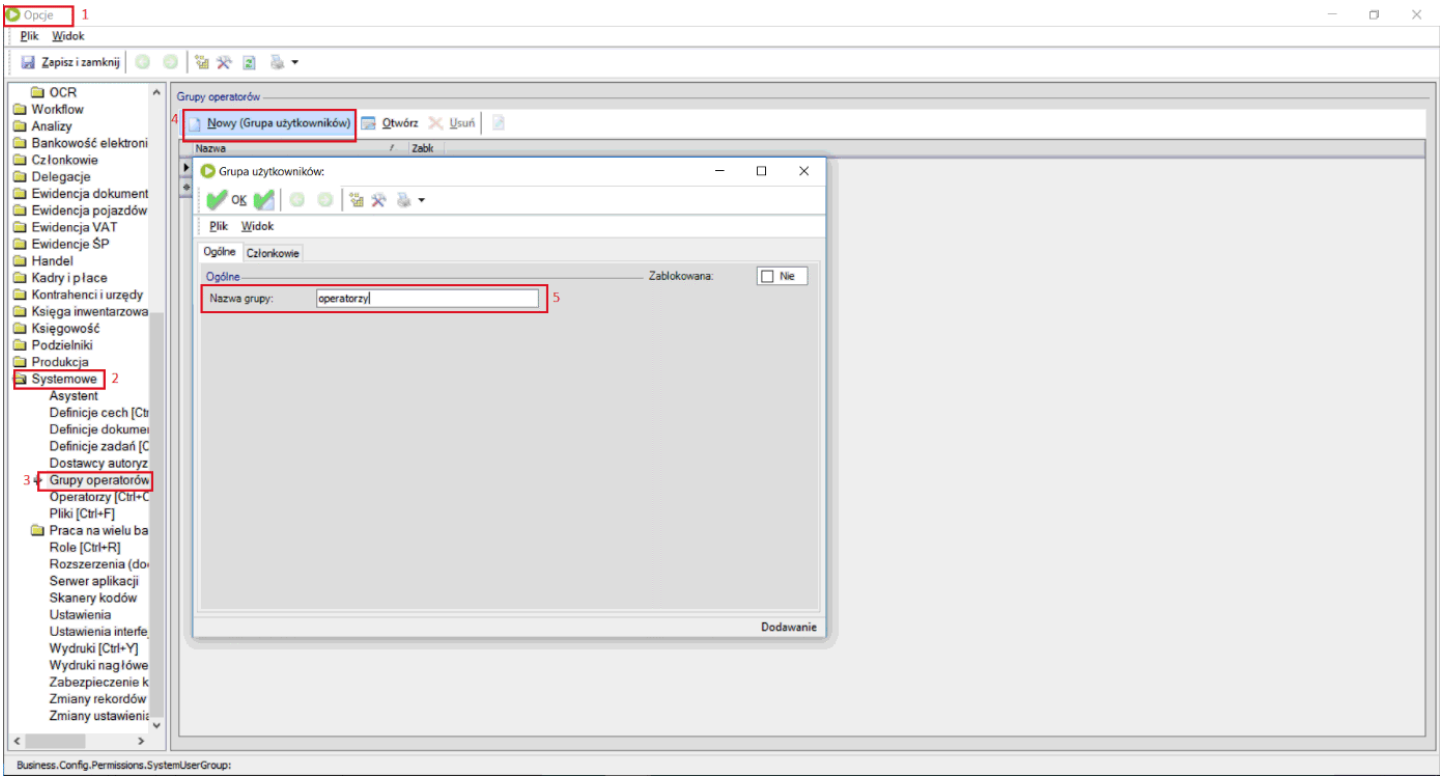
Gdzie wewnątrz pętli następuje sprawdzenie, czy w kolekcji pobranej za pomocą klucza WgEAN istnieje już towar o takim samym numerze EAN jak wiersz, który chcemy dodać.

Plik *.dbinit.xml

Plik *.dbinit.xml wypełnia dodane tabele odpowiednimi wartościami. Dane zawarte w tym pliku są wczytywane do bazy danych zaraz po utworzeniu jej struktury. Również podczas dokonywania konwersji danych plik jest wczytywany, dokonując konwersji w standardowych zapisach programu.

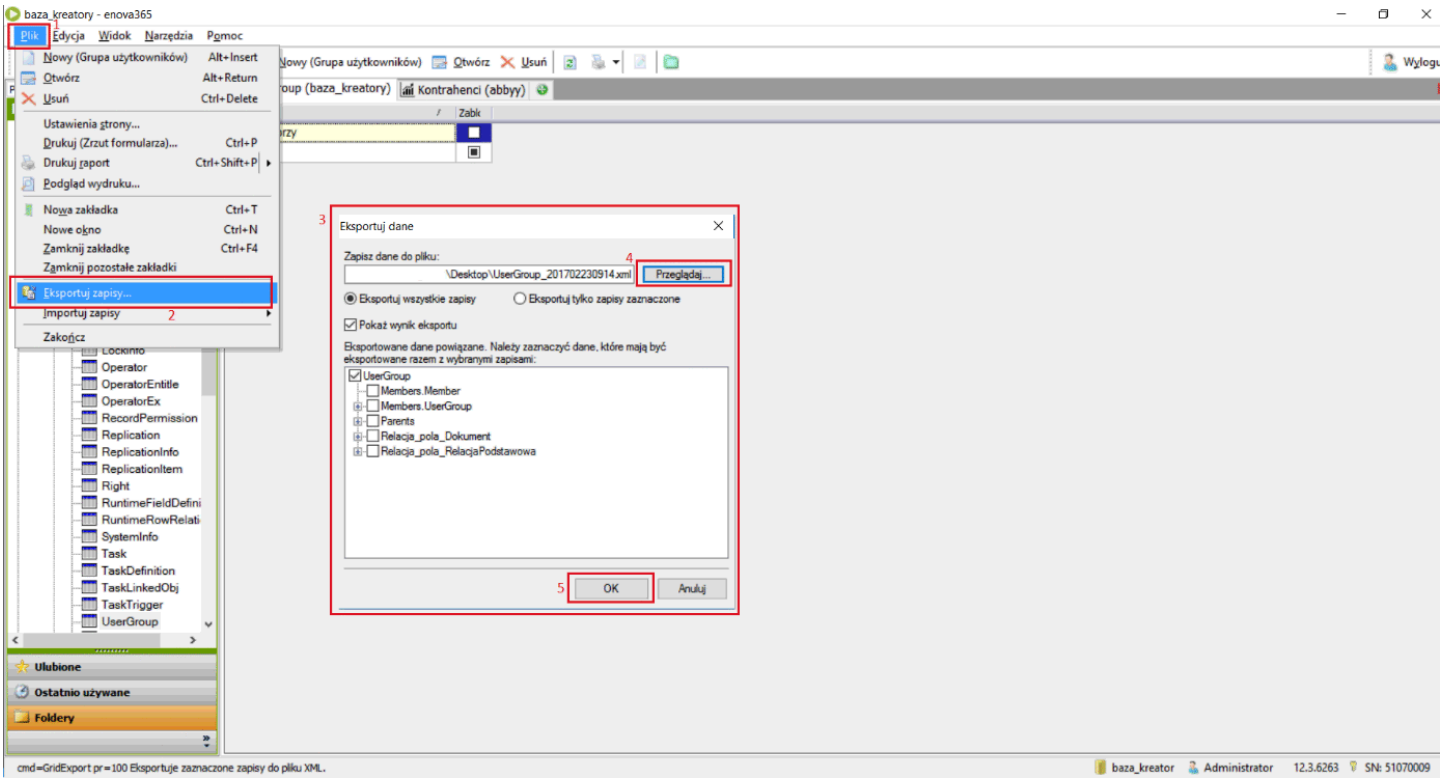
Na początek powinniśmy zacząć od utworzenia nowej konfiguracji w programie enova365. Przykładowo chcemy dodać do standardowej konfiguracji nową grupę operatorów, która się będzie nazywać ‘operatorzy’. Do tej grupy będzie należał jedynie administrator.

1. Odpalamy program enova365, przechodzimy do opcji (ctr +F9).
2. Następnie klikamy na folder Systemowe.
3. Przechodzimy do zakładki Grupy operatorów.
4. Klikamy przycisk ‘Nowa grupa’.
5. Odpowiednio ustawiamy konfigurację, dzięki której utworzymy nową grupę.



Po utworzeniu grupy możemy przejść do wyeksportowania jej.

1. Przechodzimy do tabeli UserGroup i klikamy Plik.
2. Klikamy przycisk Eksportuj zapisy.
3. Powinno się pojawić okno Eksportuj dane.
4. Za pomocą przycisku ‘Przeglądaj’ wybieramy ścieżkę do której zapiszemy plik.
5. Następnie wybieramy przycisk ‘OK’.

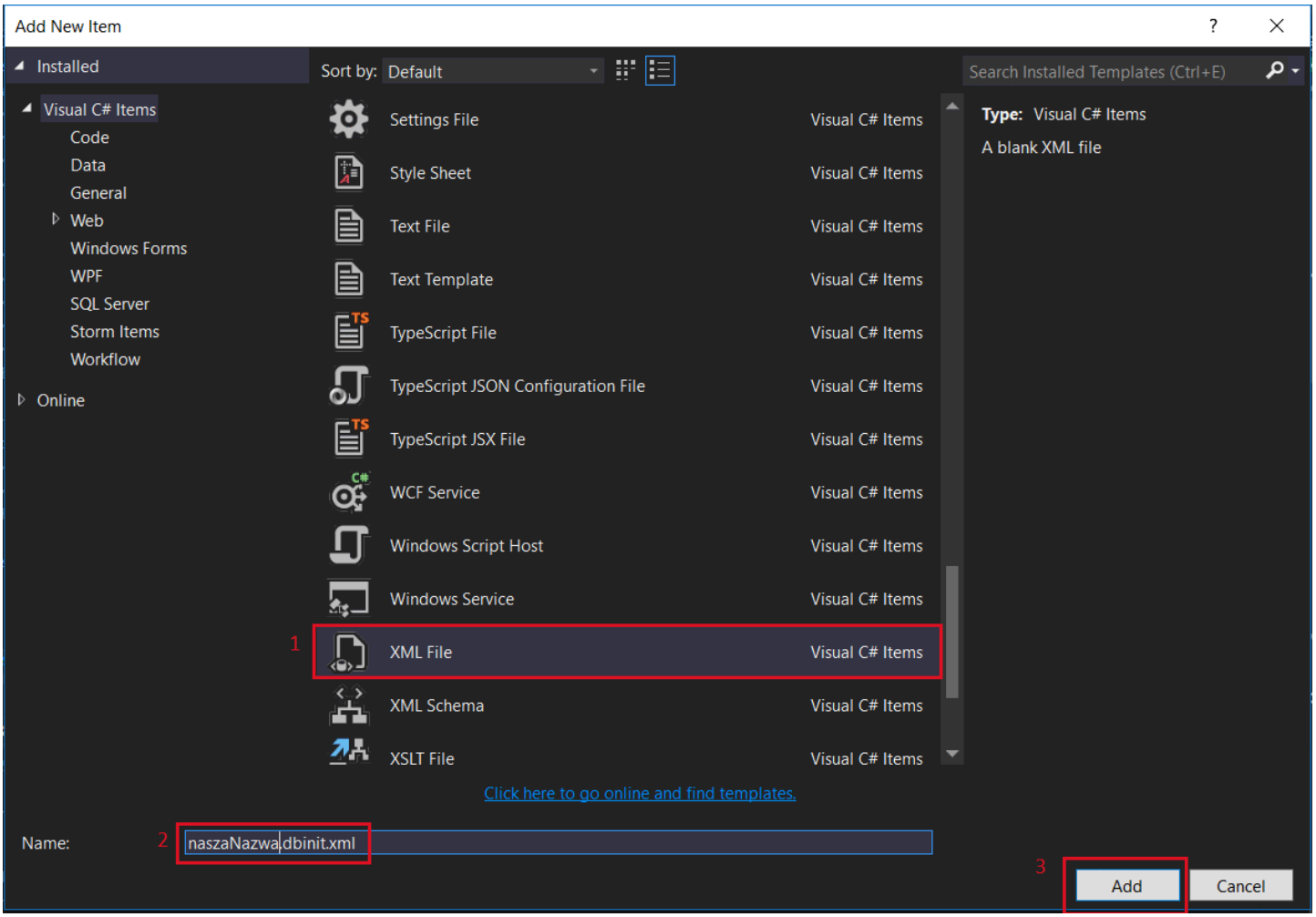


Na podstawie powyższego opisu utworzy nam się plik *.XML, który zawiera kod:

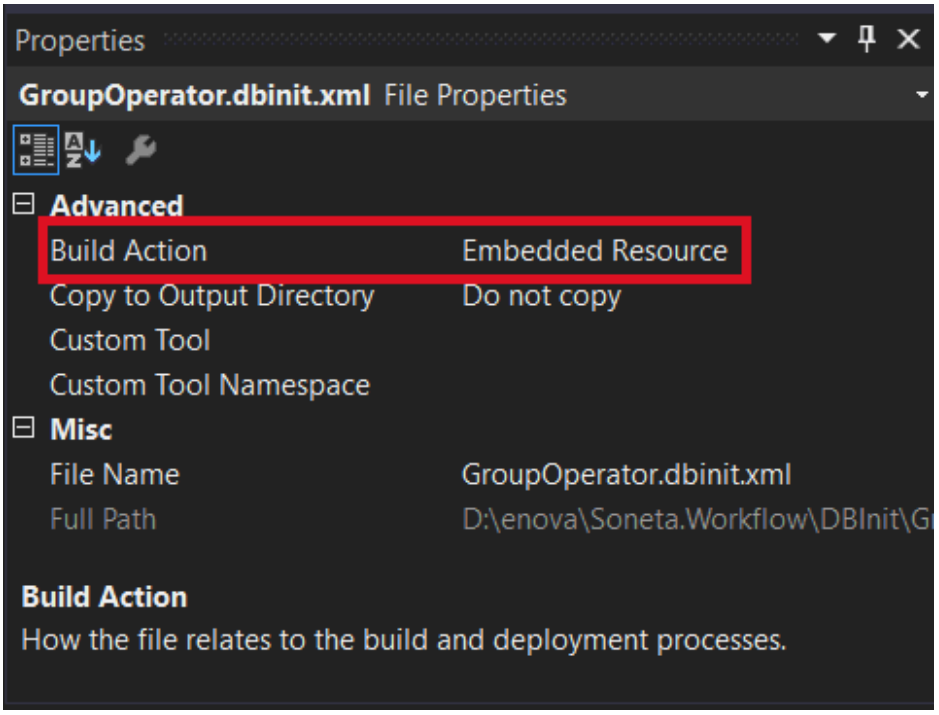
```
<?xml version="1.0" encoding="Unicode"?>
<session xmlns="http://www.soneta.pl/schema/business">
   <UserGroup id="UserGroup_1" guid="225780ee-01fc-4835-b3cc-e2252e4df8da">
   <Name>operatorzy</Name>
   <Locked>False</Locked>
   <Members>
     <UserGroupMember id="UserGroupMember_1">
       <UserGroup>UserGroup_1</UserGroup>
       <Member>Operator:00000000-0015-0001-0001-000000000000</Member>
       <Locked>False</Locked>
     </UserGroupMember>
   </Members>
  </UserGroup>
</session>
```

Na podstawie tego kodu możemy utworzyć plik *.dbinit.xml.

1. Wybieramy z listy tworzenie pliku XML File.
2. Następnie ustawiamy nazwę i dodajemy rozszerzenie .dbinit.xml, na przykład GroupOperator.dbinit.xml.
3. Klikamy przycisk dodania nowego pliku do projektu.



W kolejnym kroku powinniśmy przejść do właściwości dodanego pliku. W nich ustawiamy parametr Build Action na Embedded Resource. Bez ustawienia tej właściwości dany plik *.dbinit.xml nie będzie wczytany do enova365.



Następnym krokiem jest wklejenie wcześniej wygenerowanego kodu do pliku GroupOperator.dbinit.xml.

Poniżej znajduje się przykład prawidłowo napisanego *.dbinit.xml.

```
<?xml version="1.0" encoding="Unicode"?>
<session xmlns="http://www.soneta.pl/schema/business" versionName="nazwa">
  <UserGroup id="UserGroup_1_120000" guid="225780ee-01fc-4835-b3cc-e2252e4df8da" dbversion="120000">
    <Name>operatorzy</Name>
    <Locked>False</Locked>
    <Members>
      <UserGroupMember id="UserGroupMember_1_120000">
        <UserGroup>UserGroup_1_120000</UserGroup>
        <Member>Operator:00000000-0015-0001-0001-000000000000</Member>
        <Locked>False</Locked>
      </UserGroupMember>
    </Members>
  </UserGroup>
</session>
```

W tagu session dodajemy parametr versionName="nazwa'.

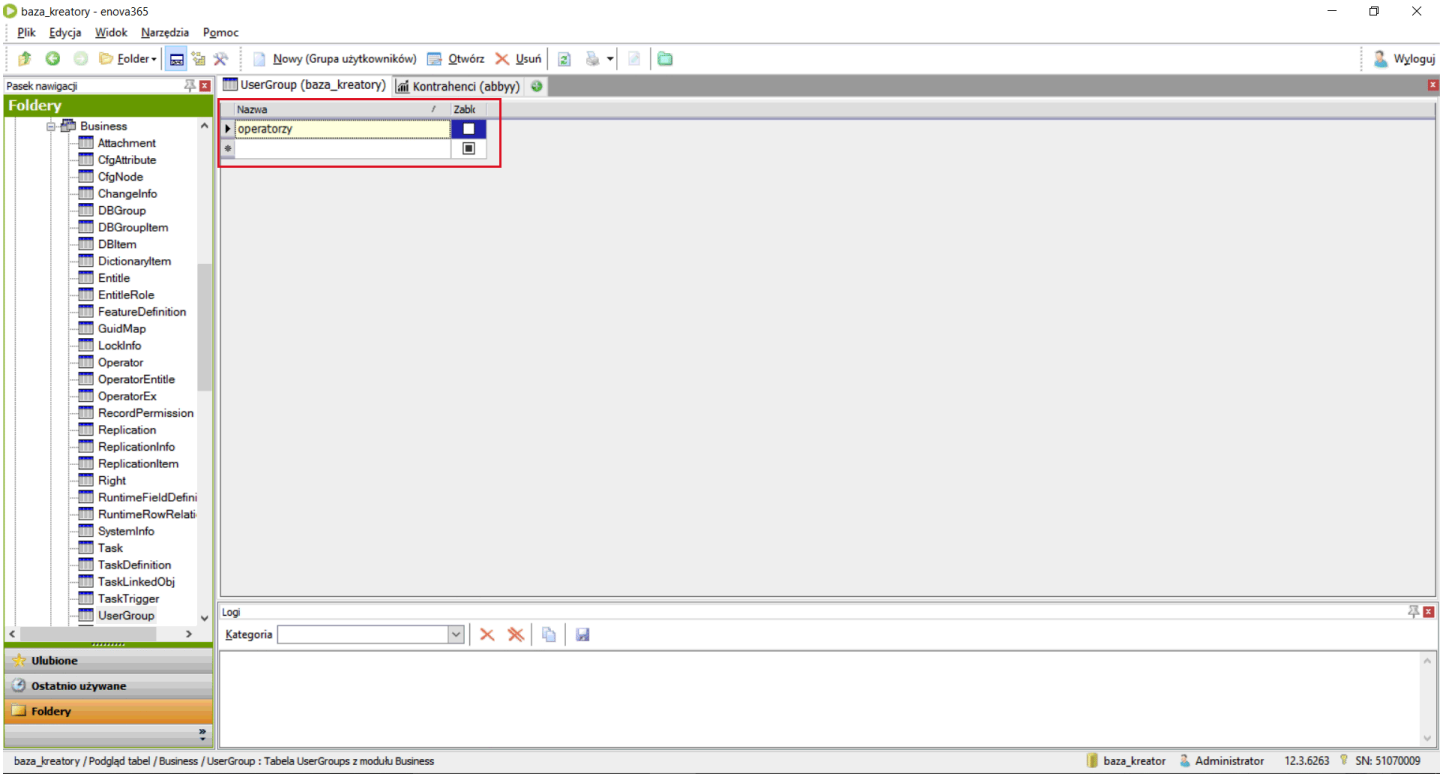
versionname – nazwa numerów wersji bazy danych, która będzie sprawdzana. W tym przypadku jest to 'nazwa'. Jest to ta sama nazwa, która znajduje się w pliku nazwa.business.xml, w elemencie głównym module w atrybucie versionName

W tagu UserGroup dodajemy parametr dbversion="120000".

dbversion - numer wersji bazy danych, do której ma dodana nowa funkcjonalność. W tym przypadku jest to wersja 12.0.

Ważnym elementem jest również dobrze ustawione id, najlepiej żeby zależało od wersji, w której jest wywoływane.

Po wykonaniu wszystkich powyższych czynności grupa operatorów pojawi się w bazie danych.



#Wersjonowanie plików *.dbinit.xml#

Wersjonowanie plików jest uzależnione od ustawień w versionName. W kolejnych wersjach możemy jednak zmieniać dodatkowe ustawienia. Przykładowo chcemy żeby w wersji 12.3 grupa jednak nazywała się ‘grupa zarządzająca’.

Do istniejącego pliku GroupOperator.dbinit.xml dodajemy następujące linie kodu:

```
  <UserGroup id="UserGroup_1_120300" guid="225780ee-01fc-4835-b3cc-e2252e4df8da" dbversion="120300"
updateonly ="true">
    <Name>grupa zarządzająca</Name>
  </UserGroup>
```

Dodajemy nowy tag UserGroup, w którym ustawiamy:

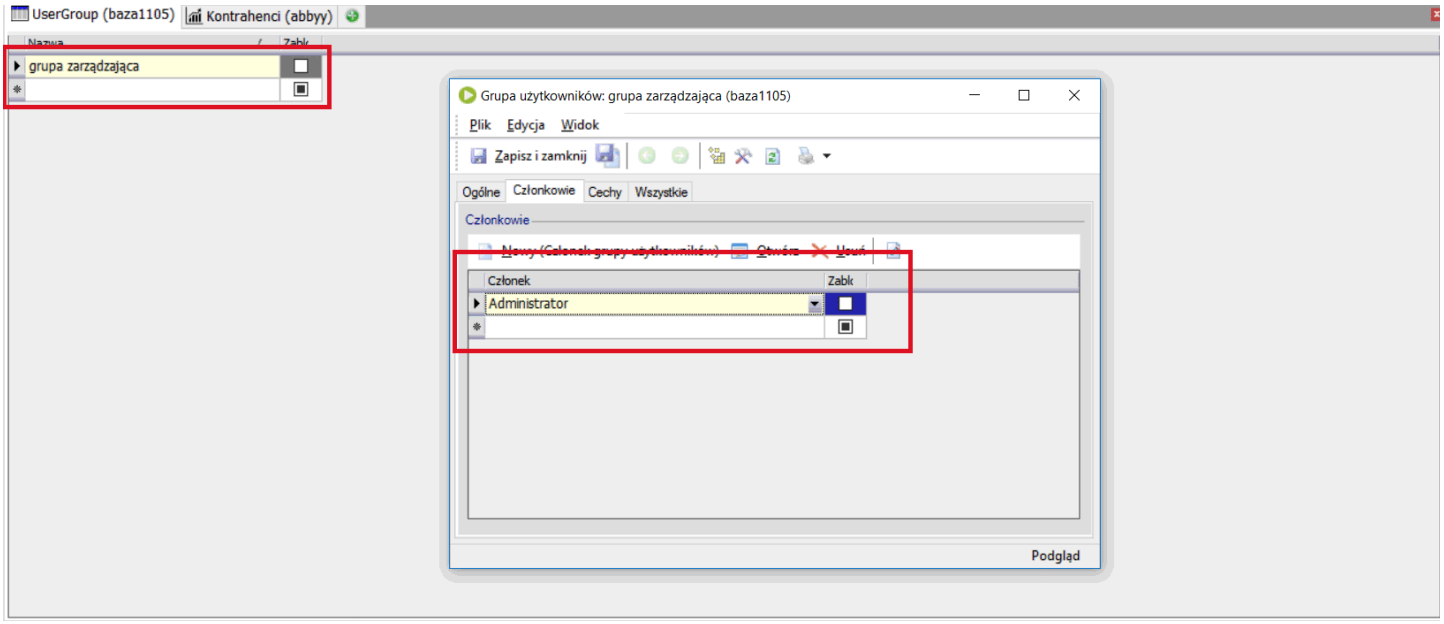
- id="UserGroup_1_120300" - id może być uzależnione od wersji bazy danych
- guid="225780ee-01fc-4835-b3cc-e2252e4df8da" - GUID odnoszący się do grupy użytkowników, którą chcemy zmienić
- dbversion="120300" - wersja baz danych, od której ma nastąpić ta zmiana
- updateonly ="true" - parametr ten aktualizuje dane, a nie całkowicie tworzy je od nowa.

W kolejnych liniach należy dodać tagi, które zawierają zmieniane parametry oraz wartość do nich przypisaną. W tym przypadku jest zmieniany parametr name (tag:< Name>) na wartość grupa zarządzająca.

Poniżej znajduje się kompletny przykład kodu:

```
  <?xml version="1.0" encoding="Unicode"?>
  <session xmlns="http://www.soneta.pl/schema/business" versionName="enova">
    <UserGroup id="UserGroup_1_120000" guid="225780ee-01fc-4835-b3cc-e2252e4df8da"
dbversion="120000">
      <Name>operatorzy</Name>
      <Locked>False</Locked>
      <Members>
        <UserGroupMember id="UserGroupMember_1_120000">
          <UserGroup>UserGroup_1_120000</UserGroup>
          <Member>Operator:00000000-0015-0001-0001-000000000000</Member>
          <Locked>False</Locked>
        </UserGroupMember>
      </Members>
    </UserGroup>
    <UserGroup id="UserGroup_1_120300" guid="225780ee-01fc-4835-b3cc-e2252e4df8da" dbversion="120300"
updateonly ="true">
      <Name>grupa zarządzająca</Name>
    </UserGroup>
  </session>
```

Wynik działania powyższego kodu otrzymany po konwersji do 12.3:


UserGroup (baza1105)
Kontrahenci (abbyy)
grupa zarządzająca
Grupa użytkowników: grupa zarządzająca (baza1105)
Plik
Edycja
Widok
Zapisz i zamknij
Ogólne
Członkowie
Cechy
Wszystkie
Członkowie
Członek
Zablk
Administrator
Podgląd

Wydruki

W skład grupy *Wydruki* wchodzą wszystkie artykuły zawierające wiedzę o mechanizmach tworzenia wykresów.

Jak użyć w swoim zewnętrznym programie (dodatku) programu Soneta.PdfPrinter.exe?

Soneta.PdfPrinter.exe jest dodatkowym programem który realizuje wydruki do plików pdf. Dzięki niemu możemy zamienić dowolny plik html na pdf i podpisać go.

Program wywołujemy jako proces konsoli podając poszczególne argumenty.

Przy standardowym wywołaniu:

0. zabezpieczenie przed nieuprawnionym wywołaniem – to marker „asdfghjkl”

1. komenda do wykonania:

    - printer – drukuj do drukarki;
    - printerprompt – drukuj do drukarki z możliwością jej wyboru;
    - previewpdf – drukuj do pdf z podglądem;
    - pdf – stwórz plik pdf;

2. ścieżka do pliku htm

3. nazwa drukarki w systemie na którą ma być wykonane drukowanie

4. nazwa pliku pdf do którego ma być wykonany zapis

5. autor pliku pdf – tekst/nazwa autora np. „Autor”

6. podpis cyfrowy – przełącznik „S” (podpis) lub „SV” (widoczny podpis)

Przy alternatywnym wywołaniu w formie servera:

0. włączamy server – przełącznik /server
1. ścieżka do pliku html

Przy alternatywnym wywołaniu w formie analizatora plików html:

0. włączamy analizator – przełącznik /analyze
1. ścieżka do pliku html

Przy alternatywnym uruchomieniu służącym do podpisania pliku pdf podpisem cyfrowym:

0. włączanie podpisywanie pdf’a - przełącznik /sign
1. ścieżka do pliku pdf do podpisania
2. podpis cyfrowy – przełącznik „S” (podpis) lub „SV” (widoczny podpis)
3. autor pliku pdf – tekst/nazwa autora np. „Autor”

Uruchomienie drukarki w kodzie może wyglądać np. tak:

```
var psi = new ProcessStartInfo();

psi.FileName = @"D:\Soneta.PdfPrinter.exe";

psi.Arguments ="asdfghjkl pdf \"D:\\Cennik.htm\" \"\" \"D:\\Cennik.pdf\" \"Podstawowy
```

```
operator programu\" \"_\"";

        psi.UseShellExecute = false;

        var process = Process.Start(psi);
```

Program zamienia plik Cennik.htm na plik Cennik.pdf bez użycia podpisu cyfrowego ustawiając autora pliku na „Podstawowy operator programu”.

Powyższy kod zakłada, że plik programu Soneta.PdfPrinter.exe znajduje się na dysku D, oraz że konwertowany plik Cennik.htm oraz wynikowy Cennik.pdf mają znajdować się w tym samym miejscu.

Jak używać obiektu AspReportGenerator do drukowania we własnych workerach lub dodatkach? *(dotyczy tylko "nowej" drukarki)

Artykuł dotyczy generowania wydruków ASPX z bezpośrednim użyciem klasy **AspReportGenerator** (dawniej **AspGenerator**). Jest to metoda przestarzała. Rekomendowanym mechanizmem generowania wydruków do pliku jest **ReportResult** w powiązaniu z **IReportService**. Mechanizm ten umożliwia generowanie wydruków zarówno w technologii ASPX jak i REPX. Posiada także implementację odpowiednią dla enova365 w wersji HTML.
Uwaga!
Wsparcie dla **AspGenerator** było zapewnione jedynie do wersji 2406. Od wersji 2410 ta metoda nie jest już dostępna i należy skorzystać z wyżej wskazanego mechanizmu ReportResult.

AspReportGenerator służy do generowania wydruków z szablonów zapisanych w plikach aspx. Każdy raport może zostać wygenerowany jako plik HTML by później być skonwertowany do Pdf lub w formie innych plików takich jak Word, RTF, itd.

Przykład uruchomienia wydruku do pliku PDF:

Po przygotowaniu w workerze wydruku (czym tutaj się nie będziemy zajmować!) przystępujemy do utworzenia obiektu AspReportGenerator:

```
using Soneta.Printer;
var generator = new AspReportGenerator
{
    Name = "Test",
    TemplateFileName = @"D:\test.aspx",
    Destination = AspGenerator.Destinations.PDF,
    OutputFileName = @"D:\Test.pdf"
};
```

Słówko kluczowe var oznacza, że kompilator sam rozpozna typ zmiennej.

1. We właściwości **Name** obiektu przechowujemy nazwę raportu.

2. W **TemplateFileName** podajemy ścieżkę do szablonu raportu - pliku aspx. Na potrzeby artykułu używamy testowego wydruku umieszczonego na dysku D. Można też użyć ścieżek zależnych do katalogu Reports np. handel\cennik.aspx.

3. **Destination** określa jaki typ wyjściowy generator ma zwrócić. Jest kilkanaście możliwości:

    - Default - opcja standardowa
    - Printer - wydruk na domyślną drukarkę
    - PrinterPrompt - wydruk na drukarkę z potwierdzeniem

- Preview - podgląd wydruku
- PDF - dokument pdf
- PreviewPDF - podgląd dokumentu pdf (historyczne, w nowej drukarce podgląd jest zawsze podglądem pdf)
- AttachmentsPDF - załączniki pdf
- EmailPDF - wysyła email z załączonym plikiem pdf
- Explorer - wyrzut czystego HTML do domyślnej przeglądarki internetowej
- Sheet - dokument arkusza kalkulacyjnego
- Word - dokument edytora tekstowego Word
- AttachmentsWord - załącznik w postaci dokumentu edytora tekstowego Word
- RTF - dokument edytora tekstowego Word Pad
- AttachmentsRTF - załącznik w postaci dokumentu edytora Word Pad
- Notepad - zrzut do notatnika
- NotepadText - zrzut czystego tekstu wydruku bez znaczników HTML
- File - zapis do pliku
- RunApplication - uruchom aplikację podajac wydruk w parametrze
- AspxSample - pokaż przykład wywołania aspx

4. **OutputFileName** docelowy plik do którego ma trafić wynik operacji (dla opcji które zapisujemy do pliku, takie jak przykładowy PDF).

Powyższy przykład wygeneruje nam plik Test.pdf w lokalizacji D:\.

Podpisywanie pliku pdf podpisem cyfrowym przez obiekt generatora

Podpis cyfrowy działa tylko dla **Destination** PDF, AttachmentsPDF oraz EmailPDF.

```
var generator = new AspReportGenerator
{
    Name = "Test",
    TemplateFileName = @"D:\test.aspx",
    Destination = AspGenerator.Destinations.PDF,
    OutputFileName = @"D:\Test.pdf",
    Sign = true
};
```

Po wykonaniu powyższego przykładu w "nowej" drukarce Pdf pojawi się okienko wyboru certyfikatu i po jego wyborze generator utworzy podpisany plik Test.pdf. Aby podpis był widoczny na wydruku należy użyć takiej konstrukcji.

```
var generator = new AspReportGenerator
{
    Name = "Test",
    TemplateFileName = @"D:\test.aspx",
    Destination = AspGenerator.Destinations.PDF,
    OutputFileName = @"D:\Test.pdf",
    Sign = true,
    VisibleSignature = true
};
```

Enkrypcja pliku pdf

"Nowa" drukarka umożliwia także enkrypcję plików pdf. Hasło będzie wymagane do otworzenia pliku w dowolnej przeglądarce pdf.

```
var generator = new AspReportGenerator
{
    Name = "Test",
    TemplateFileName = @"D:\test.aspx",
    Destination = AspGenerator.Destinations.PDF,
    OutputFileName = @"D:\Test.pdf",
    Encrypt = "hasło"
};
```

Property **Encrypt** określa hasło jakie będzie musiał wpisać użytkownik aby uzyskać podgląd pliku.

Uruchamianie Generatora

Po odpowiednim ustawieniu właściwości obiektu generatora uruchamiamy go poleceniem **Print**. Print wygląda następująco:

```
public override bool Print(Control mainControl, Context context, bool preview, IEnumerable
enumerable)
```

mainControl to controlka do której zostaną zaczepione okna. Jeśli jej nie ma można podać null. **context** będzie zawierał kontekst dla operacji drukowania. Prawdopodobnie będzie to

aktualny kontekst uzupełniony pewnymi elementami (np. jeśli nie chcemy wywoływać okna parametrów możemy wypełnić je w kontekście). **preview** ustawione na true pokaże podgląd dokumentu nawet jeśli **Destination** nie zostało ustawione na podgląd. **enumerable** pozwala na dodanie dodatkowych obiektów do wydruku.

W naszych przypadkach pokazanych powyżej wystarczy, że wywołamy:

```
generator.Print(null, context, false, null) ;
```

Generowanie i podpisywanie załączników w aspx i repx

Przykład workerów, które działają z wzorcami sprzedaz.aspx oraz sprzedaz.repx. Workery można wywołać w dokumencie handlowym i mają za zadanie pobrać szablon dokumentu, a następnie podpisać dokument certyfikatem kwalifikowanym, ustawić nazwę pliku pdf i zapisać jako załącznik do bazy danych.

1. Worker dla aspx

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Soneta.Business;
using Soneta.Business.UI;
using Soneta.Handel;
using Soneta.Types;
using Soneta.Business.Db;
using System.IO;
using Microsoft.Extensions.DependencyInjection;

[assembly: Worker(typeof(Prezentacja.GenerujZalacznik), typeof(DokumentHandlowy))]

namespace Prezentacja
{
    class GenerujZalacznik
    {
         //Podniesienie okna parametrów wydruku i przekazanie do kontekstu
        [Context]
        public ParametryWydrukuDokumentu Param { get; set; }

        [Context]
        public Context Cx { get; set; }


        [Context]
        public DokumentHandlowy Dh { get; set; }

        [Action("DrukujemyZałącznik", Priority = 0, Target = ActionTarget.ToolbarWithText)]
        public void Drukuj()
        {
            using (var t = Dh.Session.Logout(true))
            {

                IReportService rs;            <- referencja do IReportService
```

```
                rs = Cx.Session.GetRequiredService<IReportService>();


                ICertificateService cs;      <- referencja do ICertyficateService

                cs = Cx.Session.GetRequiredService<ICertyficateService>();

                //<- opcjonalnie parametry wydruku można na stałe ustawić w kontekście

                //Cx.Set(new ParametryWydrukuDokumentu(Cx) { Oryginał = true, Duplikat = false,
IloscKopii = 0 });

                var bm = BusinessModule.GetInstance(Cx.Session);

                var report = new ReportResult
                {
                    TemplateFileSource = Soneta.Business.UI.AspxSource.Storage,
                    TemplateFileName = "aspx\\sprzedaz.aspx",
                    Format = Soneta.Business.UI.ReportResultFormat.PDF,
                    Context = Cx,

                    Sign = true or false,                     //opcjonalnie - podpis dokumentu
certyfikatem kwalifikowanym, podniesienie okna wyboru certyfikatu

                    VisibleSignature = true  or false  //opcjonalnie - sygnatura na wydruku, o
prawidłowym podpisaniu dokumentu

                };

                //stworzenie obiektu klasy Attachment
                Attachment zalW = new Attachment(Dh, AttachmentType.Attachments);

                bm.Attachments.AddRow(zalW);

                //wygenerowanie streama ze zwróconego tymczasowego pliku i załadowanie go do obiektu
klasy Attachemnts
                using (Stream streamW = rs.GenerateReport(report))
                {
                   //Jeśli byłaby potrzeba skorzystania z automatycznego podpisania dokumentu
certyfikatem kwalifikowanym to można skorzystać z metody SignPdfFile.
                   //using (MemoryStream ms = new MemoryStream())

                   //{

                   //metoda SignPdfFile, która podpisuje certyfikat o konkretnym odcisku palca,
posiada dwa parametry(byte[] data, string thumbprint = null)
                   //drugi parametr to odcisk palca pobierany z certyfikatu kwalifikowanego

                      //streamW.CopyTo(ms);
                      //byte[] bytes = ms.ToArray();


                      //bytes = cs.SignPdfFile(bytes, "0fd4e338726c4448e90e87240b20fe525066ca58");
```

```
                        //ms.Seek(0, SeekOrigin.Begin);
                        //ms.Write(bytes, 0, bytes.Length);
                        //ms.Seek(0, SeekOrigin.Begin);
                        //zalW.LoadFromStream(ms);

                    //}
                zalW.LoadFromStream(streamW);
                }

                //ustawienie nazwy pliku pdf

                string nameW = Dh.Numer.ToString().Replace("/", "");
                zalW.Name = nameW + ".pdf";

                //koniec transakcji

                t.Commit();
            }
        }
    }
}
```

2. Worker dla repx

Aby uzyskać dostęp do przestrzeni nazw **XtraReportSerialization** dla wzorca **"sprzedaz.repx"** należy w tym przypadku w referencjach projektu dodać odniesienie do biblioteki **Soneta.Handel.Reports.dll** w odpowiedniej wersji.

Nie jest to zalecane przez nas postępowanie w odniesieniu do korzystania z Soneta.SDK, ale do momentu udostępnienia tej klasy w modułach dostarczanych przez Soneta.SDK nie ma innej możliwości obejścia tego problemu.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Soneta.Business;
using Soneta.Business.UI;
using Soneta.Handel;
using Soneta.Types;
using Soneta.Business.Db;
using System.IO;
using Microsoft.Extensions.DependencyInjection;

[assembly: Worker(typeof(Prezentacja.GenerujZalacznik), typeof(DokumentHandlowy))]

namespace Prezentacja
```

```
{
    class GenerujZalacznik
    {
        //REPX - generacja zalacznika z repx i podpisanie go

        [Context]
        public Context Cx { get; set; }

        [Context]
        public DokumentHandlowy Dh { get; set; }

        [Action("DrukujemyZałącznik", Priority = 0, Target = ActionTarget.ToolbarWithText)]
        public void Drukuj()
        {
            using (var t = Dh.Session.Logout(true))
            {
                // referencja do IReportService
                IReportService rs = Cx.Session.GetRequiredService<IReportService>();

                // referencja do ICertificateService
                ICertificateService cs = Cx.Session.GetRequiredService<ICertificateService>();

                // opcjonalnie parametry wydruku można na stałe ustawić w kontekście
                Cx.Set(new XtraReportSerialization.Sprzedaz.MyParametryWydruku(Cx)); //{ Duplikat =
false });

                var bm = BusinessModule.GetInstance(Cx.Session);
                var report = new Soneta.Business.UI.ReportResult
                {
                    Context = Cx,
                    Format = Soneta.Business.UI.ReportResultFormat.PDF,
                    DataType = typeof(DokumentHandlowy),
                    TemplateFileName = "XtraReports/Wzorce
użytkownika/handel/dokhandlowy/sprzedaz.repx",
                    Preview = false,
                    Sign = true,   //opcjonalnie - podpis dokumentu certyfikatem kwalifikowanym,
podniesienie okna wyboru certyfikatu
                };

                //stworzenie obiektu klasy Attachment
                Soneta.Business.Db.Attachment zalW = new Soneta.Business.Db.Attachment(Dh,
AttachmentType.Attachments);
                bm.Attachments.AddRow(zalW);

                //wygenerowanie streama ze zwróconego tymczasowego pliku i załądowanie go do obiektu
klasy Attachemnts
                 using (Stream streamW = rs.GenerateReport(report))
                {
                   //Jeśli byłaby potrzeba skorzystania z automatycznego podpisania dokumentu
certyfikatem kwalifikowanym to można skorzystać z metody SignPdfFile.
                   //using (MemoryStream ms = new MemoryStream())
                   //{
                   //metoda SignPdfFile, która podpisuje certyfikat o konkretnym odcisku palca,
```

```
posiada dwa parametry(byte[] data, string thumbprint = null)
                    //drugi parametr to odcisk palca pobierany z certyfikatu kwalifikowanego

                        //streamW.CopyTo(ms);
                        //byte[] bytes = ms.ToArray();

                        //bytes = cs.SignPdfFile(bytes, "0fd4e338726c4448e90e87240b20fe525066ca58");
                        //ms.Seek(0, SeekOrigin.Begin);
                        //ms.Write(bytes, 0, bytes.Length);
                        //ms.Seek(0, SeekOrigin.Begin);
                        //zalW.LoadFromStream(ms);
                    //}

                zalW.LoadFromStream(streamW);
                }

                //ustawienie nazwy pliku pdf
                string nameW = Dh.Numer.ToString().Replace("/", "");
                zalW.Name = nameW + ".pdf";

                //koniec transakcji
                t.Commit();
            }
        }
    }
}
```

Używając DataType = typeof(Towary), TemplateFileName = „XtraReports/Wzorce użytkownika/handel/dokhandlowy/sprzedaz.repx odnosimy się do raportów zapisywanych w bazie danych. Jeśli chcemy skorzystać z wzorców standardowych musimy te linijki zamienić na TemplateType = typeof(XtraReportSerialization.Sprzedaz).

Więcej informacji jak działają Rezultaty Raportów dostępne pod linkiem: oraz linkiem.

Podstawowe wiadomości o Workerach dostępne są tutaj.

3. Przykład workera który wykorzystuje ReportResult i eksportuje plik PDF przez OutPutHandler

```
using System;
using System.IO;
using Soneta.Business;
using Soneta.Business.UI;
using Soneta.Tools;
using Microsoft.Extensions.DependencyInjection;
```

```
[assembly: Worker(typeof(Soneta.Handel.OdpalWydrukWorker), typeof(Soneta.Handel.DokumentHandlowy))]

namespace Soneta.Handel
{
    public class OdpalWydrukWorker
    {
        [Action("Wydruk 1",
        Mode = ActionMode.SingleSession,
        Target = ActionTarget.Menu | ActionTarget.ToolbarWithText)]
        public void Drukuj1(Context context)
        {
            //context.Set(new XtraReportSerialization.Sprzedaz.MyParametryWydruku(context)
            //    { Rabat = false, Duplikat = false, WgCen = false });

            var rr = new ReportResult
            {
                Context = context,
                Format = ReportResultFormat.PDF,
                DataType = typeof(DokumentHandlowy),

                TemplateFileName = @"XtraReports/Wzorce użytkownika/sprzedaz.repx",
            };

            IReportService service;

            service = context.Session.GetRequiredService<IReportService>();




            var path = Environment.ExpandEnvironmentVariables(@"%userprofile%\Desktop\wydruk1.pdf");
            GenerateFile(service.GenerateReport(rr), path);
        }

        [Action("Wydruk 2",
        Mode = ActionMode.SingleSession,
        Target = ActionTarget.Menu | ActionTarget.ToolbarWithText)]
        public object Drukuj2(Context context)
        {
            //context.Set(new XtraReportSerialization.Sprzedaz.MyParametryWydruku(context)
            //    { Rabat = false, Duplikat = false, WgCen = false });

            var rr = new ReportResult
            {
                Context = context,
                Format = ReportResultFormat.PDF,
                DataType = typeof(DokumentHandlowy),

                TemplateFileName = @"XtraReports/Wzorce użytkownika/sprzedaz.repx",
                OutputHandler = (stream) => {
                    var path =
Environment.ExpandEnvironmentVariables(@"%userprofile%\Desktop\wydruk2.pdf");
                    GenerateFile(stream, path);
```

```
                    return "OK";
                }
            };
            return rr;
        }

        private void GenerateFile(Stream stream, string path) {
            using (var fh = File.Create(path))
            {
                stream.Seek(0L, SeekOrigin.Begin);
                CoreTools.StreamCopy(stream, fh);
                fh.Flush();
            }
            if (!System.IO.File.Exists(path))
                throw new Exception("IO ERROR: Nie udało się zapisać pliku");
        }

    }


}
```

W przypadku wydruku **sprzedaz.repx** należy przełączyć parametr [Context(CreateOnly=false na true)]. To sprawi, że automatycznie stworzy instancję tego obiektu (tutaj MyParametryWydruku) i tym samym nie będzie wymagać ich w kontekście. Dzięki tej fladze CreateOnly te parametry z domyślnymi wartościami tworzą się automatycznie podczas wykonywania wydruku

Zastosowanie Report Result w tasku globalnym

Przykład taska globalnego, który ma za zadanie: zapisać plik pdf na dysk, zapisać załącznik do bazy danych i dołączyć do dokumentu oraz stworzyć załącznik podpiąć go do wiadomości email i wysłać poczta CRM.

Opis taska:

Task otwiera sesje więc na początek dobrze sobie przygotować obiekty, które będą rezprezentowały następujące moduły.

```
    [AutoAction("Przykładowa akcja automatyczna")]public void AkcjaAutomatyczna_FVZAL(){

    System.Console.WriteLine(@"Tworzenie załącznika do dokumentu handlowego zapis do bazy i export na dysk do katalogu");

   //zmienne przechowujące dane biznesowe związane z modułami

               HandelModule hm = HandelModule.GetInstance(Session);

               BusinessModule bm = BusinessModule.GetInstance(Session);

               ZadaniaModule zm = ZadaniaModule.GetInstance(Session);

               CRMModule crm = CRMModule.GetInstance(Session);
```

Zbiór danych pobranych z tabeli dokumentów handlowych i pofiltrowanych.

```
  //Pobieram do subtable dokumenty handlowe utworzone poprzedniego dnia

  SubTable dokumenty = hm.DokHandlowe.WgDaty[Date.Today.AddDays(-1)];

  RowCondition condition = new FieldCondition.In("Stan", StanDokumentuHandlowego.Zatwierdzony, StanDokumentuHandlowego.Zablokowany);

  condition &= new FieldCondition.Equal("Definicja.KierunekPlatnosci", KierunekPlatnosci.Przychod);
```

```
  //zbiór przefiltrowanych dokumentów sprzedaży

  dokumenty = dokumenty[condition];
```

Do celów testowych tworzymy nową sesje dla każdego nowego dokumentu oraz w pętli foreach iterujemy każdy dokument handlowy w zbiorze.

```
 if (dokumenty != null)
            {
                  foreach (DokumentHandlowy dok in dokumenty)
                        {

                            try
                                        {
                                          using(var session = Row.Session.Login.CreateSession(false,
false))

                                       {
                                               WiadomoscRobocza wr = new WiadomoscRobocza();

                                               System.Console.WriteLine(@"Rozpoczęto zapis załącznika
do dokument handlowy nr {0}.", dok.Numer.NumerPelny);



                                               string directory = "C:\\Katalog\\";

                                               string filepath = System.IO.Path.Combine(directory,
GenerateFileName(dok));



                                               Context context = Context.Empty.Clone(session);

                                               DokumentHandlowy dok1 = session[dok] as
DokumentHandlowy;
                                               context.Set(dok1);


                                               //Dla REPX
                                               //context.Set(new
XtraReportSerialization.Sprzedaz.MyParametryWydruku(context){Rabat = false , Duplikat = false, WgCen
= false });

                                               //Dla ASPX

                                               context.Set(new ParametryWydrukuDokumentu(context)
{Oryginał = true, Duplikat = false, IloscKopii = 0});
```

```
                                                //Wywołanie IReportService

                                                Soneta.Business.UI.IReportService report;

                                                report = session.GetRequiredService<IReportService>();


                                                //Generacja tymczasowego pliku pdf w ReportResult

                                                  Soneta.Business.UI.ReportResult rr = new
Soneta.Business.UI.ReportResult()
                                                  {

                                                        Context = context,
                                                       Preview = false,
                                                       Format =
Soneta.Business.UI.ReportResultFormat.PDF,

                                                       TemplateFileSource =
Soneta.Business.UI.AspxSource.Storage,
                                                       TemplateFileName = "sprzedaz.aspx",

                                                       //Tylko dla REPX
                                                       //DataType = typeof(DokumentHandlowy),
                                                       //TemplateFileName = "XtraReports/Wzorce
użytkownika/sprzedaz.repx",



                                                  };


                                               //Utworzenie streama z tymczasowego pliku PDF

                                               var streamW = report.GenerateReport(rr);

                                               //Metoda zapisująca plik na dysk

                                               SaveOnDisk(filepath, directory, streamW);

                                               System.Console.WriteLine(@"Zakończono zapisywanie
załącznika do dokument handlowy nr {0} na dysk.", dok.Numer.NumerPelny);
```

W przypadku dodania załącznika do bazy oraz wysłania wiadomości email należy stworzyć nowa transakcję

if(dok1.Attachments.IsEmpty) {

```
using(var tran = Session.Logout(true))
                           {
                        //Załącznik do faktury sprzedaży

                                var zalW = new Soneta.Business.Db.Attachment(dok,
AttachmentType.Attachments);

                bm.Attachments.AddRow(zalW);

                    streamW.Seek(0, System.IO.SeekOrigin.Begin);

                zalW.LoadFromStream(streamW);

                    zalW.LoadIconFromFile(filepath);

                    zalW.Name = GenerateFileName(dok);

                    //Wiadomość robocza w CRM z załącznikiem - Faktura sprzedaży

                   crm.WiadomosciEmail.AddRow(wr);

                   wr.Tresc.Add(crm.SzablonyEmail.WgNazwa["Faktura"].Tresc);


                if (zm.Config.Operatorzy.Aktualny.DomyslneKontoPocztowe != null)
                                {
                                        wr.KontoPocztowe =
zm.Config.Operatorzy.Aktualny.DomyslneKontoPocztowe;

wr.Tresc.Add(zm.Config.Operatorzy.Aktualny.NowaWiadomoscPodpis.Podpis);
                                        }
                                                if (dok.Kontrahent.Kontakt.EMAIL != null)
                                        {
                                               wr.Do = dok.Kontrahent.Kontakt.EMAIL;
                                        }

                                              wr.Temat = "Faktura" + dok.Numer.ToString();

                          wr.Tresc = "Witam serdecznie,<br><br>W załączniku przesyłam fakturę";


                         var zal1 = new Soneta.Business.Db.Attachment(wr,
AttachmentType.Attachments);

                        bm.Attachments.AddRow(zal1);

                        streamW.Seek(0, System.IO.SeekOrigin.Begin);

                            zal1.LoadFromStream(streamW);
```

```
                            zal1.LoadIconFromFile(filepath);

                         zal1.Name = GenerateFileName(dok);


                        if(wr.TypWiadomosci == TypWiadomości.Robocza)

                        {

                                 System.Console.WriteLine(@"Wysyłka wiadomości z załącznikiem {0}.",
dok.Numer.NumerPelny);

                                 MailHelper.SendMessage(wr);
                                 wr.TypWiadomosci = TypWiadomości.Wysłana;




                                 }
                                 else {

                                 System.Console.WriteLine(@"Błędna wysyłka!!!");

                                   }
                                  tran.Commit();
                                  }

                                  }
                                  else {

                                  System.Console.WriteLine(@"Dokument Handlowy o nr {0} posiada już
załącznik.", dok.Numer.NumerPelny);

                                  }
                                   session.Save();

                                     }
                                     }

                            catch(Exception ex) {

                               LogMessage_d4340c01615e462d8c936553a9f0c729(ex.Message,
ConsoleColor.Red);

                                       }

                          }

                      } else {

                            System.Console.WriteLine("Brak dokumentów handlowych");
```

```
                }
}
```

Dodatkowo zostały stworzone metody prywatne

```
private static void LogMessage_d4340c01615e462d8c936553a9f0c729(string message, ConsoleColor?
consoleColor = null){
    var par = Factories.MakeSchedulerLoggerServiceParams();
    if (par == null)
    {
        return;
    }
    par.LogToConsole = true;
    par.SystemLogCategory = "Scheduler";
    par.Message = message;
    par.ConsoleColor = consoleColor;

    Factories.MakeSchedulerLoggerService().Log(par);
}

private static void SaveOnDisk(string filepath, string directory, System.IO.Stream streamW)
{
            System.IO.Directory.CreateDirectory(directory);

            var file = System.IO.File.Create(filepath);

            streamW.Seek(0, System.IO.SeekOrigin.Begin);

            Soneta.Tools.CoreTools.StreamCopy(streamW, file);

            file.Flush();


                            if (!System.IO.File.Exists(filepath))
                                throw new Exception("Nie udało się wygenerować faktury");
}

private static string GenerateFileName(DokumentHandlowy dok)
{
  string code = dok.Kontrahent.Kod;
    string fvid = String.Join("",
dok.NumerPelnyZapisany.Split(System.IO.Path.GetInvalidFileNameChars()));
    string stamp = DateTime.Now.ToString("ddMMyyyyHHmmss");
    string ext = ".pdf";

    return string.Format("{0}_{1}_{2}{3}", code, fvid, stamp, ext);
```

```
}
```

Więcej informacji odnośnie tasków globalnych jak i Harmonogramów można znaleźć pod linkiem: https://dok.enova365.pl/Topic/6318

Do poprawnego działania taska należy w bazie danych:

1. skonfigurować harmonogram zadań,
2. skonfigurować w enova domyślne konto pocztowe i przypisać je do operatora harmonogram zadań i np Administrator,
3. dodać kilka dokumentów z datą o jeden dzień do tyłu,
4. dodać nowy szablon wiadomości email,
5. dla aspx dodać plik asxp do bazy danych lub dla repx dodać odpowiednio poprawiony plik do wzorców użytkownika (poprawka zgodnie z artykułem:

https://dok.enova365.pl/Partners/35509

Mnożenie w kolumnie przez współczynnik z parametrów w raporcie ASPX

Przykład raportu opakowań, który aby uruchomić można podpiąć do listy faktur sprzedaży.

Poniżej część kodu C# z raportu, który realizuje taką funkcjonalność.

```
public class ParametryContext : ContextBase
            {
                public ParametryContext(Context cx)
                    : base(cx)
                {
                }


                private double wsp = 0;
                [Caption("Współczynnik")]
                public double Wsp
                {
                    get { return wsp; }
                    set { wsp = value; }
                }
            }

            ParametryContext parametry;

            [Context]
            public ParametryContext Parametry
            {
                get { return parametry; }
                set { parametry = value; }
            }

            public class Total: IComparable {
                readonly Towar towar;

                double Wspolczynnik;      <- Dodanie pola Wspolczynnik

                double ilość;
                double ilkarton;
                double ubkarton;
                double rkarton;
                decimal netto;
                public Total(Towar towar,double Ws) {
                    this.towar = towar;
                Wspolczynnik = Ws;        <- Rozbudowa Konstruktora
```

```
                }
```

```
SumaPozycjiWorker spw = new SumaPozycjiWorker();

                public void Add(PozycjaDokHandlowego pozycja)
                {


                    spw.Pozycja = pozycja;
                    netto += spw.Netto;
                    ilość += pozycja.Ilosc.Value;
                    ilkarton += pozycja.ZmianaMagazynu.Ilość.Value;
                    ubkarton += pozycja.ZmianaMagazynu.Ilość.Value * 0.02;
                }
                public Towar Towar {
                    get { return towar; }
                }
                public decimal Netto {
                    get { return netto; }
                }
                public double Ilość {
                    get { return ilość*Wspolczynnik; }  <- Mnożenie ilości przez zmienna Wspolczynnik
                }
                public double Ilkarton {
                    get { return ilkarton; }
                }
                public double Ubkarton {
                    get { return ubkarton; }
                }
                public double Rkarton {
                    get { return System.Math.Round(ilkarton + ubkarton,
MidpointRounding.AwayFromZero); }
                }

                public int CompareTo(object obj) {
                    Total t = (Total)obj;
                    return Towar.Nazwa.CompareTo(t.Towar.Nazwa);
                }
            }
```

```
void OnContextLoad(Object sender, EventArgs args) {
                Soneta.Handel.Forms.DokHandloweViewInfo.WParams pars =
```

```
(Soneta.Handel.Forms.DokHandloweViewInfo.WParams)dc[typeof(Soneta.Handel.Forms.DokHandloweViewInfo.WParams)];
                if (pars.Okres != Soneta.Types.FromTo.All)
                    ReportHeader.Title += "|Okres: <strong>" + pars.Okres.ToString() + "</strong>";
                if (pars.Kontrahent != null)
                    ReportHeader.Title += "|Kontrahent: <strong>" + pars.Kontrahent.ToString() + "
</strong>";
                if (pars.Definicja!=null)
                    ReportHeader.Title += "|Definicja: <strong>" + pars.Definicja.ToString() + "
</strong>";

                Row[] rows = (Row[])dc[typeof(Row[])];
                Hashtable result = new Hashtable();
                foreach (DokumentHandlowy dokument in rows) {
                    if (dokument.Definicja.DuplikatWartosci || dokument.Anulowany) continue;
                    if (dokument.Definicja.EdycjaWartosci ==
EdycjaWartosciDokumentu.PozwalajNaMniejsząKwotę) continue;
                    foreach (PozycjaDokHandlowego pozycja in dokument.Pozycje)
                    {
                        Total t = (Total)result[pozycja.Towar];
                        if (t==null) {
                            t = new Total(pozycja.Towar,parametry.Wsp); <- dodanie do konstruktora
parametry.Wspin

                            result.Add(t.Towar, t);
                        }
                        t.Add(pozycja);
                    }
                }

                ArrayList lista = new ArrayList(result.Values);
                lista.Sort();
                Grid.DataSource = lista;
            }

            public void Grid_BeforeRow( object sender, RowEventArgs args ) {
                Total t = ( Total ) args.Row;
                //if((double)t.Towar.Features["Przelicznik karton"] == 0) {
                //      args.VisibleRow = false;
                //}
                }
```

Pliki do pobrania

[raport opakowania Soneta1.aspx (7.87 KB)]

Mechanizmy komunikacji z bazami danych

W skład grupy *Mechanizmy komunikacji z bazami danych* wchodzą wszystkie artykuły zawierające wiedzę dotyczącą komunikacji z bazami danych.

Session.Save w szczegółach

W artykule opisany został szczegółowa proce zapisywania sesji do bazy danych. Przedstawienie dokładnej kolejności poszczególnych zadań pozwala na głębsze zrozumienie działania biblioteki Soneta. Dzięki temu możliwe jest tworzenie bardziej optymalnego kodu programu. Pozwala to również uniknąć pewnych podstawowych błędów programistycznych.

Stan przed - stan po

Wszystkie zmiany w obiektach biznesowych muszą zostać wykonane przed operacją `Session.Save()`. Tylko przed zapisaniem sesji edycyjnej pozwala ona na dodawanie, edycję i kasowanie obiektów.

Po udanym wykonaniu zapisania sesji przechodzi ona w tryb *read-only* i przestaje się nadawać do wykonywania jakichkolwiek obliczeń, zmian, itp. Sesja po zapisaniu przestaje być tak naprawdę użyteczna, a obiekty należy wykorzystywać tylko do odzyskania ich identyfikatorów `Row.ID` (dotyczy dodanych obiektów), które przypisywane są dopiero podczas dodawania rekordów do bazy danych. Przed ich dodaniem (zapisaniem) identyfikatory mają tymczasową wartość ujemną.

Żeby sprawdzić, czy sesja jest w stanie po zapisaniu należy użyć property `bool Session.IsSaved`. Wywołanie metody `Save()`, gdy `IsSaved` jest równe `true` zostanie zakończone wyjątkiem.

Jeżeli podczas zapisywania sesji wystąpią jakiekolwiek wyjątki `Exception`, sesja zostanie przywrócona do stanu przed wywołaniem metody `Save()`. Będzie można dokonywać zmian obiektów tak jakby zapisanie się nie odbyło. Będzie możliwe również ponowna próba zapisania sesji do bazy danych.

Metody `Save()` nie można również wywoływać, gdy otwarte są jakiekolwiek transakcje biznesowe. Wszystkie transakcje biznesowe muszą być wcześniej zamknięte. Jest to powód, dla którego nie jest możliwe zapisywanie danych w podformularzu, ponieważ podformularz otwarty jest we własnej transakcji biznesowej. Następujący kod zostanie zakończony wyjątkiem:

```
Session session = ....;
using (ITransaction t = session.Logout(false)) {
  session.Save(); // <= Exception
}
```

Transakcja bazy danych

Duża część zapisywania danych sesji jest realizowana w transakcji bazodanowej. Dzięki temu sesja zostaje zapisana w bazie tylko w całości. Nie jest możliwe zapisanie tylko części rekordów. Wykorzystanie tego mechanizmu wymaga trzymania się pewnych założeń.

- Transakcja bazy danych jest otwierana tylko na krótki okres czasu. Dlatego nie jest to dobre miejsce na dokonywanie długotrwałych obliczeń.
- W trakcie transakcji nie można wyświetlać żadnych okien dialogowych (interface WinForms), ani realizować innych iterakcji z operatorem. Może to wydłużyć czas otwarcia transakcji w sposób niekontrolowany.
- Jest to dobre miejsce do modyfikacji tzw wspólnych zasobów programowych (numerowanie dokumentów, zliczanie sum, itp)
- Wszystkie obiekty zmieniane w transakcji są blokowane na włączność. Pozostałe stanowiska zostaną zablokowane jeżeli również będą próbowany zmieniać te same obiekty. Dlatego należy dbać by modyfikować tylko niezbędne rekordy.

- Wszystkie obiekty, które zostaną tylko odczytane w transakcji również zostają zablokowane. Dzięki temu mamy pewność poprawnie policzonych sum, itp. Należy dbać by w transakcji odczytywać tylko obiekty, które musimy odczytać.

Powtarzanie zapisu transakcyjnego

Wszystkie operacje biznesowe wykonane w transakcji bazy danych mogą spowodować zablokowanie operacji. Dzieje się tak, gdy na innym stanowisku modyfikowane są te same dane `DeadLockSqlException`. Zapis sesji został wyposażony w mechanizm ponownej próby zapisu danych. Jeżeli wystąpi zablokowanie operacji wszystkie zmiany dokonane w transakcji bazy danych zostaną wycofane, sama transakcja zostanie wycofana `Abort` i zostanie podjęta próba ponownego zapisania danych od momentu otwarcia transakcji bazy danych. Program podejmie 4 takie próby zanim zgłosi wyjątek.

Wywołanie Session.Save wewnątrz transakcji bazy danych

Jest możliwe zapisywanie sesji podczas otwartej transakcji bazy danych. Dzięki temu możliwe jest przetwarzanie danych w jednej transakcji, ale wielu sesjach, co jest szczególnie przydatne podczas operacji seryjnych na wielu rekordach. Pozwala to na optymalizację ilość obiektów przetwarzanych w jednej sesji. Wykorzystując tą technikę należy jednak pamiętać, że zostaje wyłączony mechanizm powtarzania zapisów do bazy w celu uniknięcia blokowania zapisów. Dzięki tej technice możliwe jest równoległe przetwarzanie zapisów w różnych wątkach. Pewne grupy rekordów przetwarzamy w osobnych wątkach, w różnych sesjach (np po jednej sesji na wątek). Ale zapisanie danych z tych sesji odbywa się już w jednej transakcji w wątku głównym.

Przykładowy kod:

```
//
// Przetwarzanie w sesjach, może odbywać się w innych wątkach.
// Każda sesja w osobnym wątku.
//
Session session1 = ....;
Session session2 = ....;

//
// Transakcja bazy danych; zapis sesji z różnych wątków musi odbywać się w jednym wątku
//
using (Connection cn = Database.OpenConnectionWithTransaction(DatabaseType.Operational)) {
  //
  // Zapisywanie sesji w jednej transakcji bazy danych
  //
  session1.Save();
  session2.Save();

  //
  // Przetwarzanie całej sesji w transakcji bazy danych
  //
  using (Session session3 = Login.CreateSession(...)) {
    //
    // Tutaj Przetwarzanie w sesji
    //

    session3.Save();
```

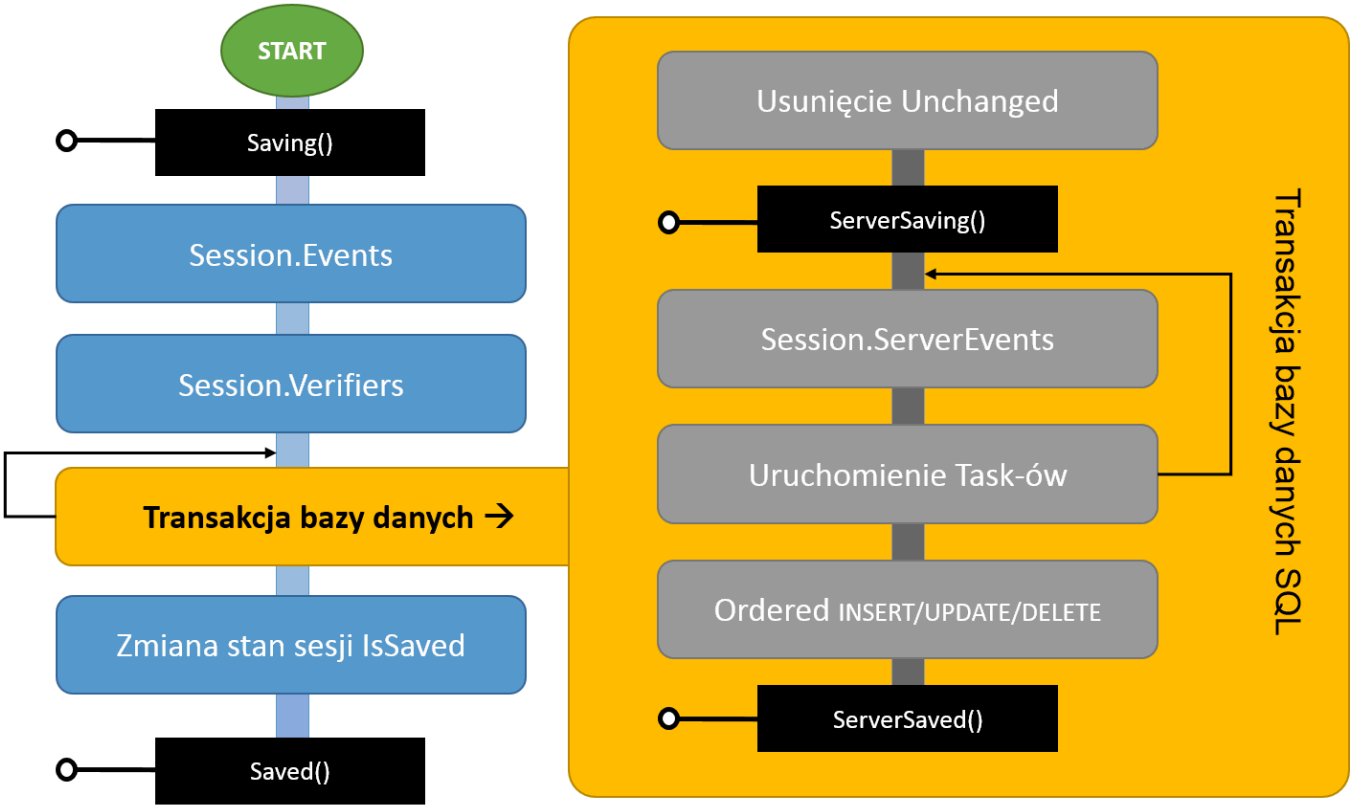
```
  }

  cn.Commit();
}

//
// Niezapomnieć o zwolnieniu zasobów
//
session1.Dispose();
session2.Dispose();
```

Kolejne kroki Session.Save()

1. Wywołanie serwisu `ISessionListener.Saving()`
2. Odpalenie zdarzeń nieserwerowych `Session.Events`
3. Weryfikacja weryfikatorów typu Error `Session.Verifiers`
4. Otwarcie transakcji bazodanowej `LOGOUT`
5. Zwolnienie obiektów niezmienionych `RowState.Unchanged`
6. Wywołanie serwisu `ISessionListener.ServerSaving()`
7. Odpalenie zdarzeń serwerowych `Session.ServerEvents`
8. Odpalenie zdarzeń Tasks
9. Weryfikacja weryfikatorów typu Error `Session.Verifiers`
10. Ewentualne powtórzenie kroków 7, 8 i 9
11. Określenie kolejności zapisu rekordów
12. Zapisanie rekordów do bazy danych
13. Wywołanie serwisu `ISessionListener.ServerSaved()`
14. Zatwierdzenie transakcji bazy danych `COMMIT`
15. Przestawienie stanu sesji na `IsSaved`
16. Wywołanie serwisu `ISessionListener.Saved()`



1. Wywołanie serwisu `ISessionListener.Saving()`

Dzięki serwisowi `ISessionListener` dodatki Soneta mają możliwość wpływania na zapisywanie sesji. Jednak jest to metoda pozwalająca na wykonanie szczególnych zadań. Lepszym rozwiązaniem jest użycie zdarzeń `Session.Events`.

2. Odpalenie zdarzeń nieserwerowych `Session.Events`

W trakcie pracy z obiektami sesji rejestrowane mogą być *events* (zdarzenia), pozwalające na ograniczenie ilości operacji biznesowych po zmianie properties. Standardowym miejscem, w którym wszystkie zarejestrowane events są odpalane jest zatwierdzenie transakcji biznesowej `ITransaction.CommitUI()`. Poza tym mogą być również uruchomione ręcznie przy użyciu instrukcji `Session.Events.Invoke()`.

Jednak może się zdarzyć, że nie wystąpią te sytuacje i zarejestrowane events nie zostaną odpalone. Dlatego dla zapewnienia ich wykonania uruchomione zostaną w trakcie zapisania sesji.

Uruchomienie events odbywa się poza transakcją bazodanową, podobnie jak to się ma w przypadku innych wywołań.

3. Weryfikacja weryfikatorów typu Error `Session.Verifiers`

Ostatnią operacją przed rozpoczęciem transakcji bazodanowej jest weryfikacja zarejestrowanych *weryfikatorów*. W tym kroku nie ma możliwości iterakcji z użytkownikiem, dlatego wszystkie weryfikatory typu *warning* oraz *information* zostaną zignorowane. Przeliczone zostaną tylko weryfikatory typu *error*, żeby niedopuścić do zapisu danych, które nie spełniają wszystkich wymaganych weryfikatorów.

W przypadku, gdy któryś weryfikator typu *error* nie zostanie spełniony zapis sesji zostanie przerwany odpowiednim wyjątkiem.

4. Otwarcie transakcji bazodanowej `LOGOUT`

W tym kroku następuje otwarcie transakcji bazy danych `BEGIN TRANSACTION`. Przed otwarciem transakcji ustanawiane jest nowe połączenie *Connection* do bazy danych. W przypadku potrzeby powtórzenia operacji zapisu danych spowodowanym np blokadą danych, operacje będą powtarzane właśnie od otwarcia transakcji.

Razem z otwarciem transakcji zmieniany jest poziom izolacji baz danych na `TRANSACTION ISOLATION LEVEL SERIALIZABLE`. Oznacza to, że wszystkie operacje odczytu danych blokują odczytane dane oraz zakresy danych tak, że inne wątki nie będą miały dostępu do tych danych. Mamy gwarancję, że dane nie zostaną zmienione do końca operacji zapisu,. Jednak należy pamiętać o tym, że może to powodować blokadę działania innych operatorów.

W przypadku, gdy metoda `Session.Save()` wywołana jest w wewnątrz zewnętrznej transakcji bazy danych, nie jest generowana żadna instrukcja otwierająca transakcję bazy danych.

5. Zwolnienie obiektów niezmienionych `RowState.Unchanged`

Obiekt sesji zarządza danymi (obiektami) wczytanymi z bazy danych. Zawiera wiele różnych obiektów, które są zmienione, dodane, skasowane, ale również tylko wczytane *niezmienione*. Podczas wykonywania kolejnych operacji na obiektach mamy mieć gwarancję, że stan naszych obiektów jest taki sam jak obiektów w bazie danych. Nikt inny nie może w tym samym czasie zmieniać tych danych. Dlatego wszystkie obiekty, które zostały tylko wczytane, a nie zostały zmienione zostają usunięte z sesji. Ponowne odwołanie się do tych obiektów spowoduje ich ponowne wczytanie z bazy. Oznacza to, że dane w tych obiektach mogą się zmienić.

To samo dotyczy zakresów wczytanych danych. Jeżeli przed rozpoczęciem zapisu danych pewien zakres danych było wczytany, ponowne odwołanie się do tego zakresu będzie wymagało jego ponowne wczytanie z bazy danych.

Ponowne wczytywanie danych oraz zakresów danych w transakcji bazy danych ma jeszcze jedną konsekwencję. Powoduje również ustanowienie blokady na te dane, ze względu na podwyższony poziom izolacji `SERIALIZABLE`. Przykładowo: wyliczenie sumy pewnego zbioru danych przed zapisem danych może zwrócić inną wartość, niż wewnątrz transakcji. Jednak o ile podczas wyliczania pierwszej sumy nie ma gwarancji co do zmian dokonanych na innych stanowiskach, to podczas wyliczania sumy w transakcji mamy gwarancję, że suma ta nie zostanie zmieniona do końca transakcji. Oczywiście konsekwencją jest to, że na innym stanowisku nie będzie możliwe wyliczenie tej samej sumy. Dane zostaną zablokowane.

Ze względu na blokowanie danych należy dokonać szczególnych starań, żeby skrócić czas trwania transakcji. Im krótszy czas, tym mniejsze prawdopodobieństwo kolizji z innymi operatorami. Jest to też powód, dla którego w transakcji nie można wykorzystywać iterakcji z użytkownikiem, nie można wyświetlać formularzy, okien dialogowych, komunikatów, itp.

6. Wywołanie serwisu `ISessionListener.ServerSaving()`

Dzięki serwisowi `ISessionListener` dodatki Soneta mają możliwość wpływania na proces zapisywania sesji. Jednak jest to metoda pozwalająca na wykonanie szczególnych zadań. Lepszym rozwiązaniem jest użycie zdarzeń `Session.ServerEvents`, które zostaną odpalone w kroku 7.

7. Odpalenie zdarzeń serwerowych `Session.ServerEvents`

Podobnie do zdarzeń *Session.Events* można rejestrować zdarzenia serwerowe `Session.ServerEvents`. Jest to mechanizm dedykowany do wykonywania obliczeń wewnątrz transakcji bazy danych. Jedną z najważniejszych tych zdarzeń jest to, że mają wyłączność na odczytywane dane.

Cały czas istnieje możliwość używania mechanizmu *Session.Events* - nieserwerowych. Każde zarejestrowanie takiego event-u, jest tak naprawdę zarejestrowaniem zdarzenia serwerowego. Na czas tego i następnych kroków zapisu danych, *Events* oraz *ServerEvents*, to jednak kolekcja.

8. Odpalenie zdarzeń Tasks

Kolejnym mechanizmem są taski, czyli zdarzenia wywoływane po zmianie obiektów biznesowych pewnego typu. Programując taski należy pamiętać, że nie należy wykorzystywać iterakcji z użytkownikiem, gdyż może to wydłużyć w sposób niekontrolowany czas trwania trwania transakcji.

9. Weryfikacja weryfikatorów typu Error `Session.Verifiers`

Ponowna weryfikacja weryfikatorów typu *error*. W procedurach *ServerEvents* oraz *tasks* dokonywane są zmiany danych. Może to powodować pojawienie się kolejnych weryfikatorów. Wszystkie weryfikatory typu *error* zostaną tutaj policzone i w przypadku niepowodzenia zostanie wygenerowany wyjątek *exception* i proces zapisywania danych zostanie przerwany. Weryfikatory typu *warning* oraz *information* są ignorowane.

10. Ewentualne powtórzenie kroków 7, 8 i 9

Odpalenie tasków oraz przeliczanie weryfikatorów może spowodować pojawienie się kolejnych zdarzeń serwerowych *ServerEvent*. Jeżeli nowe zdarzanie zostaną zarejestrowane, powtarzane są wszystkie kroki od odpalenia zdarzeń serwerowych. Dalsze przetwarzanie będzie odbywać się po

warunkiem, że żadnych zdarzeń nie będzie zarejestrowanych, nie będzie kolejnych nowych tasków do odpalenia oraz wszystkie weryfikatory typu *error* zostaną zweryfikowane pozytywnie.

11. Określenie kolejności zapisu rekordów

Obiekty mogą być modyfikowane w dowolnej kolejności. System musi zapewnić,, żeby po mimo różnej kolejności tworzenia, kasowanie i modyfikowania obiektów biznesowych zapewnić ich poprawny zapis do bazy danych. Podstawowy algorytm powoduje zapisuje zmiany w następującej kolejności:

1. `DELETE` - najpierw kasowane są wszystkie obiekty, które można skasować.
2. `UPDATE` - następnie modyfikowane są wartości pól obiektów ze zmienionymi wartościami.
3. `INSERT` - ostatnim krokiem jest dodanie nowo utworzonych obiektów (rekordów) do bazy danych.

Ne wszystkie operacje można wykonać w pierwszym przebiegu. Przykładowo, nie można skasować rekordu, gdy istnieje inny rekord w bazie danych zawierający relację *ID* do kasowanego rekordu. Nie można również dodawać nowego rekordu, gdy zawiera on relację *ID* rekordu jeszcze niedodanego. W takim przypadku zapisanie rekordu zostanie wstrzymane do kolejnego przebiegu algorytmu.

Algorytm powtarza cykl `DELETE-UPDATE-INSERT` dopóki istnieją niezapisane rekordy lub w danym cyklu nastąpiły zmiany.

Istnieją również relację, którym algorytm zapisywania nie jest w stanie rozwiązać. Wtedy cykl `DELETE-UPDATE-INSERT` nie zapisuje kolejnych rekordów pomimo, że nie wszystkie są zapisane. Wtedy generowany jest wyjątek *exception*, a proces zapisywania sesji zostaje przerwany.

Przykładem takich zapisów jest dodanie relacji cyklicznej. Przykładowo dodajemy dwa rekordy *A* oraz *B*. Obiekt *A* zawiera referencję do obiektu *B*, a obiekt *B* zawiera referencję do obiektu *A*. W tym przypadku nie jest możliwe ani dodanie, ani skasowanie takich rekordów w jednym kroku. Do zapisania rekordu *A* potrzebujemy znać *ID* obiektu *B*, które zostanie określone po jego zapisaniu. Jednak do zapisania rekordu *B* również jest potrzebne *ID* rekordu *A*, który jest właśnie zapisywany. W takim przypadku należy użyć dwóch niezależnych sesji. Oczywiście cały proces zapisywania jest znacznie bardziej skomplikowany.

Innym przypadkiem zapisywania danych, które może się nie udać jest renumeracja rekordów *LP*, gdy wartość pola *LP* jest unikalna.

12. Zapisanie rekordów do bazy danych

Po określeniu kolejności rekordów do zapisania sam proces wysłania rekordów do bazy danych, to już tylko formalność. Dla rekordów wykonywane są odpowiednio instrukcje SQL:

- nowe rekordy - instrukcje `INSERT` dodające rekord do bazy. W instrukcji wyszczególnione są wszystkie pola rekordów, nawet te ze standardowymi wartościami. Należy pamiętać, że dopiero po wykonaniu tej instrukcji obiektowi przypisywany jest *ID* przez bazę danych. Do tego momentu obiekt posiadał tymczasowy identyfikator, ujemną wartość. Dlatego jeżeli chcemy odczytać ID nowo dodanego obiektu, to możemy to zrobić tak naprawdę dopiero po tym kroku.
- zmienione rekordy - instrukcje `UPDATE` zmieniające dane w bazie zawierają tylko pola zmienione w rekordzie. Pola niezmienione nie zostaną wyszczególnione. Rekord identyfikowany jest przez pole `ID`, które zostanie odpowiednio umieszczone w sekcji `WHERE`, a także przez pole `Stamp` sprawdzające wersję rekordu w bazie danych. Jeżeli pole *Stamp* nie będzie się zgadzało zostanie wygenerowany wyjątek `Exception` o możliwych danych zmianach na innym stanowisku. Jest to tzw mechanizm lokowania optymistycznego.
- usuwanie rekordów - instrukcje `DELETE` usuwające dane z bazy. Podobnie jak w przypadku zmiany, rekord identyfikowany jest przez pole `ID`, które zostanie odpowiednio umieszczone w

sekcji `WHERE`, a także przez pole `Stamp` sprawdzające wersję rekordu w bazie danych. Jeżeli pole *Stamp* nie będzie się zgadzało zostanie wygenerowany wyjątek `Exception` o możliwych danych zmianach na innym stanowisku. Jest to tzw mechanizm lokowania optymistycznego. Żadne inne pola nie są wyszczególnione w zapytaniu usuwania.

Na samym końcu wysyłane są instrukcje SQL odpowiedzialne na aktualizację wartości cech obiektów w tabeli *Features*.

13. Wywołanie serwisu `ISessionListener.ServerSaved()`

Dzięki serwisowi `ISessionListener` dodatki Soneta mają możliwość wpływania na zapisywanie sesji. Jednak jest to metoda pozwalająca na wykonanie szczególnych zadań. Np wykonania zapytań SQL jeszcze w transakcji bazy danych, ale już po zapisaniu wszystkich obiektów biznesowych. Jest to ostatnie miejsce, w którym może wystąpić wyjątek, który wycofa stan sesji oraz zapisane w bazie danych rekordy.

14. Zatwierdzenie transakcji bazy danych `COMMIT`

Wysłanie instrukcji SQL zatwierdzającej dane zmienione wewnątrz transakcji bazy danych `COMMIT TRANSACTION`. Po zatwierdzeniu transakcji zmieniony jest również poziom izolacji na `TRANSACTION ISOLATION LEVEL SNAPSHOT` (lub w poprzednich wersjach SQL `TRANSACTION ISOLATION LEVEL READ COMMITTED`), który jest standardowo wykorzystywany podczas odczytywania nowych danych przez obiekty biznesowe.

W przypadku, gdy metoda `Session.Save()` wywołana jest w wewnątrz zewnętrznej transakcji bazy danych, nie jest generowana żadna instrukcja zatwierdzająca transakcję bazy danych.

15. Przestawienie stanu sesji na `IsSaved`

Przestawienie stanu flagi `Session.IsSaved` na wartość `true` oraz przestawienie innych parametrów obiektów biznesowych, jak `AccessRight` oraz `ReadOnly` zabezpieczających przed pomyłkowym używaniem obiektów z zapisanej sesji w dalszym przetwarzaniu.

Sesja po zapisaniu nie nadaje się już do dalszej pracy i należy jak najszybciej zwolnić zajmowane przez nią zasoby za pomocą `Session.Dispose()`. Po zapisaniu obiekty sesji mogą być wykorzystane do odczytania ich nowych *ID* (w przypadku obiektów dodanych) lub do uzyskania odpowiednich obiektów w innych sesjach.

16. Wywołanie serwisu `ISessionListener.Saved()`

Dzięki serwisowi `ISessionListener` dodatki Soneta mają możliwość wpływania na zapisywanie sesji. Jednak jest to metoda pozwalająca na wykonanie szczególnych zadań. Np wykonanie zapytań SQL zaraz po zatwierdzeniu transakcji bazy danych. Należy pamiętać, że jeżeli w tym miejscu wystąpi wyjątek, to stan sesji nie zostanie wycofany, a zapisane w bazie danych rekordy pozostaną.

Narzędzie diagnostyczne - okno deweloperskie

Zapisywanie danych przez sesję jest skomplikowaną procedurą. Dlatego nie trudno o napisanie nieoptymalnego kodu powodującego wolne działanie procedury zapisu. Do znajdowania miejsc w kodzie programu powodujących problemy w sesji znajduje się kilka miejsc badających szybkości operacji, które można przeglądać w oknie deweloperskim.

- *Save* - całkowity czas pracy całej procedury zapisu sesji.

- *Save finishing* - czas wykonania kroków po zatwierdzeniu transakcji bazy danych, czyli kroków: 15 oraz 16.
- *Save remove unused* - czas działania procedury usuwającej z sesji niezmodyfikowane dane, krok 5.
- *Save tasks* - czas wykonania task-ów, krok 8.
- *Save order rows* - czas pracy procedury określającej kolejność zapisywania rekordów, element kroku 11.
- *Save execute sql* - czasy wysłania do serwera bazy danych instrukcji aktualizujących dane, krok 12.

Wprowadzenie

Mechanizm Query służy do wykonywania zapytań agregujący wprost na serwerze bazy danych. Zapytania takie formułuje się za pomocą obiektów i wykonuje na Session. Posługiwanie się większym poziomem abstrakcji niż zapytania SQL pozwala na wykonywanie ich na różnych platformach bazodanowych obsługiwanych przez program, natomiast fakt, że agregacja danych wykonuje się na serwerze znacznie zwiększa wydajność, gdyż dane nie muszą być w całości ładowane na klienta, nie jest inicjowana logika biznesowa i klient nie wykonuje operacji agregujących.

Tworzenie Query

Załóżmy, że chcemy uzyskać sumę wszystkich opisów analitycznych za cały rok 2014 o symbolu zaczynającym się literą 'a'

```
//określamy tabelę, dla której chcemy utworzyć zapytanie
var opisyQuery = new Query.Table("OpisAnalityczny") { Distinct = true };

//określamy funkcję agregujacą, do jakiej właściwości ma być zwrócona wartość agregacji
//oraz pole w bazie podlegająca agregacji
opisyQuery += new Query.Sum(new Query.Field("Kwota.Value") { PropertyName = "Kwota" });

//określamy warunki jakie mają spełniać agregowane dane przy pomocy wyrażenia RowCondition
opisyQuery.Condition =
        new FieldCondition.Contain("Data", new FromTo(new Date(2014,1,1), new FromTo(new
Date(2014,12,31)))
        & new FieldCondition.Like("Symbol", "a*");
```

Tworzenie klasy będącej wynikiem wykonania Query

Zdefiniowane wyżej Query zakłada zwrócenie danych z jedną właściwością opisaną przez *PropertyName* o nazwie 'Kwota'. Musimy przygotować taką klasę, nie musi być ona publiczna, byle miała bezparametrowy konstruktor i omawianą wcześniej właściwość.

```
private class OpisyQueryResult
{
        public decimal Kwota { get; set; }
}
```

Wykonanie Query

Wykonanie Query wymaga dostępu do sesji i wykonania na niej metody *Execute*. W tym przypadku metoda zwróci *IEnumerable*

```
var results = session.Execute<OpisyQueryResult>(opisyQuery);
```

W tym przypadku zapytanie nie ma żadnego grupowania i zwróci jeden rekord, możemy jednak naszego enumerable'a na wszelki wypadek podsumować

```
var suma = results.Sum(oq => oq.Kwota);
```

Definiowanie pól z bazy danych

```
opisyQuery += new Query.Field("Kwota.Value") { PropertyName = "PoleKwotaValue" };
```

Funkcje agregujące

Query.Count

```
opisyQuery += new Query.Count("ID"){ PropertyName = "Ilosc" };
```

Query.Sum

```
opisyQuery += new Query.Sum("Kwota.Value"){ PropertyName = "SumaKwot" };
```

Query.Avg

```
opisyQuery += new Query.Avg("Kwota.Value"){ PropertyName = "SredniaKwot" };
```

Query.Min

```
opisyQuery += new Query.Min("Kwota.Value"){ PropertyName = "MinimalnaKwota" };
```

Query.Max

```
opisyQuery += new Query.Max("Kwota.Value"){ PropertyName = "MaksymalnaKwota" };
```

Stałe

Query.Number

```
opisyQuery += new Query.Number(20m){ PropertyName = "Kwota20" };
```

Operacje arytmetyczne

Query.Plus

```
opisyQuery += new Query.Plus(new Query.Field("Kwota.Value), new Query.Number(20m))
        {
                PropertyName = "KwotaPowiekszonaO20"
        };
```

Query.Minus

```
opisyQuery += new Query.Minus(new Query.Field("Kwota.Value), new Query.Number(20m))
        {
                PropertyName = "KwotaPomniejszonaO20"
        };
```

Query.Mul

```
opisyQuery += new Query.Mul(new Query.Field("Kwota.Value), new Query.Number(20m))
        {
                PropertyName = "KwotaPomnozonaPrzez20"
        };
```

Query.Div

```
opisyQuery += new Query.Div(new Query.Field("Kwota.Value), new Query.Number(20m))
        {
                PropertyName = "KwotaPodzielonaPrzez20"
        };
```

Funkcje dodatkowe#

Query.Round

```
opisyQuery += new Query.Round(new Query.Field("Kwota.Value") { PropertyName = "KwotaZPrecyzja0" },
0);
```

Mechnizm Query i Join kolumn

Query pozwala na konstruowanie zaopytań także z uwzględnieniem Joinów. Mamy do dyspozycji szereg operacji, które definiujemy zgodnie z zasadamy SQL podając na czym samo łączenie się odbywa.

```
            Query.Union union = new Query.Union();

            var operacjePozycjeTechn_1 = new Query.LeftOuterJoin("Operacje", new
FieldCondition.Equal("ID", pozycjeTechn_1, "Operacja"));

             var technologiePozycjeTechn_1 = new Query.InnerJoin("Technologie", new
FieldCondition.Equal("ID", pozycjaTechnRezerwacjaSurowca_1, "Technologia"));
```

Mechanizm Query a obiekty typu IRow

Query posiada mechanizm rozwiązujący relacje wynikające z istnienia kluczy obcych. Jeśli wynikiem zapytania będzie kolumna która odnosi się bezpośrednio do obiektu biznesowego IRow (np. Towar), to mechanizm analizujący wyniki zapytania Query spróbuję je "dowiązać" i w efekcie zwrócić nam obiekt biznesowy w pełni sesyjny. Przykład obrazujący mechanizm:

Pomocnicza klasa

```
public class TowarTest {
    public Towar towar { get; set; }
}
```

Tworzymy sobie proste Query i możemy potem działać na zwróconym obiektach typu Towar jak na pełnoprawnych obiektach IRow.

```
        Query.Table towary = new Query.Table("Towary");

        towary += new Query.Field("ID") { PropertyName = nameof(TowarTest.towar) };
        using (Session session = Login.CreateSession(false, false)) {
            var result = session.Execute<TowarTest>(towary);
            using (var trans = session.Logout(true)) {
                foreach (var towar in t) {
                    towar.towar.Nazwa = towar.towar.Nazwa + "_test";
                }
                trans.Commit();
            }
            session.Save();
        }
```

Mechanizm Query w osobnych zapytaniach dopyta o poszczególne obiekty Towar na podstawie ID które uzyskał na z samych już rezultatów. Pozwolić to może ograniczyć ilość obiektów biznesowych potrzebnych do pobrania w trakcie realizacji jakiegoś zapytania. Doczytywane w ten

sposób obiekty są ograniczone zwracaną ilością wierszy, jednocześnie wykorzystywany tutaj będzie mechanizm doczytywania "na zapas" obiektów pod ich ID

Filtrowanie kolekcji za pomocą Condition

W skład grupy *Filtrowanie kolekcji za pomocą Condition* wchodzą wszystkie artykuły zawierające wiedzę o filtrowania danych.

Equal

Warunek ten umożliwia zawężanie kolekcji do obiektów, w których wybrane pole równe jest podanej wartości.

Konstruktory

```
/// path: Nazwa kolumny, po której chcemy filtrować
/// value: Wartość
public Equal(string path, object value) {}

/// path: Nazwa kolumny, po której chcemy filtrować
/// table2: Nazwa tabeli, w której znajduje się porównywany obiekt
/// path2: Nazwa kolumny, w której znajduje się porównywany obiekt
public Equal(string path, ITable table2, string path2) {}Poniższy przykład prezentuje w jaki sposób
załadować listę zadań kontrahenta "Abc".

var view = ZadaniaModule.GetInstance(args.Session).Zadania.CreateView();
var kontrahent = CRMModule.GetInstance(args.Session).Kontrahenci.WgKodu["Abc"];
view.Condition &= new FieldCondition.Equal("Kontrahent", kontrahent);
```

W kolejnym przykładzie pobrana zostanie lista zadań całodziennych.

```
var view = ZadaniaModule.GetInstance(args.Session).Zadania.CreateView();
view.Condition &= new FieldCondition.Equal("Calodzienne", true);
```

NotEqual

Warunek ten umożliwia zawężenie kolekcji do obiektów, w których wybrane pole jest różne od podanej wartości.

Sygnatury konstruktorów wyglądają analogicznie jak w klasie Equal.

In

Warunek ten umożliwia zawężanie kolekcji do obiektów, w których wybrane pole równe jest jakiemukolwiek obiektowi z kolekcji parametrów.

Konstruktor

```
/// path: Nazwa kolumny, po której chcemy filtrować
/// values: Kolekcja parametrów
public In(string path, params object[] values) {}
```

Poniższy przykład prezentuje w jaki sposób załadować listę zadań, w których kontrahentem jest kontrahent "Abc" lub "Hiacynt".

```
var view = ZadaniaModule.GetInstance(args.Session).Zadania.CreateView();
var kontrahent1 = CRMModule.GetInstance(args.Session).Kontrahenci.WgKodu["Abc"];
var kontrahent2 = CRMModule.GetInstance(args.Session).Kontrahenci.WgKodu["Hiacynt"];
view.Condition &= new FieldCondition.In("Kontrahent", kontrahent1, kontrahent2);
```

Aby zawęzić kolekcję do obiektów, które **nie znajdują** się w kolekcji parametrów, możemy skorzystać z zaprzeczenia **RowCondition.Not** jak poniżej.

```
var view = ZadaniaModule.GetInstance(args.Session).Zadania.CreateView();
var kontrahent1 = CRMModule.GetInstance(args.Session).Kontrahenci.WgKodu["Abc"];
var kontrahent2 = CRMModule.GetInstance(args.Session).Kontrahenci.WgKodu["Hiacynt"];
view.Condition &= new RowCondition.Not(new FieldCondition.In("Kontrahent", kontrahent1,
kontrahent2));
```

Like

Warunek ten umożliwia filtrowanie kolekcji za pomocą maski.

Konstruktor

```
/// path: Nazwa kolumny, po której chcemy filtrować
/// mask: Maska
public Like(string path, string mask) {}
```

Poniższy przykład pokazuje w jaki sposób załadować kolekcję zadań, których nazwa rozpoczyna się od słowa "Konferencja".

```
var view = ZadaniaModule.GetInstance(args.Session).Zadania.CreateView();
view.Condition &= new FieldCondition.Like("Nazwa", "Konferencja%");
```

Kolejny przykład pokazuje w jaki sposób załadować kolekcję kontrahentów, których trzecia litera kodu to "l".

```
var view = CRMModule.GetInstance(args.Session).Kontrahenci.CreateView();
view.Condition &= new FieldCondition.Like("Kod", "__l%");
```

Null

Warunek ten umożliwia zawężanie kolekcji do obiektów, w których wybrane pole jest lub nie jest ustawione.

Konstruktor

```
 /// path: Nazwa kolumny, po której chcemy filtrować
 /// isNull: true|false
public Null(string path, bool isNull) {}
```

Poniższy przykład prezentuje w jaki sposób załadować kolekcję zadań, które nie mają przypisanego kontrahenta.

```
var view = ZadaniaModule.GetInstance(args.Session).Zadania.CreateView();
view.Condition &= new FieldCondition.Null("Kontrahent", true);
```

Kolejny przykład pokazuje w jaki sposób załadować kolekcję zadań, w których **ustawiono** przedstawiciela.

```
 var view = ZadaniaModule.GetInstance(args.Session).Zadania.CreateView();
 view.Condition &= new FieldCondition.Null("Przedstawiciel", false);
```

Less

Warunek ten pozwala zawęzić kolekcję do obiektów, w których wartość wybranej kolumny jest **mniejsza** od danej wartości.

Greater

Warunek ten umożliwia zawężenie kolekcji do obiektów, w których wartość wybranego pola jest **większa** od danej wartości.

Warunki **LessEqual** oraz **GreaterEqual** zachowują się analogicznie do warunków Less i Greater, przy czym uwzględniane są również domknięcia (<= oraz >=).

Konstruktory (konstruktory Greater, LessEqual oraz GreaterEqual wyglądają analogicznie)

```
/// path: Nazwa kolumny, po któej chcemy filtrować
/// value: Wartość
public Less(string path, object value) {}

/// path: Nazwa kolumny, po któej chcemy filtrować
/// table2: Nazwa tabeli, w której znajduje się porównywany obiekt
/// path2: Nazwa kolumny, w której znajduje się porównywany obiekt
public Less(string path, ITable table2, string path2) {}
```

Pierwszy przykład pokazuje w jaki sposób załadować kolekcję projektów, których dopuszczalny koszt jest mniejszy od 10 000.

```
var view = zm.Projekty.CreateView()`;
view.Condition &= new FieldCondition.Less("Koszt", new Currency(10000.0));
```

Kolejny przykład pokazuje jak załadować kolekcję projektów, których oczekiwany przychód jest większy niż 100 000.

```
var view = zm.Projekty.CreateView();
view.Condition &= new FieldCondition.Greater("Przychod", new Currency(100000.0));
```

Ostatni przykład pokazuje w jaki sposób załadować kolekcję zadań, których daty mieszczą się w danym okresie.

```
var view = ZadaniaModule.GetInstance(args.Session).Zadania.CreateView();
view.Condition &= new FieldCondition.LessEqual("DataOd", pars.Okres.To);
view.Condition &= new FieldCondition.GreaterEqual("DataDo", pars.Okres.From);
```

Exists

Warunek ten umożliwia wykonanie podzapytania, które bada istnienie określonych danych w bazie. Podzapytanie faktycznie nie produkuje żadnych danych, a jedynie zwraca wartość true lub false.

Konstruktory

```
/// tableName: Nazwa tabeli wiążącej
/// pathTable: Nazwa pola w tabeli wiążącej
public Exists(string tableName, string pathTable) {}


/// tableName: Nazwa tabeli wiążącej
/// pathTable: Nazwa pola w tabeli wiążącej
/// condition: Dodatkowy warunek
public Exists(string tableName, string pathTable, RowCondition condition) {}

/// field: Nazwa pola w tabeli źródłowej - domyślnie "ID"
/// tableName: Nazwa tabeli wiążącej
/// pathTable: Nazwa pola w tabeli wiążącej
/// condition: Dodatkowy warunek
public Exists(string field, string tableName, string pathTable, RowCondition condition) {}
```

Załóżmy, że chcielibyśmy zwrócić kolekcję wszystkich zadań kontrahentów, którzy posiadają wystawione dokumenty na bieżący miesiąc.

```
var view = ZadaniaModule.GetInstance(args.Session).Zadania.CreateView();
view.Condition = new RowCondition.Exists("Kontrahent", "DokHandlowe", "Kontrahent",
                new FieldCondition.GreaterEqual("Data", Date.Today.FirstDayMonth()) &
                new FieldCondition.LessEqual("Data", Date.Today.LastDayMonth()));
```

ERP przetworzy powyższy kod na następujące zapytanie SQL.

```
select top 100 t0.[ID],t0.[Guid],t0.[Definicja],...,t0.[Stamp]
    from Zadania t0
where exists (select * from DokHandlowe t1 where t0.[Kontrahent]=t1.[Kontrahent] and t1.
[Data]>=CONVERT(datetime,'05/01/2015',101) and t1.[Data]<=CONVERT(datetime,'05/31/2015',101))
order by t0.[Kontrahent],t0.[DataOd],t0.[CzasOd],t0.ID
```

Narzędzia diagnostyczne

W skład grupy *Narzędzia diagnostyczne* wchodzą wszystkie artykuły zawierające wiedzę z diagnozowania przepływu kodu.

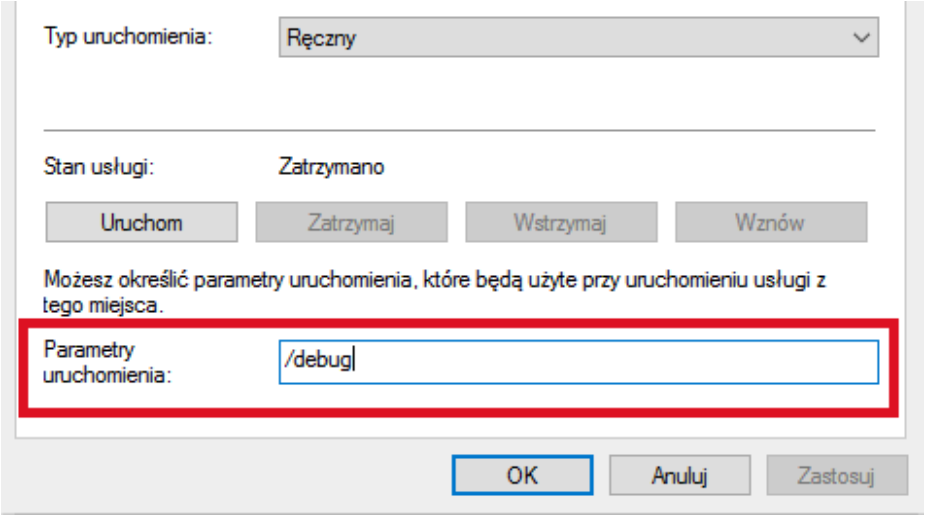
Debugowanie kodu kalkulatorów

Aby debugować kod kalkulatorów wymagany jest `Visual Studio`.

Włączenie sesji debugowanie jest realizowane przez uruchomienie aplikacji z przełącznikiem w wersji **.NET** jest to `--debug` :

- `SonetaExplorer32.exe /debug`
- `SonetaExplorer.exe /debug`
- `SonetaServer /console /debug`
- lub dodanie argumentu wywołania do serwisu:

UWAGA - należy zachować szczególną ostrożność przy wywoływaniu SonetaServer z opcją debug ponieważ w przypadku błędów lub pułapek zatrzymywane jest wykonywanie algorytmu i oczekiwanie na reakcję użytkownika



Uruchomienie serwisu SonetaServer do debugowania

W algorytmie należy także zaznaczyć w którym miejscu debuger ma się zatrzymać (oznaczyć pułapkę, breakpoint). Można tego dokonać poprzez metody klasy systemowej (wszystkie poniższe metody są statyczne):

- System.Diagnostics.Debugger.Launch()
- System.Diagnostics.Debugger.Break()
- System.Diagnostics.Debug.Assert() - parametr `condition` należy ustawić na wartość `false`

Powyższe metody należy używać w odpowiedni sposób aby debuger się uruchomił i zatrzymał na wskazanej linii. Z tego powodu powstała metoda pomocnicza, która wykorzystując powyższe metody zapewnia poprawne uruchomienie debugera. Dodatkowo metoda ta została opatrzona atrubutem [Conditional("DEBUG")] dzięki czemu nie trzeba jej usuwać z kodu kalkulatora po wyłączeniu sesji debugowania.

```
Soneta.Business.DebuggerSession.MarkLineAsBreakPoint();
```

Okienko Progress

W trakcie wykonywania długotrwałych operacji potrzebne jest wyświetlenie pewnych informacji na tzw okienku progress. Do wyświetlania tych informacji służy standardowy mechanizm **Trace** dostępny w .NET. Wszystkie informacje, które mają się pojawić w tym okienku należy wypisywać w kategorii `"Progress"`. W podobny sposób można sterować również wskaźnikiem postępu operacji, który informuje użytkownika, w którym momencie przetwarzania danych system się znajduje. Wtedy do `Trace` wysyłamy liczby typu `int` będące procentową wartością postępu (z zakresu 0 do 100) operacji.

Aby ułatwić użycie Trace, została przygotowana klasa TraceInfo. Przykładowy kod używa jej, przetwarza listę towarów i wyświetla w oknie progress odpowiednie informacje o stanie przetwarzania:

```
//
// Wyliczamy listę towarów, które zostaną przetworzone
//
List<Towar> lista = PoliczListęTowarów();
//
// Operację powtarzamy dla wszystkich towarów w liście
//
for (int i = 0; i < lista.Count; ++i) {
    //
    // W okienku progres wyświetlamy nazwę przetwarzanego towaru
    //
    TraceInfo.WriteProgress("Przetwarzany towar: " + lista[i]);
    //
    // W postaci wartości procentowej informujemy użytkownika o postępie przetwarzania danych
    //
    TraceInfo.SetProgressBar(100 * i / lista.Count);
    //
    // Oczywiście należy wykonać operację na danym towarze
    //
    PrzeliczTowar(lista[i]);
}
//
// Po zakończeniu przetwarzania można również ukryć informację o procentowym postępie obliczeń,
jeżeli dana procedura będzie wykonywała jeszcze inne zadania.
// Ukrywanie informacji nie jest koniczne.
//TraceInfo.HideProgressBar();
```

Logi systemowe

Wykonywanie pewnych operacji biznesowych może wymagać raportowania do systemu pewnych dodatkowych informacji, które po zakończeniu całej operacji mogą być wyświetlone użytkownikowi. Do zbierania takich informacji również można posłużyć się mechanizmem **Trace**. Żeby wysłać informację do takiego loga należy użyć jednej z metod `Write` klasy `Trace`. Można również używać kategorii logów, które powodują, że informacje będą zapisywane do różnych logów w

zależności od kategorii. Kategoria o nazwie `"Progress"` jest zarezerwowana do obsługi okna progresu *(patrz punkt wyżej)*.

Wypisanie informacji do loga nie powoduje jego automatycznego wyświetlenia. Użytkownik sam musi wyświetlić okienko logów. Jednak jeżeli chcemy, żeby okno loga zostało automatycznie wyświetlone należy do loga wysłać napis `"SHOWOUTPUT"`. Najlepiej użyć do tego metody `TraceInfo.ShowOutput(string categoria)`. Spowoduje to, że gdy będą wypisywane logi dla danej kategorii, to zostanie również otwarte okno.

```
//
// Zasygnalizowanie, że gdy pojawią się wpisy w logu ""Naliczanie danych"" mają być wyświetlone na
ekranie
//
TraceInfo.ShowOutput("Naliczanie danych");
//
// Utworzenie loga o nazwie "Naliczanie danych" oraz do niego pewnych informacji.
//
Trace.WriteLine("Rozpoczęto naliczanych danych", "Naliczanie danych");
for (int i = 0; i < 10; ++i)
    Trace.WriteLine("Linia loga #" + i, "Naliczanie danych");
//
// Utworzeniu pierwszego wpisu w logu "Błędy" spowoduje również otwarcie okna z logiem.
//
TraceInfo.WriteException("Nastąpił błąd przeliczania towarów");
```

Zapisywanie treści logów systemowych

`TraceCatcher` służy do lokalnego zbierania komunikatów diagnostycznych emitowanych przez `Trace(object)`, ale tylko dla wskazanych kategorii i tylko w aktywnym kontekście asynchronicznym. To narzędzie jest przydatne gdy istnieje potrzeba zapisania wszystkich komunikatów z konkretnego przebiegu operacji biznesowej.

Klasa współpracuje z `TraceCollector`, który jest globalnym `TraceListener`.
Przepływ jest następujący:

1. Tworzony jest `TraceCatcher` za pomocą metody statycznej `TraceCollector.GetCatcher(param string[] categories))`. Zostaje on zarejestrowany pod wybranymi kategoriami.
2. Gdy aplikacja wywołuje `Trace.Write*` z kategorią, `TraceCollector` przekazuje wiadomość do zarejestrowanych catcherów.
3. `TraceCatcher` zapisuje wiadomość do wewnętrznego bufora tylko wtedy, gdy:
    - jest aktywny w bieżącym AsyncLocal (state.Value == this),
    - opcjonalny filtr AcceptMessage zwróci True.

Zakres przechwytywania (ważne)

`TraceCatcher` przechwytuje wpisy wyłącznie w obrębie aktualnego przepływu async. To oznacza, że nie zbiera automatycznie wpisów z innych kontekstów asynchronicznych. Dzięki temu w instalacji serwerowej zostanie zebrany tylko log z bieżącej operacji użytkownika. Uwaga, w przypadku korzystania z `Concurrent` dla każdego wątku należy utworzyć oddzielny obiekt.

Odczyt

Odczytanie zapisanych treści za pomocą metody `ToString()` Jeśli nic nie zapisano, właściwość `IsEmpty` zwraca True, a `ToString()` zwraca pusty string. `GetLines()` zwraca komunikaty podzielone na linie.

Filtrowanie wiadomości

Pole `AcceptMessage (Func<string, bool>?)` pozwala odrzucać lub przepuszczać wpisy per komunikat. Przykładowe zastosowania:

- przepuszczanie tylko błędów,
- odrzucanie „szumu” diagnostycznego,
- ograniczenie do wpisów z określonym prefiksem.

Cykl życia i sprzątanie zasobów

`TraceCatcher` implementuje `IDisposable`. Wywołanie `Dispose()`:

- wyrejestrowuje go z `TraceCollector`,
- zapobiega dalszemu przechwytywaniu.

W praktyce instancję najlepiej używać go w bloku `using { }`.

Typowy scenariusz użycia

```
using System.Diagnostics;
using Soneta.Tools;

// 1. Utworzenie catchera dla wybranej kategorii
using (var catcher = TraceCollector.GetCatcher("ImportDokumentow")) {

    // 2. (Opcjonalnie) ustawienie filtra wiadomości
    catcher.AcceptMessage = message => !message.Contains("DEBUG",
StringComparison.OrdinalIgnoreCase);

    // 3. Wykonanie fragmentu kodu, który loguje przez Trace z tą samą kategorią
    Trace.WriteLine("Start importu", "ImportDokumentow");
    Trace.WriteLine("Przetworzono 120 pozycji", "ImportDokumentow");
    Trace.WriteLine("DEBUG: szczegóły diagnostyczne", "ImportDokumentow");
    Trace.WriteLine("Koniec importu", "ImportDokumentow");

    // 4a. Odczyt całości jako jeden tekst
    string log = catcher.ToString();

    // 4b. Odczyt linia po linii
    foreach (var line in catcher.GetLines()) {
        Console.WriteLine(line);
    }
    TraceInfo.ShowOutput("ImportDokumentow");
}
```

Bardzo ważna uwaga

Nie wolno rejestrować własnych `TraceListener` w kolekcji `Trace.Listeners`! Będzie to powodować wyjątek **InvalidOperationException kolekcja zmodyfikowana!**

Dokumentacja biblioteki testowej

Biblioteka FormTest

ActiveExplorer ActiveExplorer – ustawia instancję na aktywny eksplorer.

```
                [Test]
        public void WinForm_Grid_TableViewWorkers() {
                NavigateFolder("Handel/Zakup/Faktury zakupu");

                MouseClick("name:Grid");
                SendKeys("Zatwierdzony1");

                Thread.Sleep(1000); // Czekamy na lokator
                SendKeys("{Esc}");

                Invoke(() => {
                        Assert.IsFalse(ActiveExplorer.FindCommand("Zatwierdźdokument").Enabled);

Assert.IsTrue(ActiveExplorer.FindCommand("Przenieśdokumentydobufora").Enabled);
                });
        }
```

ActiveExplorerPage ActiveExplorerPage – ustawia instancję na aktywną stronę eksplorera.

```
[Test]
        public void WinForm_Grid_NonOptimal_Cancel_Sort() {
                ActiveExplorerPage.Context.Login.SetValue(typeof(GridStateKeeper),
"Handel/TowaryIUslugi", null);

                NavigateFolder("Handel/Towary i usługi");

                MouseClick("name:Grid:header:Workers.StanMagazynu.StanRazem");

                MouseClick("name:NonOptimalCancelButton");

                Assert.AreEqual("BIKINI", GetText("name:Grid:0:Kod"));
                Assert.AreEqual("BUT_NAR_42", GetText("name:Grid:1:Kod"));
        }
```

ActiveForm
ActiveForm – spośród wszystkich okien wybiera pierwsze (lub domyślne) okno. Zwraca typ Business Form, może korzystać z metod tam zawartych. Przykład:

```
        [Test]
        public void WinForm_Demo_Invoke() {
                NavigateFolder("Handel/Zakup/Faktury zakupu");

                Invoke(() => {
                        InvokeCommand("NewRow");
```

```
                });

                Invoke(() => {
                        ActiveForm.SelectedFieldName = "Kontrahent";
                });

                Invoke(() => {
                        ActiveForm.SelectedControl.Text = "ABC";
                });

                Invoke(() => {
                        var c = ActiveForm.FindControlByFieldName("Obcy.Numer");

                        Assert.IsFalse(c.IsSelected);
                        c.Select();

                        Assert.IsTrue(c.IsSelected);
                        Assert.AreEqual("Obcy.Numer", ActiveForm.SelectedControl.FieldName);
                });
```

ActiveContainer ActiveContainer – zwraca ActiveForm jeśli nie jest pusty, i w przeciwnym razie zwraca ActiveExplorer.

```
        [Test]
        public void WinForm_Grid_SelectedData() {
                NavigateFolder("Handel/Towary i usługi");

                var t = Session.GetTowary().Towary.WgNazwy.ToArray<Towar>();

                MouseClick("name:Grid:7:2");
                MouseClick("name:Grid:1:1", modifiers: MouseModifiers.Control);
                MouseClick("name:Grid:3:2", modifiers: MouseModifiers.Control);
                MouseClick("name:Grid:6:2", modifiers: MouseModifiers.Control);

                Invoke(() => {
                        var ids = ((Towar[])Grid.SelectedData).Select(towar => towar.ID);
                        AssertAssembler.AreArraysEqual(ids, t[1].ID, t[3].ID, t[6].ID, t[7].ID);

                        Assert.IsTrue(ActiveContainer.StatusLine.Contains(" 4 "));

                        MouseClick("name:Grid:3:2", modifiers: MouseModifiers.Control);
                });
        }
```

IsUIThread IsUIThread – informacja, czy aktywny jest wątek UI ##WaitForClose WaitForClose() – zawiesza wątek UI dopóki żądany formularz się nie zamknie. ##TestSetup TestSetup() – metoda konfigurująca test. Bazuje na metodzie określonej w klasie TestBase, z której dziedziczy. Jeśli użytkownik zakłóci test – porusza kursorem (userPointInterrupted), przerwie test z informacją "Test was interrupted by user mouse moving", Ustawia wątek na „Test”. Zawiesza wątek do momentu uruchomienia programu (startExplorer();). Ustawia kursor dokładnie w środku okna głównego. Jeśli program się nie uruchomił wywołuje metodę NavigateFolder(""); (?) ##TestTearDown TestTearDown() – zamyka test. Bazuje na metodzie określonej w klasie TestBase,

z której dziedziczy. Jeśli Test przeszedł pozytywnie, wszystkie okna powinny być zamknięte. Jeśli test failuje, metoda TearDown zamyka wszystkie otwarte okna. ##ClassSetup ClassSetup() - bazuje na metodzie określonej w klasie TestBase, z której dziedziczy. Uruchamia DatabaseUiInitializer inicjalizując Bazę Danych. ##ClassTearDown ClassTearDown() - bazuje na metodzie określonej w klasie TestBase, z której dziedziczy. Zamyka wszystkie otwarte okna. ##Sleep Sleep(int ms) – zatrzymuje wykonywanie testu (np. żeby poczekać na wczytanie się danych z bazy danych).

```
[Test]
    public void WinForm_GridFilter_LocatorInner() {
        NavigateFolder("Handel/Towary i usługi");

        MouseClick("name:Grid:0:0");

        SendKeys("*narciar");

        LogTimer.IsTracing = true;

        Sleep(1000);

        string sql = LogTimer.CalcSqlStat(removeStrings: false).Split('\n').First(l =>
l.Contains("from Towary t0")).Trim();

        bool _isUnicodeStrings = ((MsSqlDatabase)Login.Database).IsUnicodeStrings;

        Assert.AreEqual(
            String.Format("1: select top ? * from Towary t0 where (t0.[Kod] like {0}'%narciar%' or
t0.[Nazwa] like {0}'%narciar%' or t0.[EAN] like {0}'%narciar%' or t0.[PrefiksGTIN1?] like
{0}'%narciar%')
                and t0.[Typ]<>? and (t0.[Blokada] is null or t0.[Blokada]=?) order by t0.[Nazwa],t0.
[ID]", _isUnicodeStrings ? "N" : null),
            sql);

        Invoke(() => {
            Assert.AreEqual(8, GridView.DataRowCount);
            Assert.AreEqual(8, GridView.ListData.Count);
        });
    }
```

Invoke Invoke(WF.MethodInvoker action) – wywołuje metodę i czeka aż się wykona.

```
    [Test]
    public void WinForm_Grid_ExtSelColumn_UnselectAll()
    {
        NavigateFolder("Handel/Towary i usługi");

        MouseClick("name:Grid");

        GridView.SelectAll();
        SendKeys(" ");

        // CTRL + SHIFT + A
        SendKeys("+^{a}");
```

```
        Invoke(() => {
            Assert.IsFalse(GridView.IsExtSelection);
        });
    }
```

InvokeImmediately InvokeImmediately(WF.MethodInvoker action) – wywołuje metodę z argumentu. Sprawdza, czy uzytkownik, nie zakłócił testu poruszając myszką. Jeśli nie ma ustawionego mainForm, metoda się nie wywoła. Jeśli aktywny jest wątek UI, zostanie wywołana metoda z argumentu. W przeciwnym razie próbuje wykonać metodę w bloku try-catch. Jeśli żaden wyjątek nie jest złapany w bloku, (istnieje inna przyczyna, dla której nie można wywołać żądanej metody), wyrzucany jest fail w Assercie. ##DelayPlaying DelayPlaying(int ms) – opóźnia wykonywanie całego testu o podaną liczbę ms.

```
[Test]
        public void WinForm_Lookup_Enter_Empty() {

                NavigateFolder("Handel/Zakup/Faktury zakupu");
                InvokeCommand("NewRow");

                AssertCaption("Zakup: ZK/?/");

                MouseClick("Kontrahent");

        SendKeys("{ENTER}f");
        DelayPlaying(1000);

        SendKeys("%{F4}");

        SendKeys("{ENTER}");
        AssertCaption("Wybierz zapis");

        Invoke(() => {

            var grid = (Grid)ActiveContainer.FindControlByName("Grid").Control;

            Assert.AreEqual(4, grid.GridView.ListData.Count);

            var locatorText = (Control)ActiveContainer.FindControlByName("LocatorText").Control;

            Assert.AreEqual("f", locatorText.Text);
        });

        SendKeys("%{F4}");
        InvokeCommand("FormCancel");
        }
```

Wait Wait() – czeka liczbę ms określoną w DelayPlaying, w wątku UI lub Test. ##NavigateFolder NavigateFolder(string path) – nawigacja po folderach aplikacji, podając pełną ścieżkę, z separatorem „/”.

```
NavigateFolder("Handel/Zakup/Faktury zakupu");
NavigateFolder("Handel/Towary i usługi");
```

##NavigatePage NavigatePage(string name) – w okienku formularza, przechodzi do strony o nazwie ‘name’.

```
        [Test]
        public void WinForm_Grid_Focused_AfterOpenForm() {
                NavigateFolder("Kontrahenci i urzędy/Kontrahenci");

                MouseClick("name:Grid:1:Kod");
                Assert.AreEqual("Abc", GetText("name:Grid:focused:focused"));
                InvokeCommand("OpenRow");

                NavigatePage("Rozrachunki/Rozrachunki");



                MouseClick("New RozrachunkiPodmiotuExtender.Rozrachunki:1:1");

                var grid = (Grid)ActiveForm.FindControlByFieldName("New
RozrachunkiPodmiotuExtender.Rozrachunki").Control;

                Assert.AreEqual(1, grid.GridView.FocusedRowHandle);
        }
```

AssertCaption AssertCaption(string text) – zwraca tytuł okienka. Jeśli jest aktywny formularz z nazwą, zwraca ją, jeśli nie, zwraca nazwę aktywnego eksplorera.

```
    [Test]
    public void WinForm_Lookup_Click_Empty() {

        NavigateFolder("Handel/Zakup/Faktury zakupu");
        InvokeCommand("NewRow");

        AssertCaption("Zakup: ZK/?/");

        MouseClick("Kontrahent");
        SendKeys("{ENTER}f");
        DelayPlaying(1000);

        SendKeys("%{F4}");
         MouseClick("Kontrahent", x: -4);
        AssertCaption("Wybierz zapis");

        Invoke(() => {

            var grid = (Grid)ActiveContainer.FindControlByName("Grid").Control;
            Assert.AreEqual(4, grid.GridView.ListData.Count);
```

```
            var locatorText = (Control)ActiveContainer.FindControlByName("LocatorText").Control;

            Assert.AreEqual("f", locatorText.Text);
        });
        SendKeys("%{F4}");
        InvokeCommand("FormCancel");
    }
```

SendKeys SendKeys(string text) – wpisuje tekst do aktywnego obiektu. Sprawdza, czy na aplikacji jest fokus.

```
        [Test]
        public void WinForm_DataForm_SessionDispose_NewCancel() {
        int c = Login.Database.SavingCacheCounter;

                NavigateFolder("Handel/Zakup/Faktury zakupu");

                InvokeCommand("NewRow");

                MouseClick("Obcy.Numer");

                SendKeys("OBCY8");

        var session = ActiveForm.Context.Session;

        InvokeCommand("FormCancel");

        Assert.IsTrue(session.IsClosed);
        Assert.AreEqual(Login.Database.SavingCacheCounter, c);
        }
```

GetText GetText(string name) – zwraca tekst z obiektu o nazwie ‘name’. Przejmuje controlę na obiekt, jeśli obiekt jest nullem, zwraca jego wartość Text, w przeciwnym wypadku zwraca wartość GetText().

```
        [Test]
        public void WinForm_Control_Refresh_After_Resize() {
                NavigateFolder("Handel/Zakup/Faktury zakupu");

                InvokeCommand("NewRow");

                MouseClick("Obcy.Numer");

                SendKeys("OBCY8");

                Assert.AreEqual("OBCY8", GetText("Obcy.Numer"));

                Invoke(() => {
                        var form = (WF.Form)ActiveForm.Control;
                        form.Width += 5;
                });
```

```
                Assert.AreEqual("OBCY8", GetText("Obcy.Numer"));

                Invoke(() => {
                        var form = (WF.Form)ActiveForm.Control;
                        form.Width -= 5;
                });

                Assert.AreEqual("OBCY8", GetText("Obcy.Numer"));

                InvokeCommand("FormCancel");
        }
```

InvokeCommand InvokeCommand(string cmd, bool? checkOption = null) – wywołuje komendę podaną jako argument. Nazwy komend można znaleźć w trybie debug w soneta explorer (String po ‘cmd=...’). Korzystając z cmd, znajduje odpowiednią comand typu ActionComand. Wyrzuca wyjątek, jeśli nie ma takiej komendy. Jeśli comenda jest typu CheckCommand ( ).

```
        [Test]
        public void WinForm_Grid_Deleting() {
                NavigateFolder("Handel/Zakup/Faktury zakupu");

                MouseClick("name:Grid");

                SendKeys("wiele");
                Thread.Sleep(1000);

                InvokeCommand("OpenRow");
                Thread.Sleep(1000);

                MouseClick("new DokumentOgolneExtender.Pozycje:1:Towar");

                Invoke(() => {
                        Assert.AreEqual(4, GridPozycje.GridView.DataRowCount);
                        Assert.AreEqual(4, GridPozycje.GridView.ListData.Count);
                });

                InvokeCommand("DeleteRow");

                SendKeys(" "); // Potwierdzić kasowanie

                Invoke(() => {
                        Assert.AreEqual(3, GridPozycje.GridView.DataRowCount);
                        Assert.AreEqual(3, GridPozycje.GridView.ListData.Count);
                });

                InvokeCommand("FormCancel");
        }

InvokeCommand("DeleteRow");
InvokeCommand("FormCancel");
```

```
InvokeCommand("Renumeruj");
InvokeCommand("NewRow");
InvokeCommand("ShowOptions");
InvokeCommand("FormOK");
InvokeCommand("OpenRow");
```

InvokeCommand("FormCancel"); - specjalna komenda pozwalająca szybko i bez zapisywania i bez pytania zamknąć formularz.

Inny sposób zamknięcia:

```
InvokeCommand("FormClose");
AssertCaption("Ostrzeżenia i komunikaty");
MouseClick("text:Nie");
```

MouseMove MouseMove(int x, int y) – zmienia pozycję kursora na tą, zadaną w argumencie

MouseMoveClick MouseMoveClick(int x, int y) – zmienia pozycję kursora na tą, zadaną w argumencie i klika w to miejsce ##MouseClick MouseClick(string name, int x = int.MinValue, int y = int.MinValue,

MouseModifiers modifiers = MouseModifiers.None) – klika w obiekt o nazwie 'name'. Wylicza współrzędne obiektu i przekazuje do kliknięcia. Zwykle potrzebne do uzupełnienia formularza: najpierw MouseClick na pole, które chcemy uzupełnić, później SendKeys(); [Test] public void WinForm_DataForm_SessionDispose_NewSave() { int c = Login.Database.SavingCacheCounter;

```
        NavigateFolder("Handel/Zakup/Faktury zakupu");

        InvokeCommand("NewRow");

        MouseClick("Obcy.Numer");

        SendKeys("OBCY1234");

        MouseClick("Data");

        var session = ActiveForm.Context.Session;

        SendKeys("{Esc}");
        SendKeys("{Enter}");

        InTransaction(() => {
            var firma = Session.GetMagazyny().Magazyny.Firma;
            foreach (var dok in Session.GetHandel().DokHandlowe.WgMagazynuObcy[firma, "OBCY1234"])
                dok.Delete();
        });
        SaveDispose();

        Assert.IsTrue(session.IsClosed);
        Assert.AreEqual(Login.Database.SavingCacheCounter, c);
    }
```

MouseWheel MouseWheel – kręci scrollem myszki o zadaną liczbę kroków, w zadanym kierunku. [Test] public void WinForm_SortCommand() { NavigateFolder("Handel/Towary i usługi");

```
        MouseClick("name:Grid:13:2");
        string templateText = GetText("name:Grid:focused:Kod");

        //Scroll kółkiem myszy w dół
        MouseWheel(130, WheelRotation.Down);
        //Scroll kółkiem myszy w górę
        MouseWheel(100, WheelRotation.Up);

        InvokeCommand("GridSort");

        Assert.AreEqual(templateText, GetText("name:Grid:focused:Kod"));

    }
```

MouseRightClickMenu MouseRightClickMenu – prawy klawisz myszy (menu kontekstowe). Klika, a następnie wybiera zadaną pozycję z menu kontekstowego [Test] public void WinForm_Grid_Sum_SelectedEx() { NavigateFolder("Handel/Towary i usługi");

```
                InvokeCommand("GridSum");

                Sleep(300);

                MouseClick("name:Grid:5:Nazwa");

                SendKeys(" ");

                string footer = "czekaj";

                for (int i = 0; i<40 && footer.Contains("czekaj"); ++i) {
                        footer = GetText("name:Grid:footer:Ceny.Hurtowa.Netto");
                        Sleep(200);
                }

                Assert.IsFalse(footer.Contains("czekaj"));

                MouseRightClickMenu("name:Grid:footer:Nazwa", 5);

                Assert.AreEqual(Session.GetTowary().Towary.WgNazwy.Count.ToString(),
GetText("name:Grid:footer:Nazwa"));
        }
```

MouseClick("name:Grid:0:0"); MouseClick("name:Grid:0:0") – Nawigacja po gridzie. Klika na grid o nazwie ‘name’ (istnieją gridy o innych nazwach, np MouseClick("Params:new:0");). W miejsce 0:0 możemy podać współrzędne komórki grida, lub właśność, np. focused oznaczający zaznaczoną komórkę, lub nazwa kolumny jako druga współrzędna. [Test] public void WinForm_Grid_Locator_SecondGrid() { NavigateFolder("Handel/Sprzedaż/Faktury sprzedaży");

```
MouseClick("name:Grid");

SendKeys("^+i");

SendKeys("{Down}{Enter}");

MouseClick("name:_gridTowary:0:0");
Assert.AreEqual(GetText("name:_gridTowary:focused:Kod"), GetText("new
DokumentWizardTowaryExtender.Magazyny:0:Towar.Kod"));

MouseClick("name:_gridTowary:0:Workers.Szybka transakcja.Ilosc");
SendKeys("{Enter}");
Assert.AreEqual(GetText("name:_gridTowary:focused:Kod"), GetText("new
DokumentWizardTowaryExtender.Magazyny:0:Towar.Kod"));

MouseClick("name:_gridTowary:0:1");
MouseClick("name:_gridTowary:0:Kod");
SendKeys("^fkij{Enter}");

Assert.AreEqual(GetText("name:_gridTowary:focused:Kod"), GetText("new
DokumentWizardTowaryExtender.Magazyny:0:Towar.Kod"));

SendKeys("{Down}");
Assert.AreEqual(GetText("name:_gridTowary:focused:Kod"), GetText("new
DokumentWizardTowaryExtender.Magazyny:0:Towar.Kod"));

MouseClick("name:_gridTowary:0:0");
SendKeys("KOM");
Sleep(1000);
Assert.AreEqual(GetText("name:_gridTowary:focused:Kod"), GetText("new
DokumentWizardTowaryExtender.Magazyny:0:Towar.Kod"));

InvokeCommand("FormCancel");
}
```

SendKeys("{ }”) Symbole klawiszy SendKeys("{Down}{Enter}") – symbole klawiszy Klawisze o nazwach można wywołać przez nawiasy klamrowe: {Down}, {Enter},{Esc}, {Left}, {Right}, {F4}, {Tab} Równoczesne naciśnięcie uzyskujemy przez wpisanie klawiszy obok siebie: ("{Down}{Enter}"), "{Down}{Down}{Down}{Enter}" Klikukrotne naciśnięcie klawisza uzyskujemy przez wpisanie liczby obok nazwy: {Down 4} równoważne "{Down}{Down}{Down}{ Down }" Spacja nie ma symbolu, wpisujemy : SendKeys(" "); Znaki specjalne – klawisze Ctrl, Alt, Shift uzyskujemy przez symbole (bez {}): + =SHIFT ^ = CTRL % =ALT SendKeys("%{F3}"); SendKeys("+^{a}"); SendKeys("^{End}"); SendKeys("{Home}+^{End}"+note);

Biblioteka Business Form Do testów można używać wprost metod z biblioteki Business Form, działające na formularzach WindowsForm. ##SelectedFieldName SelectedFieldName – pole, get zwraca Name zaznaczonego pola, lub wyjątek, gdy jest ono nullem. Set wywołuje SelectPageByTypeOrName z argumentem formFieldName, czyli zaznacza pole formularza o podanym Name. Działa tak, jak kliknięcie na dane pole.

```
Invoke(() => {
ActiveForm.SelectedFieldName = "Kontrahent";
```

```
});
```

SelectedControl SelectedControl - zwraca obiekt typu BusinessControl, szukając po obiektach metodą FindFocus – zwraca kontrolkę, na której jest obecnie fokus – która jest aktywna. Może korzystać z metod zawartych w bibliotece BusinessControl.

```
Invoke(() => {
ActiveForm.SelectedControl.Text = "ABC";
        });
```

FindControlByFieldName FindControlByFieldName - zwraca obiekt typu BusinessControl, szukając po obiektach wg Name. Zwraca element typu BusinessControl, ale nie zmienia atrybutu tego elementu na IsSelected. Zaznacza ten obiekt metodą Select() z biblioteki BusinessControl Invoke(() => { var c = ActiveForm.FindControlByFieldName("Obcy.Numer"); Assert.IsFalse(c.IsSelected); c.Select();

```
                Assert.IsTrue(c.IsSelected);
                Assert.AreEqual("Obcy.Numer", ActiveForm.SelectedControl.FieldName);
                });
```

Przykładowe Testy

Opis testów z klasy DemoTest #WinForm_Demo_ConfigFile()

```
1       |               [Test]
2       |               public void WinForm_Demo_ConfigFile() {
3       |
4       |                       NavigateFolder("Kontrahenci i urzędy/Kontrahenci");
5       |
6       |                       //
7       |                       // Sprawdzamy, czy wczytała się konfiguracja listy kontrahentów z
ClassSetup
8       |
9       |                       // InitializeConfigFile
10      |                       // Wtedy są tylko dwie kolumny
11      |                       //
12      |
13      |                       MouseClick("name:Grid:0:0");
14      |
15      |                       Assert.AreEqual("!INCYDENTALNY",
GetText("name:Grid:focused:focused"));
16      |                       SendKeys("{Right}");
17      |                       Assert.AreEqual("!INCYDENTALNY",
GetText("name:Grid:focused:focused"));
18      |                       SendKeys("{Right}");
19      |                       Assert.AreEqual("Abc", GetText("name:Grid:focused:focused"));
20      |                       SendKeys("{Right}");
21      |                       Assert.AreEqual("ABC Sp. z o.o",
GetText("name:Grid:focused:focused"));
22      |
23      |               }
```

Test ma sprawdzić, czy prawidłowo został wczytany układ grida. Powinien zawierać tylko dwie kolumny. Klika myszką na pierwszą w lewym górnym rogu komórkę, przemieszcza się po gridzie strzałką w prawo i sprawdza czy ma odpowiednią zawartość. Po zakończeniu testu aplikacja zamyka się.

4 - Przechodzi do folderu Kontrahenci i urzędy/Kontrahenci.

13 - Klika na komórkę 0:0 (pierwszą w lewym górnym rogu) grida Kontrahenci.

15 - Sprawdza, czy Tekst zaznaczonej komórki (bo focused) to "!INCYDENTALNY".

16 - Przesuwa się w prawo (naciska klawisz strzałka w prawo).

17 - Sprawdza, czy Tekst zaznaczonej komórki (bo focused) to "!INCYDENTALNY".

18 - Przesuwa się w prawo (naciska klawisz strzałka w prawo) (ponieważ grid ma tylko dwie kolumny, przechodzi do drugiego wiersza – komórka 1:0).

19- Sprawdza, czy Tekst zaznaczonej komórki (bo focused) to "Abc".

20 - Przesuwa się w prawo (naciska klawisz strzałka w prawo).

21- Sprawdza, czy Tekst zaznaczonej komórki (bo focused) to " ABC Sp. z o.o ".

WinForm_Demo_Sample()

```
1       |               [Test]
2       |               public void WinForm_Demo_Sample() {
3       |                       NavigateFolder("Handel/Zakup/Faktury zakupu");
4       |
5       |                       InvokeCommand("NewRow");
6       |
7       |                       AssertCaption("Zakup: ZK/?/");
8       |
9       |                       MouseClick("Kontrahent");
10      |
11      |                       SendKeys("ABC");
12      |
13      |                       MouseClick("Obcy.Numer");
14      |
15      |                       Assert.AreEqual("ABC Sp. z o.o (Abc)", GetText("Kontrahent"));
16      |                       Assert.AreEqual(Date.Today.ToString(), GetText("Data"));
17      |
18      |                       SendKeys("obcy");
19      |
20      |                       MouseClick("Data");
21      |
22      |                       MouseClick("new DokumentOgolneExtender.Pozycje");
23      |
24      |                       SendKeys("bikini{ENTER}");
25      |
26      |                       MouseClick("Data");
27      |
28      |                       Date d = Date.Today-1;
29      |                       SendKeys($"{d.Year}-{d.Month}-{d.Day}{{ENTER}}");
30      |
31      |                       InvokeCommand("FormCancel");
32      |
33      |                       Assert.IsNull(ActiveForm);
34      |               }
```

Test tworzy nową fakturę zakupu i uzupełnia jej pola.

3 - Przechodzi do folderu Handel/Zakup/Faktury zakupu.

5 - Wywołuje metodę NewRow – nowy rekord na gridzie – czyli nową fakturę zakupu- Otwiera się formularz faktury- Efekt ten sam co kliknięcie przycisku Nowy (ZK – Zakup).

7 - Sprawdza, czy tytuł okienka to "Zakup: ZK/?/" (fokus ustawia się automatycznie na nowo otwarte okienko – formularz).

9 - Klika na pole ‘Kontrahent’.

11 - Wpisuje „ABC” (w aktywne pole – ‘Kontrahent’).

13 - Klika na pole Numer obcy (opisany przez Obcy-Numer).

15 - Sprawdza czy w polu ‘Kontrahent’ jest tekst "ABC Sp- z o-o (Abc)", (czyli, czy dobrze wpisano nazwę kontrahenta).

16 - Sprawdza czy w polu Data ustawiła się automatycznie dzisiejsza data.

18 - Wpisuje tekst „obcy” (do aktywnego pola Numer obcy).

20 - Klika na pole Data.

22 - Klika na pole zawierające grid z pozycjami faktury (domyślnie klika na sam środek pola), co daje efekt aktywacji pierwszego wiersza na tym gridzie.

24 - Wpisuje ‘bikini’ i zatwierdza Eneterem (do aktywnego pola Towar na gridzie).

26 - Klika na pole Data.

28 - Definiuje zmienną d, wczorajsza data.

29 - Wpisuje wczorajszą datę (ze zmiennej d) w formacie Rok-Miesiąc-Dzień i zatwierdza Enterem (w aktywne pole Data).

31 - Zamyka formularz bez zapisywania (i bez alertów).

33 - Sprawdza, czy nie ma aktywnych, otwartych formularzy.

WinForm_Demo_Invoke()

```
1       |               [Test]
2       |               public void WinForm_Demo_Invoke() {
3       |                       NavigateFolder("Handel/Zakup/Faktury zakupu");
4       |
5       |                       Invoke(() => {
6       |               InvokeCommand("NewRow");
7       |                       });
8       |
9       |                       Invoke(() => {
10      |               ActiveForm.SelectedFieldName = "Kontrahent";
11      |                       });
12      |
13      |                       Invoke(() => {
14      |               ActiveForm.SelectedControl.Text = "ABC";
15      |                       });
16      |
17      |                       Invoke(() => {
18      |               var c = ActiveForm.FindControlByFieldName("Obcy.Numer");
19      |
20      |               Assert.IsFalse(c.IsSelected);
21      |               c.Select();
22      |
23      |               Assert.IsTrue(c.IsSelected);
24      |               Assert.AreEqual("Obcy.Numer", ActiveForm.SelectedControl.FieldName);
25      |                       });
26      |
27      |                       SendKeys("obcy");
```

```
28      |
29      |                       Invoke(() => {
30      |               ActiveForm.SelectedFieldName = "Data";
31      |                       });
32      |
33      |                       MouseClick("new DokumentOgolneExtender.Pozycje:new:1");
34      |
35      |                       Sleep(3000);
36      |
37      |                       SendKeys("bikini{ENTER}");
38      |
39      |                       Invoke(() => {
40      |               var c = ActiveForm.FindControlByFieldName("Data");
41      |
42      |               c.Select();
43      |                       });
44      |
45      |                       InvokeCommand("FormClose");
46      |                       AssertCaption("Ostrzeżenia i komunikaty");
47      |
48      |                       MouseClick("text:Nie");
49      |               }
```

3 - Przechodzi do folderu Handel/Zakup/Faktury zakupu.

5 - Wywołuje komendę NewRow – nowy rekord – otwiera formularz z nową fakturą zakupu.

9 - Ustawia pole Kontrahent jak wybrany (selected) element.

13 - Zmienia tekst w zaznaczonym polu na „ABC” (nie zapisuje tekstu po zmianie pola).

17 - Znajduje obiekt o Name Obcy.Numer – tj numer dokumentu.

20 - Sprawdza, że element Obcy.Numer, nie jest wybrany (selected).

21 - Wybiera element Obcy.Numer (ten sam efekt co przez kliknięcie na niego).

23 - Sprawdza, że element Obcy.Numer, jest wybrany (selected).

24 - Sprawdza, że element, który jest zaznaczony ma name Obcy.Numer.

27 - Wpisuje tekst „obcy” (w aktywne pole Obcy.Numer).

29 - Zaznacza pole Data.

33 - Klika na pierwszą pozycję grida z pozycjami faktury, dokładnie na wiersz o nazwie „new” i pierwsza kolumna.

35 - Czeka 3 sekundy, tak, żeby możliwe było wpisywanie tekstu w polu.

37 - Wpisuje tekst BIKINI w aktywne pole.

39 - Znajduje pole Data.

42 - Zaznacza pole Data.

45 - Zamyka formularz.

46 - Sprawdza czy okno, które się pojawiło ma tytuł „Ostrzezenia i komunikaty”.

48 - Klika ‘Nie’ na ostrzeżeniu.

WinForm_Demo_Lista()

```
1       |                [Test]
2       |               public void WinForm_Demo_Lista() {
3       |                       NavigateFolder("Handel/Towary i usługi");
4       |
5       |                       MouseClick("name:Grid:0:0");
6       |
7       |                       Assert.AreEqual("Towary Typ<>Receptura and Blokada=False",
GetText("name:Grid"));
8       |
9       |                       Assert.AreEqual("BIKINI", GetText("name:Grid:0:Kod"));
10      |                       Assert.AreEqual("200,00 PLN",
GetText("name:Grid:9:Ceny.Podstawowa.Netto"));
11      |
12      |                       SendKeys("^{End}");
13      |                       SendKeys("^{Home}");
14      |
15      |                       SendKeys("{PgDn}");
16      |                       SendKeys("{PgDn}");
17      |                       SendKeys("{PgDn}");
18      |                       SendKeys("{PgDn}");
19      |
20      |                       SendKeys("^{Home}");
21      |
22      |                       MouseClick("name:Grid:1:Kod");
23      |                       MouseClick("name:Grid:10:Nazwa");
24      |                       MouseClick("name:Grid:13:Kod");
25      |                       MouseClick("name:Grid:4:Workers.StanMagazynu.StanZamówień");
26      |                       MouseClick("name:Grid:2:Kod");
27      |
28      |                       MouseClick("name:Grid:0:Nazwa");
29      |                       MouseClick("name:Grid:0:Kod");
30      |                       MouseClick("name:Grid:0:Workers.StanMagazynu.StanZamówień");
31      |
32      |                       MouseClick("name:Grid:0:0");
33      |                       MouseClick("name:Grid:0:1");
34      |                       MouseClick("name:Grid:0:2");
35      |                       MouseClick("name:Grid:0:3");
36      |                       MouseClick("name:Grid:0:4");
37      |
38      |                       MouseClick("name:Grid:focused:-1");
39      |                       MouseClick("name:Grid:focused:-1");
40      |                       MouseClick("name:Grid:focused:-1");
41      |
42      |                       MouseClick("name:Grid:focused:+1");
43      |                       MouseClick("name:Grid:focused:+1");
44      |                       MouseClick("name:Grid:focused:+1");
45      |
```

```
46      |                       MouseClick("name:Grid:0:focused");
47      |                       MouseClick("name:Grid:1:focused");
48      |                       MouseClick("name:Grid:2:focused");
49      |
50      |                       MouseClick("name:Grid:focused:Kod");
51      |                       MouseClick("name:Grid:focused:EAN");
52      |                       MouseClick("name:Grid:focused:Workers.StanMagazynu.StanZamówień");
53      |
54      |                       MouseClick("name:Grid:header:Kod");
55      |                       MouseClick("name:Grid:header:Nazwa");
56      |                       MouseClick("name:Grid:header:Kod");
57      |                       MouseClick("name:Grid:header:Kod");
58      |
59      |                       MouseClick("name:Grid:+1:focused");
60      |                       MouseClick("name:Grid:+1:focused");
61      |                       MouseClick("name:Grid:+1:focused");
62      |                       MouseClick("name:Grid:+1:focused");
63      |
64      |                       MouseClick("name:Grid:-1:Nazwa");
65      |                       MouseClick("name:Grid:-1:Nazwa");
66      |                       MouseClick("name:Grid:-1:Nazwa");
67      |                       MouseClick("name:Grid:-1:Nazwa");
68      |               }
```

3- Przechodzi do tabeli Towary i usługi

5- Klika na grid na pierwszą komórkę (o współrzędnych 0:0)

7- Sprawdza, czy grid jest typu "Towary Typ<>Receptura and Blokada=False",

WinForm_Demo_Lista_Grupy()

```
1       |               [Test]
2       |               public void WinForm_Demo_Lista_Grupy() {
3       |                       NavigateFolder("Handel/Towary i usługi");
4       |                           Sleep(2000);
5       |                           if (GetText("name:GroupsTree") == null)
6       |                               InvokeCommand("GridGroupByFeatures");
7       |
8       |                           MouseClick("name:GroupsTree:Producent");
9       |
10      |                           MouseClick("name:GroupsTree:Tyrolia");
11      |
12      |                           //
13      |                           // Można również włączyć zaznaczanie wielu grup na raz
14      |                           //
15      |                           InvokeCommand("FeaturesTreeMultiCheck");
16      |
17      |                           //
18      |                           // A następnie klikając z lewej strony elementów można zaznaczać
poszczególne grupy
19      |                           //
20      |
```

```
21      |                           MouseClick("name:GroupsTree:Tyrolia", 4);
22      |
23      |                           MouseClick("name:GroupsTree:Salomon", 4);
24      |                }
```

##WinForm_Demo_Zakladki()

```
1       |                 [Test]
2       |                public void WinForm_Demo_Zakladki() {
3       |
4       |                        NavigateFolder("Handel/Towary i usługi");
5       |
6       |                        MouseClick("name:Grid");
7       |
8       |                        SendKeys("BIKINI");
9       |
10      |                        InvokeCommand("OpenRow");
11      |
12      |                        NavigatePage("TowarJednostki");
13      |
14      |                        NavigatePage("Magazyn/Obroty");
15      |
16      |                        MouseClick("name:PagesTree:Zamówienia");
17      |
18      |                        MouseClick("name:PagesTree:Magazyn");
19      |
20      |                        MouseClick("name:PagesTree:Obroty");
21      |
22      |                        InvokeCommand("FormCancel");
23      |
24      |                        NavigateFolder("Handel/Zakup/Faktury zakupu");
25      |
26      |                        InvokeCommand("NewRow");
27      |
28      |                        MouseClick("name:PagesTabs:Marża");
29      |
30      |                        InvokeCommand("FormClose");
31      |
32      |                        MouseClick("text:Nie");
33      |                }
```

4 - Przejdź do "Handel/Towary i usługi".

6 - Kliknij na gdrid.

8 - Wpisuje ‘bikini’ – wpisuje się w lokator, pozostawiając na liście tylko towar Bikini.

10 - Wywołuje komendę OpenRow – otwiera zaznaczony rekord (ten sam efekt co kliknięcie przycisku otwórz, lub dwukrotne kliknięcie na nazwę towaru.

12 - Przechodzi do zakładki Jednostki (przez odwołanie do nazwy zakładki).

14 - Przechodzi do zakładki Obroty w Magazyn (przez odwołanie do ścieżki zakładki).

16 - Przechodzi do zakładki Zamówienia (przez odwołanie do typu elementu – PagesTree i nazwy zakładki).

18 - Rozwija folder zakładki Magazyn (przez odwołanie do typu elementu – PagesTree i nazwy zakładki).

20 - Przechodzi do zakładki Obroty (przez odwołanie do typu elementu – PagesTree i nazwy zakładki).

22 - Zamyka formularz bez zapisywania.

24 - Przechodzi do faktur zakupu.

26 - Wywołuje komendę NewRow – nowy rekord, otwiera się formularz faktury.

28 - Klika na zakładkę typu PagesTab o nazwie Marża.

30 - Zamyka formularz (FormClose powoduje wyświetlenie alertu Czy zapisać zmiany).

32 - Klika na element o tekście Nie.

Język *pseudo JavaScript*

Do programowania skryptów sterujących testami posługujemy się językiem zbliżonym do **JavaScript**.

Mogą być używane następujące konstrukcje JS

- Zmienne **var**

```
var zmienna = 1234;
```

- Komentarze **//**

```
// Po dwóch znakach slash tekst tekst jest ignorowany
```

- Pętla **while**

```
while (zmienna > 10000) {
        // Jakikolwiek kod
}
```

- Pętla **for**

```
for (var i = 0; i < 10; ++i) {
        // Jakikolwiek kod
}
```

- Warunki **if**

```
if (zmienna > 10000) {
        // Jakikolwiek kod
}

if (zmienna > 10000) {
        // Jakikolwiek kod
}
else {
        // Jakikolwiek kod
}
```

Wyszukiwanie pól

Do znajdowania różnych elementów UI na stronie HTML możemy używać wyrażeń podawanych jako napisy w języku **JavaScript**.

- **"name"** - wyszukuje element o ID (z serwera) zwracane przez `test.listElements()`

- **"name*"** - wyszukuje element o ID (z serwera) rozpoczynającym się podaną nazwą zwracane przez `test.listElements()`
- **"text:caption"** - wyszukuje element z podanym caption, w całości lub części
- **"name:row:col"** - wyszukuje komórkę w gridzie gdzie:
  - **row = new** - wiersz dodawania in place, wtedy nie podajemy kolumny: **"name:new"**
  - **row = header** - nagłówek listy
  - **row = filter** - wiersz filtrujący
  - **row = focused** - wiersz, który jest aktualnie focused
  - **row = +liczba** - n-ty kolejny wiersz w stosunku do focused: **name:+1:Nazwa**
  - **row = -liczba** - n-ty poprzedni wiersz w stosunku do focused: **name:-1:Nazwa**
  - **row = liczba** - n-ty wiersz na liście, numerowane od 0, pierwszy wiersz to: **name:0:Nazwa**
  - **col = name** - kolumna o ID (z serwera) zwracane przez `test.listElements()`
  - **col = name** * - kolumna o ID (z serwera) rozpoczynająca się podaną nazwą zwracane przez `test.listElements()`
  - **col = text** - kolumna o podanym caption, w całości lub części
  - **col = liczba** - n-ta kolumna na liście, gdzie 0 to kolumna zaznaczania
  - **"name:lookup"** - element w rozwiniętym lookup-ie (combo), gdzie lookup określa tytuł pozycji w całości lub w części

Klasa Assert

- `failed(msg: string = null): void;`
- `areEqual(expected: any, value: any, msg: string = null): void;`
- `areNotEqual(expected: any, value: any, msg: string = null): void;`
- `greater(valueL: any, valueR: any, msg: string = null): void;`
- `greaterOrEqual(valueL: any, valueR: any, msg: string = null): void;`
- `less(valueL: any, valueR: any, msg: string = null): void;`
- `lessOrEqual(valueL: any, valueR: any, msg: string = null): void;`
- `isTrue(condition: any, msg: string = null): void;`
- `isFalse(condition: any, msg: string = null): void;`
- `isNull(condition: any, msg: string = null): void;`
- `isNotNull(condition: any, msg: string = null): void;`
- `isUndefined(condition: any, msg: string = null): void;`
- `isNotUndefined(condition: any, msg: string = null): void;`
- `contains(expected: string, value: string, msg: string = null): void;`
- `doesNotContain(expected: string, value: string, msg: string = null): void;`
- `noRequest(): void;`
- `request(name: string, action: string, editValue?: string, id?: number): void;`
- `requests(list: name: string, action: string, editValue?: string, id?: number }[]): void;`

Klasa Test

- `play(name: string): void;`
- `sleep(ms: number);`
- `quit();`
- `get Window(): Services.WindowElement;`
- `listElements(showHidden?: boolean): void;`
- `listTests(name?: string): void;`
- `showSteps(): void;`
- `transaction(): void;`
- `rollback(): void;`
- `findElement(name: string): Services.DataElement;`
- `findElementByText(text: string): Services.DataElement;`
- `getRt(name: string): E: HTMLElement, X: number, Y: number, W: number, H: number };`
- `highlight(name: string): void;`

- `getText(name: string): string;`
- `sendKeys(text: string): void;`
- `click(name: string, dx?: number, dy?: number): void;`
- `clickRight(name: string, dx?: number, dy?: number): void;`
- `move(name: string, dx?: number, dy?: number): void;`
- `moveBy(dx: number, dy: number): void;`
- `post(name: string, action: string, editValue?: string): void;`
- `close(): void;`
- `navigateFolder(folder: string): void;`
- `navigatePage(page: string): void;`

Uruchomienie przeglądarki z SonetaServer

Logowanie przez panel (baza Demo)

- `/console /noscheduler /explorersettings /launch:\"http://localhost:1080/Login/Demo\`

Logowanie bezpośrednio jako Administrator (baza Demo)

- `"/console /noscheduler /explorersettings /launch:\"http://localhost:1080/Login/Demo?user=Administrator&pwd=\`

Logowanie bezpośrednio jako Administrator do nowego UI (baza Demo)

- `"/console /noscheduler /explorersettings /launch:\"http://localhost:1080/Login/Demo?user=Administrator&pwd=&deviceType=BrowserExp\""`

  deviceType=Browser - stary UI
  deviceType=BrowserExp - nowy UI
  deviceType=Mobile - wersja mobilna

Logowanie bezpośrednio jako Administrator z automatycznym uruchomieniem testu (przykładowy test Nawigacja.js w katalogu Scripter/samples)

- `"/console /noscheduler /explorersettings /launch:\"http://localhost:1080/Login/Demo?user=Administrator&pwd=&play=samples/Nawigacja.js\""`

Logowanie bezpośrednio jako Administrator z automatycznym uruchomieniem wszystkich testów z wskazanego katalogu (samples/testyHTML)

- `"/console /noscheduler /explorersettings /launch:\"http://localhost:1080/Login/Demo?user=Administrator&pwd=&play=samples/testyHTML/""`

Gwiazdka w nazwach testów (plików JS)

*kategoriaTestu_lista.js** - *wywoła wszystkie skrypty zaczynające się na kategoriaTestu* i kończące się na _lista.js

*kategoriaTestu_** - wywoła wszystkie skrypty z określonej kategorii testów

Pozostało...

```
//
// Natychmiastowe zamknięcie testu
//
Test.quit();
```

```
//
// UWAGA: We wszystkich metodach Assert argument msg nie jest konieczny
//
Assert.areEqual(14, 3 * 4 + 2);
Assert.isTrue(10<14);

//
// Zgłoszenie asercji do loga
// Assert.failed(msg);
//
Assert.failed("Ten komunikat jest zawsze logowany");

//
// Weryfikacja na wartość (value), warunek poprawności value===expected
// Assert.areEqual(expected, value, msg);
//
Assert.areEqual(14, 3 * 4 + 2, "Coś nie tak z tą matematyką");

//
// Weryfikacja na inną wartość (value), warunek poprawności value!==expected
// Assert.areNotEqual(expected, value, msg);
//
Assert.areNotEqual("Ma nie być to", "Więc nie jest", "Coś nie tak z tą matematyką");

//
// Weryfikacja spełnionego warunku (condition), warunek poprawności !!condition
// Assert.isTrue(condition, msg);
//
Assert.isTrue(10<14, "Ten warunek jest prawdziwy");

//
// Weryfikacja niespełnionego warunku (condition), warunek poprawności !condition
// Assert.isFalse(condition, msg);
//
Assert.isFalse(10 < -10, "Ten warunek jest fałszywy");

//
// Weryfikacja wartości null, warunek poprawności expression===null
// Assert.isNull(expression, msg);
//
Assert.isNull(null, "To jest wartość null");

//
// Weryfikacja wartości innej niż null, warunek poprawności expression!==null
// Assert.isNotNull(expression, msg);
//
Assert.isNotNull(123, "To na pewno nie jest null");

//
// Weryfikacja wartości Undefined, warunek poprawności typeof(expression)=="undefined"
// Assert.isUndefined(expression, msg);
//
Assert.isUndefined(xyz, "To jest wartość niezdefiniowana");
```

```
//
// Weryfikacja wartości innej niż Undefined, warunek poprawności typeof(expression)!=="undefined"
// Assert.isNotUndefined(expression, msg);
//
Assert.isNotUndefined(123, "Ta wartość nie jest niezdefiniowana");


//
// Przeniesienie do wskazanego folderu
//
Test.navigateFolder("Handel/Kartoteki/TowaryIUslugi");

Test.play("Test");

Test.listElements();

Test.findElement("List");

Test.highlight("List");

Test.click("List", dx, dy);

Test.sleep(1000);
```

Inicjalizacja programu i modułu oraz klasy Schema

API Soneta umożliwia rozszerzenie logiki działania systemu poprzez mechanizmy takie jak dodatki, taski i cechy. W artykule zostaną przedstawione sposoby jak z poziomu dodatku zaimplementować sprzężenie własnej logiki z systemem.

Przedstawimy w artykule sposoby dodania własnej logiki, nie poprzez workery powszechnie wywoływane z poziomu UI, tylko na poziomie startu programu lub inicjalizacji modułu wnoszonego przez dodatek.

Artykuł opisuje zastosowanie atrybutu *ProgramInitializer* oraz implementację metody *Initialize* posiadaną przez każdy dodawany moduł. Kolejnym poruszonym tematem jest generowanie po stronie *business.cs*, odpowiednich dla każdego obiektu biznesowego, klas Schema. Zastosowanie klas Schema posłuży jako dodanie przykładowej logiki do systemu w momencie uruchomienia programu lub w momencie inicjalizacji własnego modułu.

Uwaga!

Do artykułu jest dołączony projekt dodatku prezentujący przykładową implementację.

Schema

API umożliwia rozszerzenie logiki działania w przypadku operacji na obiektach biznesowych. Oznacza to, że dla przykładowej operacji dodania obiektu biznesowego Pracownik można zadać dodatkowe działanie systemu. Operacje rozszerzające logikę działania to: *dodawanie, usuwanie i modyfikacja* obiektu biznesowego, a także wyliczanie praw dostępu do obiektu oraz wyliczanie czy obiekt jest w trybie *ReadOnly*.

Własną logikę w przypadku modyfikacji obiektu można dodać na poziomie edycji konkretnej właściwości obiektu, np. do właściwości **Imię** obiektu **Pracownik** można dodać logikę działania w momencie rozpoczęcia i zakończenia edycji właściwości. Dodając własną logikę związaną z obiektem biznesowym należy posłużyć się obiektami Schema.

Na podstawie zdefiniowanej struktury w plikach *business.xml* generowane są pliki modułów *business.cs*, które zawierają m.in. statyczne klasy Schema odpowiednie dla każdego obiektu biznesowego. Klasy Schema posiadają szereg publicznych metod umożliwiających dodanie logiki do konkretnej operacji. Logikę można dodawać zarówno do bazowych obiektów biznesowych jak **Pracownik**, *DokumentHandlowy* oraz obiektów wnoszonych przez dodatek.

Metody klasy Schema jako parametr przyjmują delegat. W API istnieją typy generycznych delegatów, które należy zastosować w implementacji, np. metody w klasie **PracownikSchema** jako parametr przyjmują obiekt typu `RowDelegate<PracownikRow>`, który jest zdefiniowany w taki sposób:

```
public delegate void RowDelegate<TRow>(TRow row);
```

Tak zdefiniowany delegat, w przypadku metod z klasy *PracownikSchema*, powinien referować do metody, gdzie parametrem jest obiekt typu **PracownikRow**.

Pozostałe definicje delegatów wykorzystywane przez metody klas Schema:

`public delegate void RowAccessRightsDelegate<TRow>(TRow row, ref AccessRights accessRights);` wykorzystywany przy rozszerzeniu logiki wyliczania praw dostępu do danego typu obiektu biznesowego.

`public delegate void RowReadOnlyDelegate<TRow>(TRow row, ref bool readOnly);` wykorzystywany przy rozszerzeniu logiki wyliczania trybu *ReadOnly* danego typu obiektu biznesowego.

Atrybut ProgramInitializer

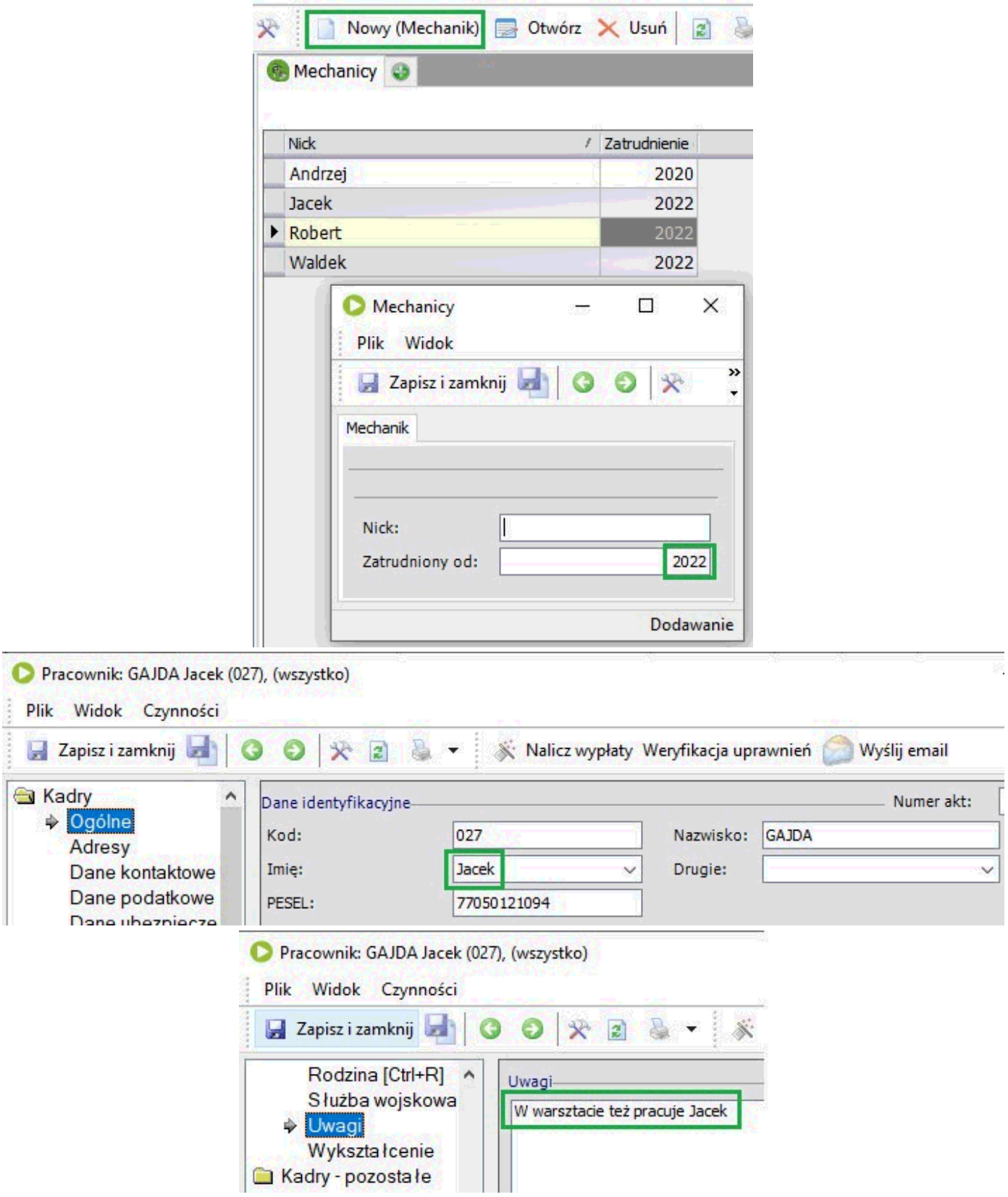
W celu dodania własnej logiki w momencie uruchomienia programu należy posłużyć się atrybutem *ProgramInitializer*. Rejestrowany jest na poziomie *assembly* a jako parametr przyjmuje typ klasy implementującej interface *IProgramInitializer*. Interface obliguje klasę do implementacji metody *IProgramInitializer.Initialize()*, w której zamieszcza się własną logikę. Może być ona oparta o opisane mechanizmy Schema. Zintegrowanie takiej logiki z systemem następuje w momencie uruchomienia programu, gdy operator nie jest jeszcze zalogowany.

Przykład

W załączonym projekcie przykładową logiką dodaną w momencie inicjalizacji modułu jest dodanie sufiksu do kodu kontrahenta gdy następuje zmiana kodu. W przykładzie został użyty mechanizm Schema prezentujący użycie delegatu odpowiedniego dla obsługi zdarzeń typu **BeforeEdit**.

```
namespace ProgramInitializerDemo.Workshop
{
    public partial class WorkshopModule
    {
        protected override void Initialize()
        {
            var rowDelegate = new RowDelegate<CRMModule.KontrahentRow, string>(AddSufixToClientCode);
            CRMModule.KontrahentSchema.AddKodBeforeEdit(rowDelegate);
        }

        private void AddSufixToClientCode(CRMModule.KontrahentRow row, ref string code)
        {
            string sufix = " (*)";
            if (!code.EndsWith(sufix))
                code += sufix;
        }
    }
}
```



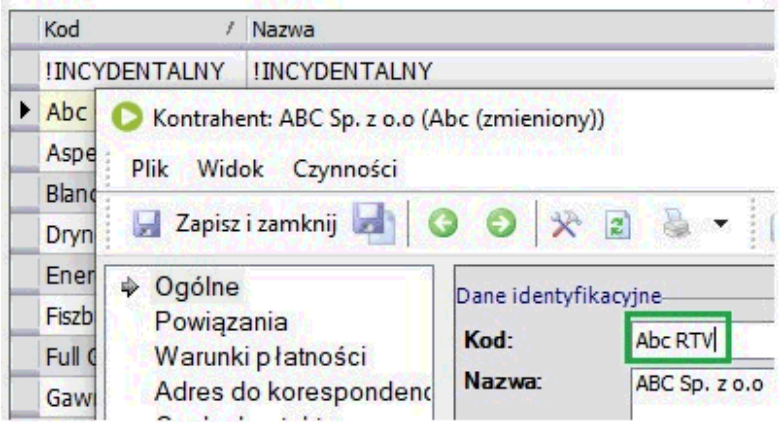
Utworzenie rekordów z poziomu UI

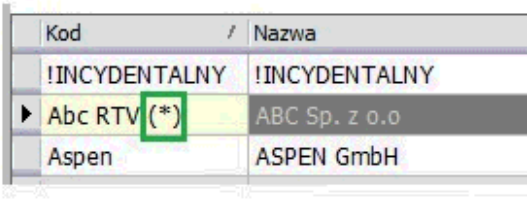
Inicjalizacja modułu

Przy wykorzystaniu API możliwe jest wpięcie dodatkowej logiki do działania systemu w momencie inicjalizacji własnego modułu. Inicjalizacja modułów następuje podczas logowania operatora oraz zawsze gdy tworzona jest instancja modułu, np. w danej sesji po raz pierwszy zostanie wywołane *session.GetAddon()*.

W celu zaimplementowania rozwiązania należy przesłonić metodę *Initialize()* w klasie własnego modułu, a w ciele metody dodać logikę, np. opartą o mechanizmy klas Schema. Klasa modułu

wygenerowana jest w pliku *business.cs* i nie należy jej modyfikować. Aby dodać implementację do klasy modułu należy dodać w projekcie osobny plik, a w nim implementację przy wykorzystaniu **partial class**. Należy zwrócić uwagę na to by *namespace* w jakim jest **partial class** był taki sam jak w wygenerowanym pliku *business.cs*.





Prezentacja rekordu w UI

Statyczne konstruktory modułów

W własnych klasach modułów można zaimplementować (dodanie pliku *partial class*) konstruktor statyczny, a w nim zamieścić dodatkową logikę. Z powodu charakterystyki konstruktorów statycznych takie **rozwiązanie jest niepoprawne** i może prowadzić do błędów.

W technologii .NET konstruktor statyczny zostaje wywołany tylko jeden raz w czasie działania programu. Następuje w momencie, gdy w trakcie działania programu po raz pierwszy tworzona jest instancja danego typu (klasy, która posiada statyczny konstruktor) lub po raz pierwszy wywołana jest metoda statyczna takiej klasy. Podczas działania programu takie operacje mogą być wykonywane wiele razy, więc nie ma możliwości określenia momentu wywołania konstruktora statycznego. Umieszczenie logiki w konstruktorze statycznym może prowadzić do błędu. Logika może zakładać, że inne obiekty zostały już zainicjalizowane co wcale nie musiało nastąpić. Przykładem błędu jest umieszczenie w konstruktorze statycznym modułu odwołania do klasy *ApplicationInfo*. Nastąpi wtedy takie działanie programu:

- start aplikacji, wczytywanie plików dll dodatku (tylko wczytanie dll, żaden kod dodatku się jeszcze nie wykonuje)
- budowa listy dostępnych operatorów odbywa się w taki sposób, że do bazy danych wysyłany jest request przy użyciu tzw. **internal login** (wewnętrzny login na potrzeby aplikacji).
- login tworzy sesję na potrzeby zapytania bazy o dostępnych operatorów
- konstruktor sesji tworzy kontener dostępnych tabel, do czego potrzebne jest ApplicationInfo
- tworzy się instancja ApplicationInfo
- .NET tworzy potrzebne klasy z dodatku, uruchamiając (poza kontrolą aplikacji ERP) statyczne konstruktory
- statyczny konstruktor odwołuje się do ApplicationInfo, które właśnie jest w stanie tworzenia (na stosie działania programu jest tworzenie instancji ApplicationInfo)
- powstaje pewnego rodzaju deadlock, który może zostać rozwiązany w nieoczekiwany sposób lub prowadzić do błędu programu

Pliki do pobrania

ProgramInitializerDemoApp.zip (171.09 KB)

Soneta sp. z o.o.
ul. Wadowicka 8A, 30-415 Kraków
+48 12 349 28 00
kontakt@enova.pl